

AI工程师技术栈全景图

+10个层次 · 63个核心概念 · 从Token表示到生产安全的完整生命周期



Coggle数据科学

微信扫描二维码，关注我的公众号



大模型世界中的 Token

我是 Token!

Token 到底是什么?

在大模型（如 GPT、Llama、Qwen 等）中，Token 是语言被切分后的**最小单位**。模型不直接处理“字”或“词”，而是处理 Token 序列。

可以把它理解成：

把一段话切成一块一块的小积木，模型就是通过这些“积木”来理解和生成内容的。



2 Token 长什么样?

Token 不是固定的“字”或“词”，而是由分词器（Tokenizer）根据统计规律将文本切分成的片段，长度不固定。

中文示例： “人工智能”	模型A（字级）	人	工	智	能
	模型B（词级）	人工	智能		
	模型C（子词级 BPE）	人	工	智能	
	模型D（子词级 BPE）	人	工	智能	
英文示例： “unbelievable”	模型B（词级）	unbelievable			
	模型C（子词级 BPE）	un	bel	iev	able
	模型D（子词级 BPE）	un	bel	iev	able
	模型D（子词级 BPE）	un	bel	iev	able

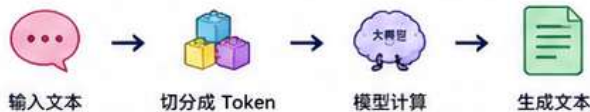
💡 目前主流大模型（如 GPT 系列）使用的是子词级 Token（如 BPE、SentencePiece）能更好地平衡词汇表大小和切分灵活性。

4 总结



Token 就是大模型理解和生成世界的“积木”。无论是输入你的问题，还是输出答案，模型眼中都是一串串 Token。

一句话总结：



小贴士

想加通一段话有多少 Token？可以使用工具：tiktoken / transformers / sentencepiece 等！

1 Token 是如何工作的?

1 输入文本

今天天气真好，
适合去公园散步。



2 分词 (Tokenization)

文本被切分成 Token



不同模型的分词规则不同，切出来的 Token 也不同！

3 模型处理 Token 序列

模型对每个 Token 的语义和上下文进行计算



4 生成输出 (Detokenization)

将输出的 Token 再组合成文本

今天天气很好，
非常适合去公园散步！



3 Token 有什么特点?



长度不固定

一个 Token 可能是一个字、一个词的部分，甚至是完整的词。



来自词表 (Vocabulary)

所有 Token 都在一个有限的词表中，模型通过词表 ID 进行计算。



影响上下文长度

模型的上下文窗口（如 8K、32K）指的是 Token 数量，而不是字数。



关系到成本

Token 数量决定了推理/训练的计算量和费用（按 Token 计费）。



语义最小单元

Token 不是严格的语言学单位，但在模型中是语义建模的最小单元。

大模型世界中的 Embedding



我把
语义变成向量!



一张图读懂 Embedding: 把语义变成向量, 让机器理解更轻松!

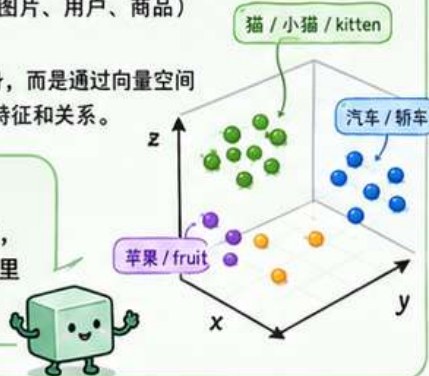
1 Embedding 到底是什么?

在大模型、搜索、推荐和机器学习中, Embedding 是把离散对象(如词、句子、图片、用户、商品)映射成稠密向量的表示方法。

模型不会直接“理解”文字本身, 而是通过向量空间中的位置与距离来捕捉语义、特征和关系。

可以把它理解成:

给每个对象一个多维坐标, 让语义相近的内容在空间里靠得更近。



2 Embedding 是如何工作的?

1 输入对象

例如文本:
“今天天气很好,
适合散步”。



2 编码器/Embedding 模型

把输入转换成
一串数字向量。



3 向量表示

例如:
[0.12, -0.83, 0.41, ...]
(通常是几十维、
几百维甚至上千维)

0.12 -0.83 0.41 ... 0.07

4 下游使用

可用于相似度计算、
检索、聚类、分类、
推荐、RAG 召回等。



不同模型产生的 Embedding 空间不同, 不能随意混用。

3 Embedding 有什么特点?



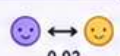
稠密向量

不同于 One-hot, Embedding 更紧凑,
能表达更多信息。



语义相似可度量

相近语义的向量通常更接近,
可用余弦相似度衡量。



高维表示

每一维通常没有直观语义,
但整体向量携带有效特征。



可训练/可迁移

可在预训练中学习到,
再用于下游任务。

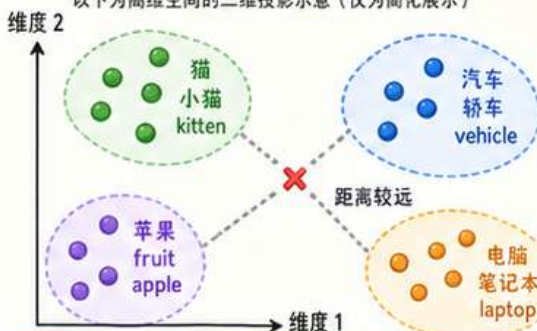


是模型的“表示层”

它是连接原始输入与模型理解能力
的重要桥梁。

4 Embedding 长什么样?

以下为高维空间的二维投影示意(仅为简化展示)



真实的 Embedding 存在于高维空间(几十到上千维)。
二维图只是为了直观展示, 不能代表真实结构。

5 Embedding 能做什么?



语义搜索

搜“退款规则”也能找到“售后政策”。



RAG 检索

为大模型找到相关知识片段,
提升回答的准确性。



推荐系统

找到兴趣相似的用户和商品,
提升点击与转化。



文本聚类/分类

自动发现主题或辅助判别,
如情感分析、意图识别等。



多模态任务

让文本、图片、音频等不同模态
映射到同一向量空间, 便于比较。



Embedding 很强, 但不等于真正理解世界;
它依赖训练数据与模型质量。



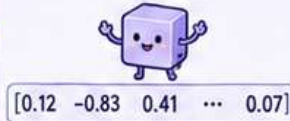
一句话总结

Embedding 就是把对象压缩成可计算的
向量表示, 让模型能够比较相似性、
捕捉语义关系, 并支持检索、推荐与
理解任务。

输入对象



编码成向量



计算距离/相似度



支持任务



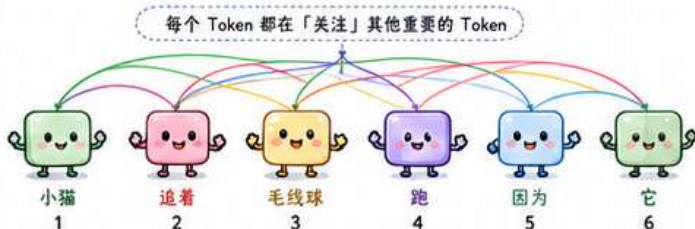
大模型世界中的 Transformer



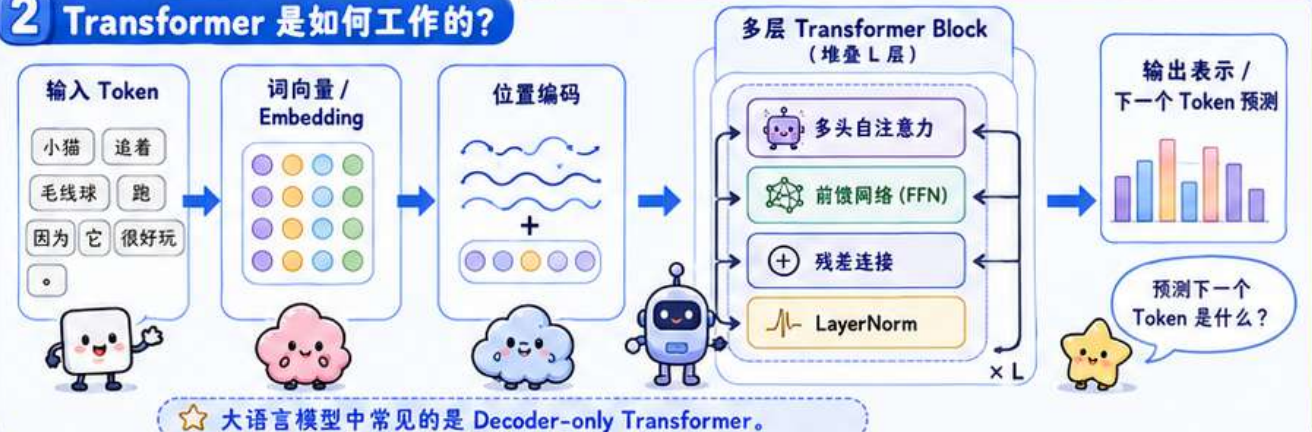
一张图读懂 Transformer: 让模型更会理解上下文!

1 Transformer 到底是什么?

Transformer 是一种基于注意力机制的神经网络架构，擅长建模序列数据和上下文关系。它不再像传统 RNN 那样主要依靠循环处理文本，而是通过并行计算所有 Token 之间的关系。每个 Token 都可以“看见”其他重要的 Token!



2 Transformer 是如何工作的?



3 自注意力 (Self-Attention) 是什么?

核心思想: 让每个 Token 关注与自己最相关的其他 Token, 从而获取有用的信息。



4 Transformer 有什么关键特点?



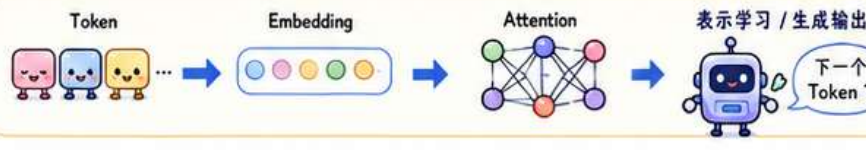
5 Encoder / Decoder 结构



6 Transformer 的优势与挑战

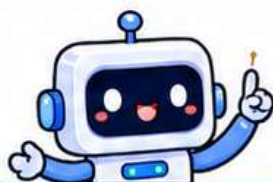


★ Transformer 是以注意力机制为核心的深度学习架构, 它通过让每个 Token 动态关注上下文, 在并行计算中高效建模序列关系, 成为现代大模型的核心基础。



大模型世界中的 Attention

一张图读懂 Attention: 让模型知道“该关注哪里”!



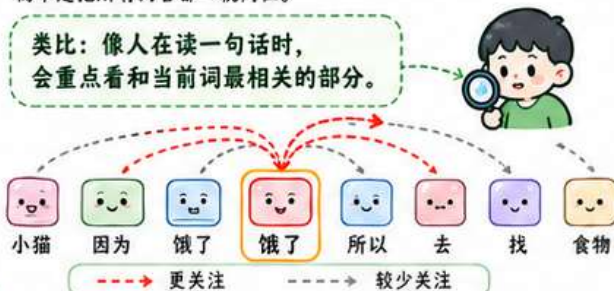
把注意力
放在重要的地方!



1 Attention 到底是什么?

Attention (注意力) 是一种机制, 让模型在处理信息时, 为输入的不同部分分配不同的重要性权重。它让模型学会在当前上下文中“该关注哪里”, 而不是把所有内容都一视同仁。

类比: 像人在读一句话时, 会重点看和当前词最相关的部分。



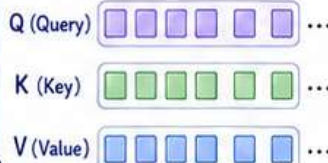
2 Attention 是如何工作的?

1 输入一串 Token



小猫 因为 饿了 饿了 所以 去 找 食物

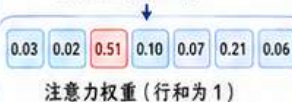
2 为每个Token生成 Q / K / V (Query / Key / Value)



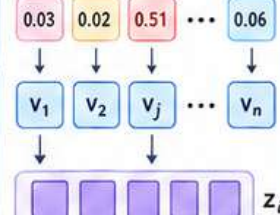
3 计算相关性分数并用 Softmax 得到权重

$$\text{score}_{i,j} = \frac{Q_i \cdot K_j^T}{\sqrt{d_k}}$$

Softmax 行归一化



4 对 Value 加权求和, 得到新的表示



i 注意力权重表示: 第 i 个 Token 会在多大程度上关注第 j 个 Token。

3 Self-Attention 是什么?

Self-Attention: 同一句序列内部互相相关。

例句: 小猫 因为 饿了 所以 去 找 食物

我们关注: “饿了”这个词



★ 同一句序列内部互相关注

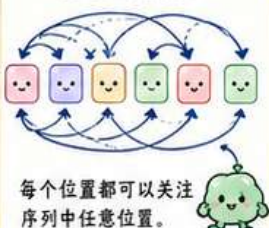
★ 能捕捉长距离依赖 (如“饿了”关注“食物”)

★ 是 Transformer 的核心机制

4 Attention 有哪些类型?

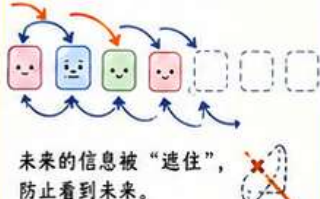
1 Self-Attention

序列内部看彼此



2 Causal / Masked Attention

只能看当前位置及之前内容, 常见于生成式大模型



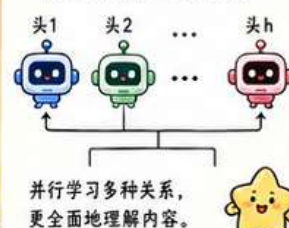
3 Cross-Attention

一个序列去关注另一个序列, 常见于多模态或编码器-解码器



4 Multi-Head Attention

从多个角度同时关注信息



5 Attention 有什么优势与局限?

优势

- 动态聚焦关键信息
把注意力放在最相关的内容上。
- 更好建模上下文关系
理解词与词之间的复杂联系。
- 擅长处理长距离依赖
能关联序列中较远的词。
- 适合并行计算
相比 RNN, 训练更高效。

局限

- 序列很长时计算和内存成本高昂
复杂度为 $O(n^2)$, 长度增加开销大。
- 注意力权重不等于真正“理解”
只是相关性度量, 并非语义理解。
- 长上下文中可能仍有信息稀释问题
注意力分布被拉平, 重要信息被稀释。
- 需要大量数据和训练
模型质量高度依赖训练数据与算力。

6 Attention 在大模型里能做什么?

文本理解

抓住关键词和上下文



文本生成

预测下一个 Token 时关注前文



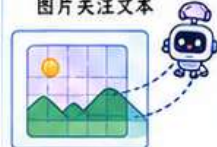
RAG / 检索增强

结合外部信息时 定位关键内容



多模态

让文字关注 图片区域, 或 图片关注文本



翻译 / 摘要 / 问答

根据任务聚焦 相关信息



一句话总结:

Attention 就是让模型在处理信息时, 动态决定“看哪里、看多少”, 因此能更好地理解上下文与生成结果。

让模型
更聪明地思考!

大模型世界中的 Context Window

一张图读懂 Context Window：模型一次能“看到”多长的上下文？



1 Context Window 到底是什么？

Context Window（上下文窗口）是模型在一次处理或生成过程中，能够同时纳入计算的 Token 范围。

它不是字符数，也不是严格的单词数，
而是 **Token 数**！

类比：像一个“工作台”或“记忆桌面”。
桌面上摆得下多少内容，模型就能同时参考多少内容。



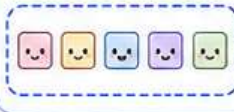
2 Context Window 是如何工作的？

1 输入文本被切成 Token

你好，世界！
今天天气真好。



2 Token 按顺序进入上下文窗口



3 模型在窗口内对这些 Token 建立关联并计算注意力（Attention）



4 生成下一个 Token 时，会参考窗口里的已有内容



完整的 Token 序列（可能很长）



当前可见范围（Context Window）



最新输入 / 待生成位置

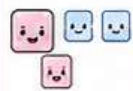


生成新 Token 后，窗口内容会继续被利用。

当内容过长时，模型需要截断、滑动窗口，或依赖系统设计（如摘要、检索、记忆机制）处理超长上下文。

3 Context Window 有什么关键特点？

1 以 Token 为单位



一切以 Token 为计量基础。

2 长度有限，不是无限记忆



有上限，桌子再大也有限。

3 决定一次可处理的信息上限



窗口越大，能处理的信息越多。

4 与成本和延迟强相关



更多 Token 会带来更高的成本和延迟。

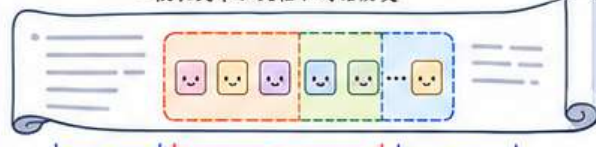
5 更长窗口不等于一定更好



还取决于模型是否能有效利用长上下文。

4 Context Window 长什么样？

一段长文本 / 文档 / 对话历史



不同窗口大小对比（示意）



不同模型支持的上下文长度不同，选择时需根据任务和成本综合考虑。

5 Context Window 能做什么？

长文问答



在长文章中定位信息并找到答案。

多轮对话



记住前面对话内容，保持连续理解。

RAG / 检索增强



同时参考检索到的多段资料再回答。

代码理解



一次查看更多文件片段和上下文。

文档总结



处理更长的报告、论文、合同等。

多模态任务（若模型支持）



结合文本与图像等多模态信息。

6 Context Window 的优势与挑战

优势



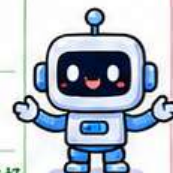
能处理更长上下文
一次可纳入更多信息，理解更全面。



降低频繁截断造成的信息丢失
更少因为截断而丢掉关键信息。



对长文分析、代码任务、RAG 更友好
复杂任务更从容，效果更有保障。



挑战



计算开销更高
更多 Token 意味着更大计算量。



推理速度可能变慢
窗口越大，生成和响应越慢。



长距离信息不一定都能被充分关注
注意力会随距离衰减。



内容过多时仍可能发生“遗忘”或注意力稀释
噪声增多，关键信息被掩盖。



小提醒：上下文窗口 ≠ 永久记忆；
它只是模型当前这次推理能看到的范围。

一句话总结：

Context Window 就是大模型一次能同时“看到并利用”的 Token 范围，它决定了模型处理长文本、长对话和复杂上下文的能力边界。



理解上下文，
用好大模型！

大模型世界中的 Positional Encoding

一张图读懂位置编码：让模型知道“顺序”和“位置”！



没有位置，
只看内容；
有了位置，
理解顺序！



1 Positional Encoding 到底是什么？

Transformer 的自注意力本身并不自然地知道 Token 的先后顺序，Positional Encoding（位置编码）就是把“位置信息”注入到 Token 表示中。

它告诉模型：谁在前面，谁在后面，彼此之间隔了多远。

类比：火车车厢的位置



知道车厢位置，才能理解整列火车的结构和关系！

2 为什么需要位置编码？

相同的词，在不同顺序下，含义完全不同！

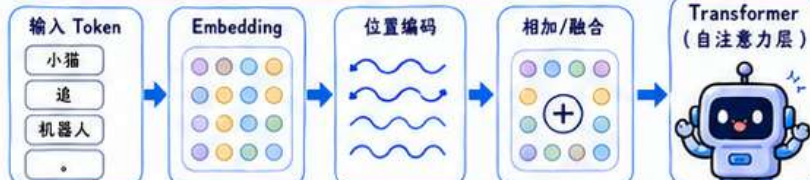
小猫 追 机器人
主体 动作 目标

VS

机器人 追 小猫
主体 动作 目标

如果没有位置信息，注意力机制只看内容的相似性，可能把两句话混淆成一样，导致理解错误！

3 Positional Encoding 是如何工作的？



位置 (pos)	0	1	2	3
Token	小猫	追	机器人	.
Embedding 向量 (示意)				
位置编码向量 (示意)				
融合后向量 (示意)				

★ 位置编码与 Token Embedding 相加后，一起送入 Transformer 处理。

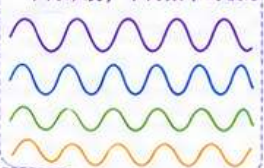
4 经典正弦位置编码

最经典的固定位置编码方法：用不同频率的正弦/余弦函数表示位置。

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

不同维度，不同频率的波形



- ★ 不同维度 i 使用不同频率，能编码丰富的位置信息。
- ★ 相邻位置的向量变化平滑，模型更容易学习。
- ★ 对更长序列有一定外推能力（可自然推广到更长位置）。



5 还有哪些位置表示方式？



绝对位置编码

给每个位置一个编号或向量。



可学习位置编码

位置向量通过训练学出来，更灵活。



相对位置编码

更关注词与词之间的相对距离或关系。



RoPE (旋转位置编码)

通过旋转操作编码位置，在大模型中广泛使用。



ALiBi (注意力偏置)

用线性偏置体现距离，简单高效，适合长上下文。

不同方法，各有优劣~

6 位置编码有什么关键特点？

1

注入顺序信息

让模型知道谁在前，谁在后。

2

帮助理解长短距离关系

更好捕捉远近依赖和结构信息。

3

与注意力机制配合使用

位置编码为注意力提供重要的位置信号。

4

不同方法适合不同模型

根据任务、数据和长度选择合适方案。

5

影响长上下文能力

位置编码的设计直接影响模型处理超长序列的能力。

7 Positional Encoding 能做什么？

帮助模型理解顺序、依赖和结构，从而更好地完成各种任务！

文本理解



文本生成



长文建模



代码理解



翻译



RAG/检索增强



多模态序列处理



8 优点与挑战

优点

- ✓ 让 Transformer 感知顺序。
- ✓ 提升上下文理解能力。
- ✓ 支持复杂的序列建模。
- ✓ 为长上下文建模奠定基础。



挑战

- ✗ 长上下文外推不总是稳定。
- ✗ 不同方法效果差异大。
- ✗ 需要与模型结构共同设计。
- ✗ 计算/显存开销需要考虑。



9 一句话总结：

★ Positional Encoding 就是把“位置信息”注入 Token 表示中，让 Transformer 不只看见内容，还能理解顺序、距离和结构。



0 → 1 → 2 → 3



位置在心中，
理解更从容！



大模型世界中的 MoE

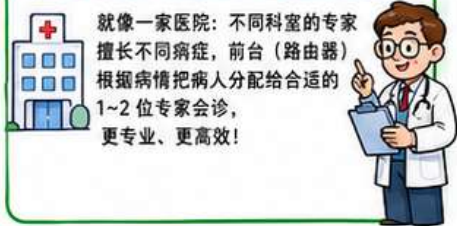
Mixture of Experts (专家混合)：让大模型按需激活、更高效地思考！



1 MoE 到底是什么？

MoE 是一种神经网络架构，不是所有参数每次都参与计算，而是把模型拆成多个“专家网络 (Experts)”，再由“路由器 (Router/Gate)”根据输入动态选择少数几个专家处理。

总参数大，但单次激活参数少。



2 MoE 是如何工作的？

① 输入 Token / 词向量

请帮我写一段快速排序的代码

T1 T2 T3 ... Tn

② 路由器根据输入给专家打分

数学专家 0.15
编码专家 0.72
语言专家 0.08
知识专家 0.35

不同的 Token 可能被分配到不同的专家！

T1 → 语言专家 T2 → 编码专家 T3 → 知识专家 ... Tn → 数学专家

③ 选择 Top-k 专家 (例如 Top-2)

数学专家
编码专家
语言专家
知识专家

④ 被选中的专家并行处理，再把结果加权合并输出

编码专家
知识专家
加权合并
输出结果

3 MoE 的核心组成

Experts (专家网络)
多个 FFN 子网络，擅长不同模式

Router / Gate (路由器)
决定把输入送给谁

Top-k 选择
只激活少量专家 (如 Top-1/2/4)

Load Balancing (负载均衡)
避免所有输入都挤向少数专家

Shared / Dense 部分 (可选)
某些层仍可能是共享参数

4 MoE 有什么关键特点？

稀疏激活



每次只用少量专家

参数规模大



总参数可非常多

计算更省



相比同等总参数的稠密模型更高效

专业分工



不同专家学习不同能力

动态路由



不同输入走不同路径

5 MoE 能带来什么好处？

扩展模型容量：在可控算力下容纳更多参数。

提升训练/推理效率：单次前向不必激活全部参数。

更强表达能力：专家分工有助于学习多样模式。

适合超大模型：是许多前沿大模型的常见设计思路。

6 MoE 的挑战与局限

- ⚠️ 路由不稳定：训练初期可能难学，收敛慢。
- ⚠️ 负载不均衡：部分专家过忙，部分专家闲置。
- ⚠️ 通信开销：分布式训练时 All-to-All 通信复杂。
- ⚠️ 专家坍塌：模型过度依赖少数专家，泛化变差。
- ⚠️ 部署复杂：工程实现和调优难度更高。
- ⚠️ 不一定总更好：效果依赖数据、路由策略和训练设计。



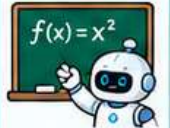
7 MoE 在大模型里能做什么？

通用对话



更贴合用户问题，更自然流畅

数学与推理



调用擅长推理的专家，提升准确性

代码生成



激活编码专家，生成高质量代码

多语言处理



不同语言模式由相应专家处理

RAG / 工具调用场景



更好地理解需求，选择合适工具或知识

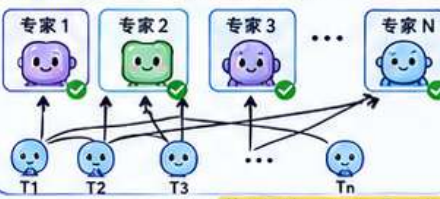
MoE 让模型在不同任务或 Token 模式下，调用更合适的内部专家！

8 MoE 与稠密模型有什么区别？

稠密模型 Dense

特点：每次前向：几乎所有参数都参与计算；结构简单；计算稳定；扩容成本高。

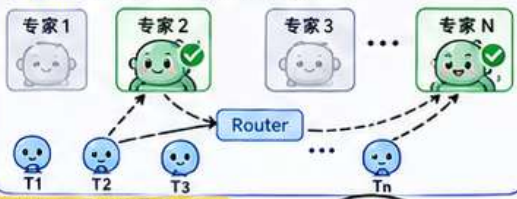
示意图：所有专家都激活 (全部参与)



MoE 模型

特点：每次前向：只激活少数专家；总参数大；更高效扩容；训练和部署更复杂。

示意图：仅 Top-k 专家激活 (如 Top-2)



核心区别：Dense 是“人人上”，MoE 是“按需叫人”！



一句话总结

MoE (专家混合) 就是把大模型变成“多专家协作系统”：

总参数很多，但每次只**动态激活**最合适的**少数专家**，从而在控制计算成本的同时**提升模型容量与能力**。





大模型世界中的 Pretraining



★ 一张图读懂 Pretraining: 让模型先大量学习, 再执行具体任务! ★

1 Pretraining 到底是什么?

Pretraining (预训练) 是模型在执行具体任务之前, 先利用海量数据进行通用学习的阶段。模型通过自监督学习, 掌握语言、知识与模式, 从而具备强大的通用能力。

先通识学习, 再专项应用

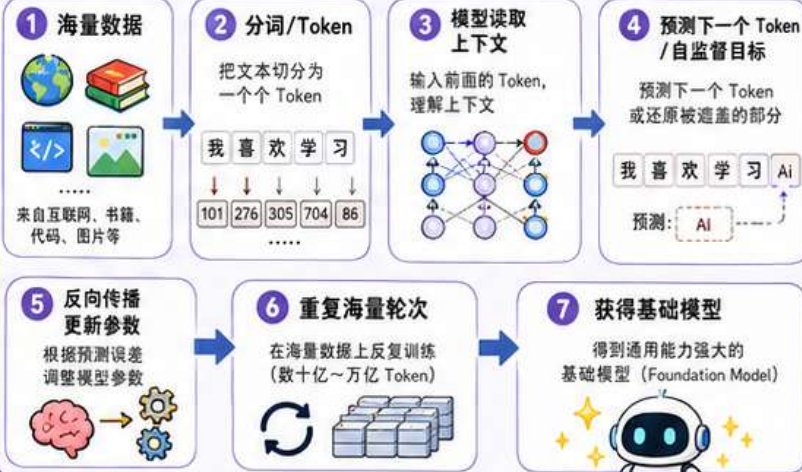


2 为什么需要 Pretraining?

从零开始直接做任务, 模型什么都不知道, 效果很差。预训练帮助模型学习词汇、语法、世界知识、推理模式和统计规律, 打好基础, 再快速适配具体任务。



3 Pretraining 是如何工作的?



4 预训练学什么?



模型学到的是模式与统计规律, 而不是真正的人类理解或意识。

5 常见预训练目标



目标的本质: 通过自监督信号, 让模型在无人工标注的情况下进行学习。

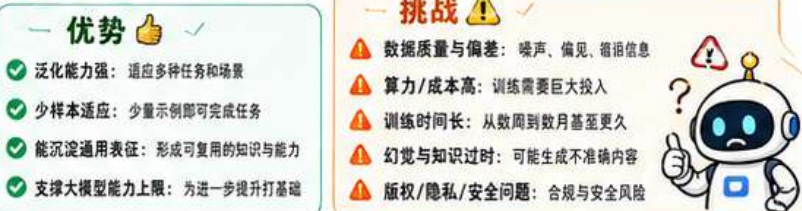
6 Pretraining 有什么关键特点?



7 Pretraining 能做什么? (驱动下游广泛应用)



8 优势与挑战



一句话总结

★ Pretraining (预训练) 就是让大模型先在海量数据上进行自监督学习, 掌握通用语言与知识模式, 再把这些能力迁移到各种具体任务中。



大模型世界中的 Post-Training

让模型从“会说”到“说得好、说得对、说得安全”



Pre-training 让模型学会了语言，
Post-training 让模型学会了做人。



1. 什么是 Post-Training?

Post-Training（后训练）是指在大规模无监督预训练（Pre-training）之后，利用有监督数据、人工反馈或特定任务数据，进一步对模型进行优化，使其更有用、更安全、更符合人类偏好。

Pre-training
(预训练)



Post-Training (后训练)



SFT
监督微调



奖励模型
训练



RLHF / DPO
对齐优化



更好的模型
(有用、安全、可靠)



2. 为什么需要 Post-Training?



预训练模型的问题

- 可能生成错误信息（幻觉）
- 不遵循指令
- 输出不安全内容
- 不符合人类价值观



Post-Training 的作用

- 提升指令遵循能力
- 减少有害或不当输出
- 对齐人类偏好和价值观
- 提升实用性和可靠性

4. 常见的 Post-Training 方法对比

方法	使用数据	核心目标	优点	缺点
SFT	人工标注的指令数据	提升指令遵循能力	简单有效	可能仍有偏差
Reward Model	人工偏好对比数据	学习人类偏好评分	可量化偏好	需要大量对比数据
RLHF (PPO)	指令数据 + 奖励模型	最大化奖励（人类偏好）	效果显著	训练复杂，易不稳定
DPO (Direct Preference Optimization)	人工偏好对比数据	直接优化偏好（无需 RL）	更稳定，训练更简单	效果略逊于 RLHF



目前主流大模型（如 ChatGPT、Claude、Gemini 等）普遍使用 SFT + RLHF 的组合范式进行后训练。

3. Post-Training 的主要阶段

(1) 监督微调 (SFT, Supervised Fine-Tuning)

使用人工标注的“指令-回答”数据，
让模型学会更好地遵循指令。



请介绍一下相对论。

相对论是由爱因斯坦提出的物理学理论，主要包括.....



指令-回答数据集



微调模型

(2) 奖励模型 (Reward Model, RM)

训练一个奖励模型，判断模型输出的
质量或偏好程度。

输出 A

相对论是物理学的重要理论，
它改变了我们对时空的理解.....



输出 B

我不知道。



输入
(指令 + 输出)



奖励模型

打分
(0 ~ 1)

(3) 对齐优化 (RLHF / DPO 等)

利用奖励模型的反馈，通过强化学习的
或直接偏好优化，进一步优化模型。



当前模型



生成多个输出



奖励模型打分



更优模型



强化学习更新
(PPO)
或 DPO 优化

5. Post-Training 带来的能力提升



更安全

减少有害、歧视、
暴力等不当内容



更可靠

减少幻觉，提
供更准确答案



更对齐

符合人类价值观
和使用习惯



更有用

在特定任务和领域
表现更出色

Post-Training 是大模型从“通才”走向“好用的助手”的关键一步。



6. 未来趋势



更高效的对齐方法

减少对大量人工数据的依赖



多模态对齐

让模型在文本、图像、语音等多模态上更对齐



个性化对齐

根据用户或场景进行个性化优化



价值观可控

让模型的价值观和行为更加可控和透明



一句话总结

Pre-training 让模型学会了语言，Post-Training 让模型学会了做人。
它让大模型更安全、更可靠、更符合人类需求，是通用人工智能的重要基石。



大模型世界中的 Sampling

一张图读懂 Sampling: 模型如何从“候选答案”中做选择?



我要怎么选下一个 Token?



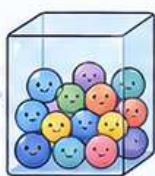
1. Sampling 到底是什么?

当模型根据上下文预测下一个 Token 时, 会得到一个概率分布。Sampling 就是从这个分布中, 按照一定策略抽取下一个 Token 的方法。

它会影响生成内容的风格、创造性、稳定性和随机性。



类比: 从概率桶里投球抽小球!



天气	0.45
今天	0.28
很好	0.15
!	0.07
其他	0.05

2. Sampling 是如何工作的?



大模型是自回归生成 (Autoregressive): 一步一步生成, 每次只预测下一个 Token。

3. 为什么不能总是选概率最大的?

总是选择概率最大的 (argmax / greedy), 虽然稳定, 但容易单调、重复, 甚至陷入平庸输出。

Greedy (贪心解码)



- ✓ 更稳定
- ✓ 更确定
- ✗ 可能单调
- ✗ 易重复、易卡壳

Sampling (采样)



- ✓ 更多样
- ✓ 更灵活
- ✗ 可能跑偏
- ✗ 有幻觉风险

VS

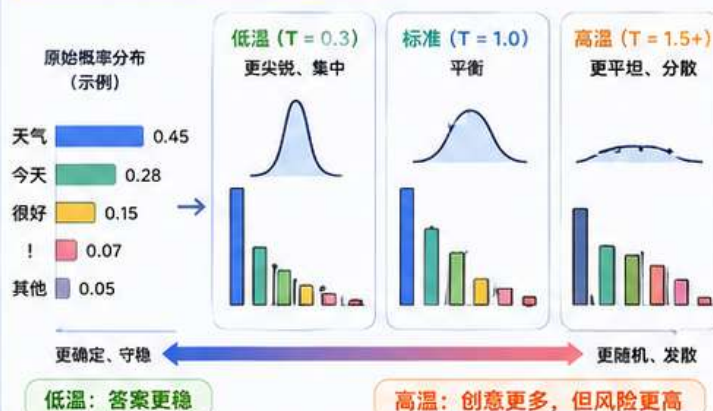
大键: 在稳定性和多样性之间找到平衡!



4. 常见 Sampling 方法

- Greedy Decoding (贪心解码)** 直接选择概率最大的 Token。
- Temperature (温度)** 调整分布尖锐程度。低温更保守, 高温更发散。
- Top-k Sampling** 只在概率最高的前 k 个候选中采样。
- Top-p / Nucleus Sampling** 在累计概率达到 p 的最小候选集合中采样。
- Repetition Penalty (重复惩罚)** 降低已生成过的 Token 概率, 减少重复。
- Beam Search (束搜索)** (搜索导向, 非随机采样) 同时保留多个高分路径, 按得分扩展, 选择最优序列。

5. Temperature 长什么样?



6. Sampling 会影响什么?

- 多样性** 采样越随, 输出越多样。
- 可控性** 合适的参数让风格更可控。
- 创造性** 更高随机性带来新想法。
- 稳定性** 随机性过高可能不稳定。
- 幻觉风险 / 跑偏风险** 越发散, 越可能跑偏或生成不准确信息。

更高的随机性 = 更多惊喜, 但也带来更高的不确定性与风险。

7. 在大模型里能做什么?

- 聊天对话** 日常聊天: 中等随机性更自然、有变化。
- 创意写作** 故事、诗歌、文案: 更高随机性激发灵感。
- 代码补全** 降低随机性: 更准确, 减少语法/逻辑错误。
- 头脑风暴** 更高随机性: 产生更多想法与角度。
- 多轮生成** 过慢任务: 合适采样保持连贯。
- 游戏 NPC / 角色扮演** 高随机性: 更生动、有个性。

需要准确、可靠 → 选择更低随机性

需要创意、变化 → 选择更高随机性

8. 如何选择合适参数?

- 事实问答 / 代码 / 翻译** → 低温 (如 0.2-0.6) + 小随机性
可选: Top-p 0.7-0.9 或无采样
- 创意写作 / 文案 / 头脑风暴** → 中高温 (如 0.8-1.2+) + Top-p 0.8-0.95
可搭配适度重复惩罚
- 通用聊天 / 日常对话** → 中等温度 (如 0.6-0.9) + Top-p 0.8-0.95
平衡稳定性与多样性

★ 没有放之四海而皆准的最佳设置! 请根据任务目标、数据与反馈不断调整。

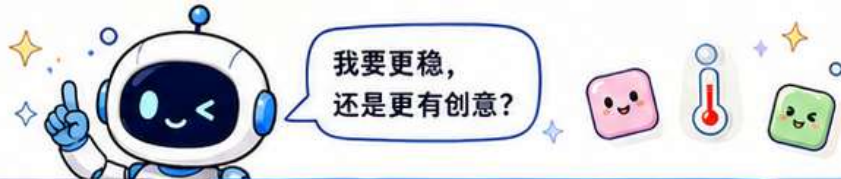
一句话总结

Sampling 就是大模型在每一步生成时, 从概率分布中选择下一个 Token 的方式; 它决定了回答到底更稳定, 还是更多样、更有创造力。



大模型世界中的 Temperature

一张图读懂 Temperature: 它如何影响模型回答的随机性、创造性与稳定性?

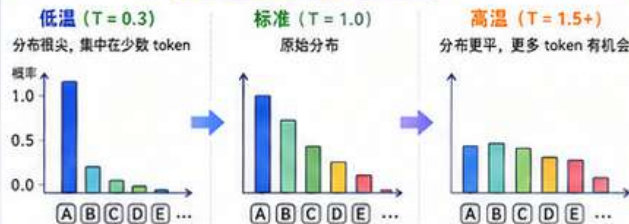


1. Temperature 到底是什么?

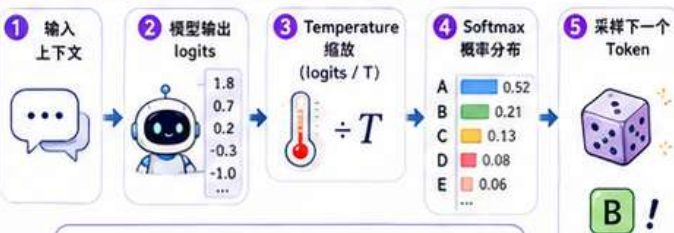
Temperature 是解码 (采样) 阶段的控制参数, 它不会改变模型学到的知识, 而是对下一 Token 候选的概率分布进行缩放重塑。

- 温度越低: 分布越“尖锐”, 高分 token 优势被放大, 输出更确定。
- 温度越高: 分布越“平坦”, 低分 token 也有机会被选中, 输出更随机。

以下 Token 的概率分布为例



2. Temperature 是如何工作的?



$$p_i = \text{softmax}\left(\frac{\text{logit}_i}{T}\right) = \frac{e^{\text{logit}_i / T}}{\sum_j e^{\text{logit}_j / T}}$$

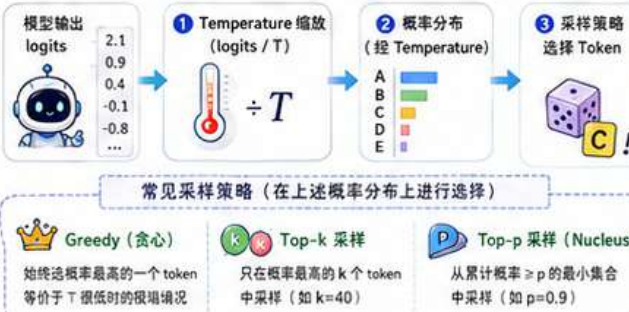
- $T < 1$: 放大高分 token 的优势, 使分布更尖锐, 更偏向最优选择。
- $T = 1$: 保持原始分布, 模型按其训练时的概率输出。
- $T > 1$: 拉平分布, 增加低概率 token 被选中的机会。

3. 不同 Temperature 会带来什么效果?

低温 (如 0.2 ~ 0.5)	中温 (如 0.7 ~ 1.0)	高温 (如 1.2 +)
更稳定, 更保守, 更一致 适合事实问答、代码、翻译	平衡稳定性与多样性 适合日常对话	更发散, 更有创意 但更易跑偏、重复、幻觉
示例问答: “水的化学分子式是什么?” 输出风格: 水的化学分子式是 H ₂ O。	示例问答: “推荐我一个周末的放松活动。” 输出风格: 可以去公园散步、看电影, 或者和朋友一起探店, 都能放松心情!	示例问答: “推荐我一个周末的放松活动。” 输出风格: 你可以去海边看日出, 和机器人一起喝咖啡, 或者参加一场外星主题派对!
更少变化 更少跑偏 更一致的答案	适度变化 自然流畅 兼顾准确与灵活	创意更多 变化更大 更容易跑偏/幻觉

4. Temperature 和 Sampling 有什么关系?

Temperature 通常与采样策略配合使用, 是控制随机性的“旋钮”之一。



结论: Temperature 先重塑概率分布, Sampling 策略再决定从哪里选。

5. Temperature 会影响什么?



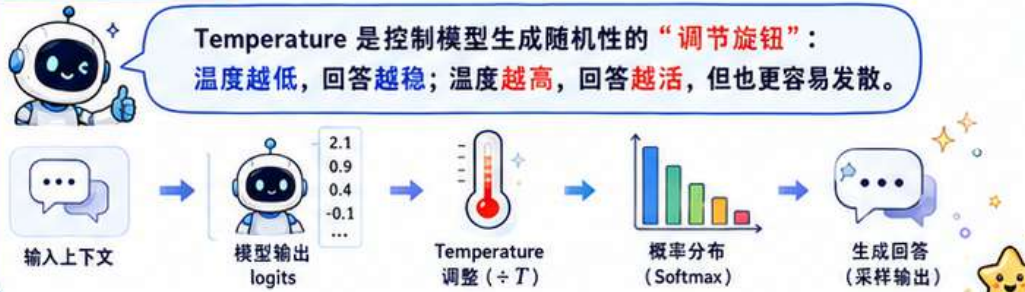
6. 什么时候该用高温? 什么时候该用低温?

温度区间	适用场景	典型任务 / 示例
低温 (0.2 ~ 0.5)	追求准确、一致、可控	• 代码补全 / 生成 • 数学推理与证明 • 信息抽取 / 结构化输出 • 严谨问答 / 知识检索 • 企业客服 / 标准化回复
中温 (0.7 ~ 1.0)	平衡稳定性与多样性	• 通用聊天 • 学习辅导 / 讲解 • 总结 / 润色 / 改写
高温 (1.2 +)	追求创意、发散与灵感	• 故事创作 / 小说写作 • 头脑风暴 / 创意构思 • 广告文案 / 标语 • 角色扮演 / 世界观设定 • 诗歌 / 歌词 / 创意写作

7. 常见误区与注意事项



8. 一句话总结





大模型世界中的 Reasoning Model

一张图读懂 Reasoning Model: 让模型不只会“回答”，还会“分析、推理与验证”！

我会一步步想清楚！



1 Reasoning Model 到底是什么？

Reasoning Model 是为多步分析、分解、规划、逻辑推理、检查等能力优化的大模型。它不仅背答案，更会尝试构建中间推理步骤，来解决更难的问题。

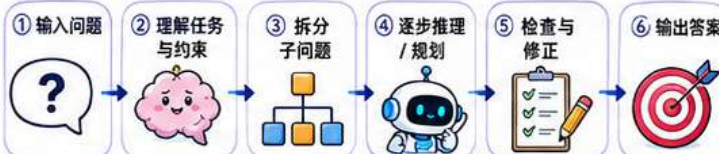
举个例子

解一道数学应用题就像这样：



★重点：更擅长复杂问题，不代表永远正确。

2 Reasoning Model 是如何工作的？



例子 如果一辆车 3 小时行驶 180 km，平均速度是多少？



💡 通过逐步推理，而不是直接猜答案，提高可靠性。

3 Reasoning Model 有什么关键特征？

- 🧩 多步推理：能连续处理多步信息
- 📦 问题分解：把复杂任务拆成小任务
- 📋 计划能力：先想步骤，再执行
- 🔍 自我检查：发现错误后修正
- 🔧 工具协同：可结合检索、代码、计算器等工具



推理能力通常来自预训练 + 后训练 + 数据与训练策略。

4 常见推理模式有哪些？

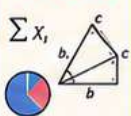
逻辑推理



条件、因果、真假判断

分析

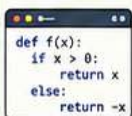
数学推理



计算、公式、数量关系

计算

代码推理



理解程序逻辑、定位 bug

验证

决策推理



比较方案、权衡约束、做规划

比较

5 Reasoning Model 与普通生成模型有什么区别？

普通生成模型	VS	Reasoning Model
生成更快，响应更迅速	速度	相对更慢，需要更多计算
擅长流畅表达、续写、创作	擅长点	擅长复杂任务、多步推理
可能跳过中间分析，直接给答案	思考深度	会拆解、分析、一步步推理
简单问答、开放式表达更突出	适用任务	数学、代码、逻辑、规划等难题
可能不严谨，易遗漏关键步骤	可靠性	更一致，可检查、可验证

★ 不是谁完全替代谁，而是能力侧重点不同。

6 Reasoning Model 能做什么？



数学题与逻辑题



代码生成 / 调试



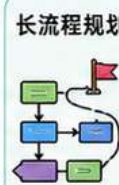
复杂问答



数据分析



工具调用与 Agent 任务



长流程规划

7 优势与挑战

优势

- ✓ 更擅长复杂任务
- ✓ 更容易解释步骤
- ✓ 更适规划、验证和工具使用

挑战

- ⚠ 推理更慢、成本更高
- ⚠ 仍可能出错或幻觉
- ⚠ 长链路推理可能累积误差
- ⚠ 不同任务效果差异大

⚠ 会推理 ≠ 一定正确，仍需评估与验证。

8 一句话总结

Reasoning Model 就是更擅长“拆解问题、逐步分析、检查修正”的大模型。它让模型从“会说”进一步走向“会想、会做、会验证”。





大模型世界中的 Multimodality

一张图读懂 Multimodality: 让模型同时理解文字、图像、语音与视频!



1 Multimodality 到底是什么?

Multimodality (多模态) 指模型能够同时接收、理解并关联多种类型的信息, 如文本、图像、语音、视频、表格等, 并进行统一推理与生成。



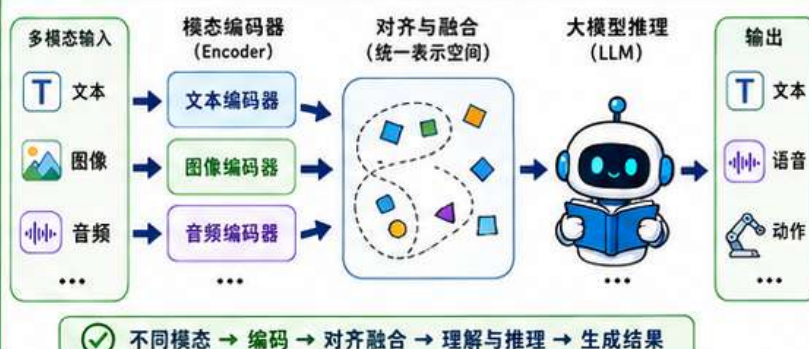
2 为什么需要 Multimodality?

人类通过多种感官理解世界, 许多真实任务需要多模态信息协同。



★ 真实世界是多模态的, 解决复杂任务必须多模态协同!

3 Multimodality 是如何工作的?

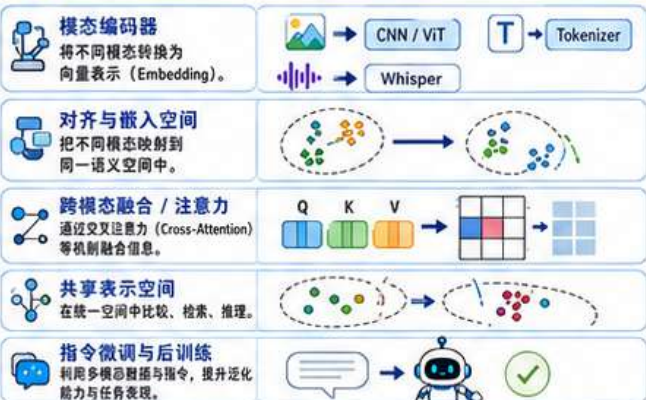


4 常见模态有哪些?



★ 不同模态互补, 组合后信息更全面、更可靠!

5 多模态模型有哪些关键技术?



核心理想: 把不同模态变成机器能理解的向量, 再统一推理与生成。

6 Multimodality 能做什么?



★ 覆盖学习、办公、娱乐、医疗、出行等众多场景!

7 优势与挑战



8 一句话总结



Multimodality 就是让大模型把“看、听、读”结合起来, 在统一表示与推理框架中理解多种信息, 从而完成更复杂、更自然的人机交互任务。



多模态, 多一点理解; 统一表示, 更强智能! ❤️



工具调用 (Tool Calling) 与 Function Calling

核心思想：
模型负责思考，
程序负责执行！

1. 工具调用 (Tool Calling) 是什么

模型根据用户需求，判断是否需要调用工具，生成调用参数，由程序执行工具并返回结果，模型再根据结果生成最终回答。



模型
(思考)



程序
(执行工具)

2. Function Calling 是什么

模型调用开发者预先定义好的函数 (Function)。模型只知道：函数名称、参数格式、功能说明。

模型生成调用请求 (示例)

```
{
  "function": "get_weather",
  "arguments": {
    "city": "北京"
  }
}
```

函数定义

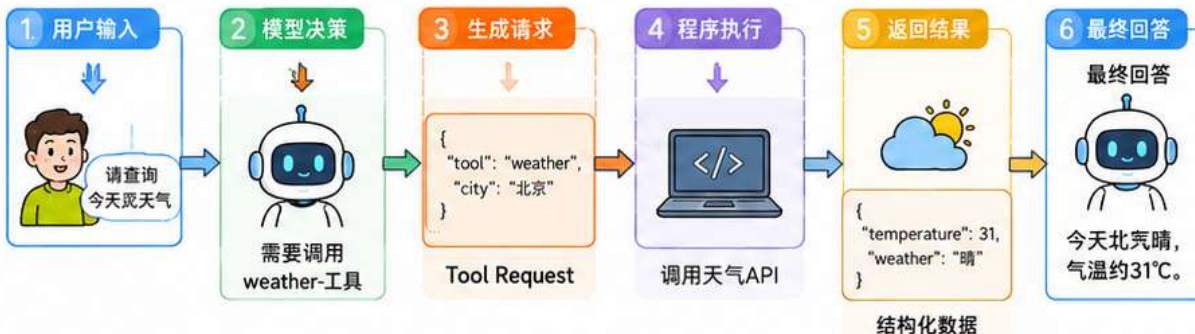
```
get_weather(city)
search_flight(date)
query_order(id)
calculate(expression) ...
```

3. 模型如何调用工具

- ① 判断是否需要工具
- ② 选择合适的工具
- ③ 生成工具调用参数 (JSON)
- ④ 将请求发送给程序
- ⑤ 等待程序返回结果
- ⑥ 根据结果生成最终回答



4. 工具执行流程



5. 工具返回结果

工具执行完成后，将结果以结构化数据返回给模型，模型再基于结果生成自然语言回答。

工具返回 (示例)

```
{
  "price": 7288
}
```

模型理解结果



生成自然语言回答

当前商品价格为7288元。

6. 多工具协作 (Multi-tool Calling)

一个复杂任务可能需要多个工具协同完成，模型会规划调用顺序，逐步完成任务。

RAG 场景示例



天气+邮件 场景示例



通过 Tool Calling，模型可以突破自身能力边界，与外部世界交互，完成更强大的任务！

7. 常见工具类型

- 搜索引擎：搜索互联网信息
- 数据库：查询/写入数据
- API 服务：调用第三方服务
- 计算器：数学计算
- 文件系统：读写文件
- 浏览器：网页操作
- 办公软件：邮件、日历等

8. 关键点

- ✅ 模型不会直接执行工具
- ✅ 工具由程序执行
- ✅ 工具返回结构化数据
- ✅ 可支持单工具或多工具协作
- ✅ 是构建 AI Agent 的基础能力



大模型世界中的 Test-time compute

—— 用更多思考，换更好的答案



先学知识，
再考试时多花点“思考时间”，
就能考出更好成绩！

什么是 Test-time compute?

在推理（测试）阶段，
通过分配更多的计算资源（时间/算力），
让模型进行更多的思考与探索，
从而提升答案的质量和可靠性。

它不改变模型参数，
只在使用时“更用心地思考”。



核心思想

大模型的能力不仅取决于“学了多少”，
也取决于“思考了多久”。



对比：传统推理 vs. Test-time compute

传统推理（一次性生成）



想一次，
直接给答案！

问题



答案
(可能不够好)

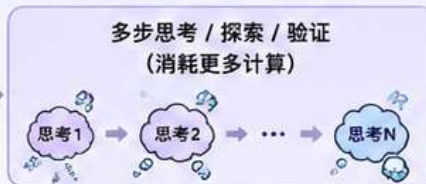


Test-time compute（多步思考/探索）



多想几步，
多试几种可能！

问题



更优答案
(更可靠)



常见的 Test-time compute 方式



1. CoT（思维链）

让模型一步步推理，显式展示思考过程。

例：先计算面积，再考虑条件...



2. Self-consistency（自洽性）

从不同角度生成多条思路，投票出最一致的答案。

例：生成多种解法，选多数意见。



3. 验证与反思（Verify & Refine）

先生成答案，再检查并反思错误，迭代改进。

例：先答一遍，再检查逻辑漏洞。



4. Tree of Thoughts（思维树）

把问题拆成多条分支，搜索更优路径。

例：像下棋一样，评估多步可能性。

本质：让模型在测试时进行“更多的搜索、验证和反思”。

一个例子：数学题

题目：

小明有 12 个苹果，给了小红 3 个，又买了 5 个。
小明现在有几个苹果？

传统推理：

$12 - 3 + 5 = 14$

Test-time compute（思维链 + 验证）：

步骤1：先算给小红后的数量： $12 - 3 = 9$

步骤2：再加上买的： $9 + 5 = 14$

步骤3：检查：如果先买再给呢？ $(12 + 5) - 3 = 14$

步骤4：两种方法结果一致，答案正确。

答案：14 个



为什么有效？

- ✓ 为模型提供更多的“思考时间”，更充分地利用其内在知识与能力。
- ✓ 通过探索与验证，减少幻觉和低级错误。
- ✓ 在不改变模型参数的情况下，显著提升复杂任务的表现。



代价与挑战

- ⚠ 需要更多计算资源与更长延迟。
- ⚠ 如何分配计算（思考到什么程度停止）是关键。
- ⚠ 并非所有问题都需要更长的思考。



总结

Test-time compute = 用计算换思考，用思考换质量。

给模型更多
计算资源（时间）

让模型更多思考、探索、验证

获得更优、更可靠的答案



一句话记住：

模型的潜力，
在思考中被释放！





知识库与 RAG (检索增强生成)

先查资料，再回答问题，让大模型拥有更准、更新的知识

RAG 核心思想：

通过检索外部知识，
将相关信息注入模型上下文，
从而生成更准确的回答。



1. 为什么需要知识库



- ❌ 知识静态：模型训练后无法更新
- ❌ 无法访问私有数据或内部系统
- ❌ 长文本直接输入成本高、效果差

✅ 知识库提供可更新、可检索、可控的外部知识源，弥补模型的不足！

2. 什么是 RAG (检索增强生成)



用户问题
(Query)



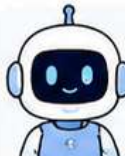
检索系统
(Retrieval)



相关文档
(Documents)



构建上下文
(Context)



大模型生成
(Generation)

最终回答：
根据资料，公司的
报销制度包括...
...

回答
(Answer)

3. 文档解析 (Document Parsing)



文本提取 → 结构解析 → 文本融合

文本切分 (Chunking)

Chunk 1 Chunk 2 Chunk 3

切分策略

- 固定长度切分
- 语义切分
- 滑动窗口
- 结构切分 (按标题层级)

4. OCR 服务

OCR
将图片/扫描件中的
文字识别为文本



扫描件/图片
(发票、表格、合同等)

OCR 识别结果
(可编辑文本)

OCR 是非数字化文档进入知识库
的重要入口。

5. 知识处理 (Knowledge Processing)



清洗
去除噪声



去重
消除重复



分段
优化 Chunk



元数据
构建 Metadata



向量化
(Embedding)

元数据示例

```
{
  "source": "HR_System",
  "department": "Finance",
  "doc_type": "policy",
  "timestamp": "2026-01-15"
}
```



向量数据库
(Vector DB)



6. 知识检索 (Retrieval)

关键词检索 (BM25)



基于词频和倒排索引，
适合精确匹配的查询。

向量检索 (Embedding)



基于语义相似度，
理解自然语言含义。

Top-K 相关文档



0.95



0.89



0.78



...

7. 知识问答 (QA Generation)



用户问题

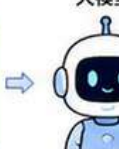
检索到的相关资料



构建 Prompt

请基于以下资料回答问题：
【资料】
...
【问题】公司出差报销的标准是什么？
请确保回答仅基于资料内容。

大模型生成



最终回答

根据资料，公司出差
报销标准包括...
...

8. 上下文增强 (Context Augmentation)



直接注入
(拼接文档)



压缩摘要
(减少 Token)



多文档融合
(整合信息)



重排序优化
(重要优先)

9. 对话式知识库 (Conversational RAG)



用户 (连续提问)

公司年假有多少天？

那试用期能休吗？

历史对话 + 当前问题
(Query 改写)



动态检索
(每轮重新检索)



构建上下文
(融合历史)



生成回答

特点

- ✅ 理解上下文，支持多轮问答
- ✅ 动态检索，保证信息相关性
- ✅ 适合构建企业知识助手、智能客服等系统



RAG 价值

- ① 引入外部知识，突破模型知识边界
- ② 知识可更新，信息更准确
- ③ 降低幻觉 (Hallucination)
- ④ 支持私有数据安全应用

大模型世界中的 System prompt

给模型立规矩的“最高优先级说明书”



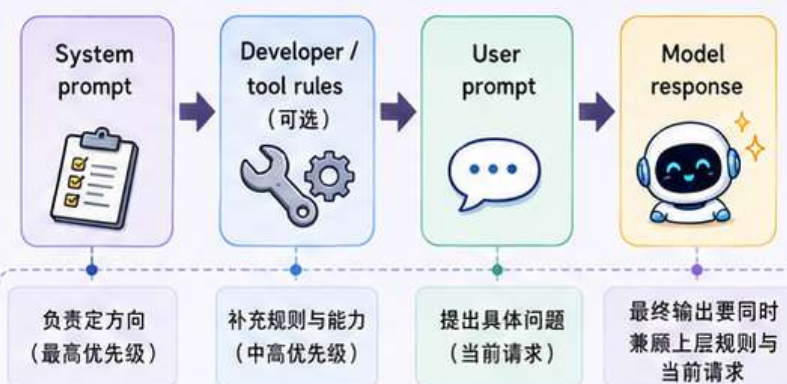
先告诉我身份、
目标和规则，
我就更知道该
怎么回答！

① 什么是 System prompt?

- System prompt 是在对话开始前提供给大模型的一组高优先级指令。
- 它用于规定模型的角色、目标、行为边界、输出风格和任务要求。
- 它通常不会被普通用户直接看到，但会持续影响模型如何回应。



② 它在对话中的位置



③ System prompt 都会规定什么?

角色定位



你是谁
(如老师、助手、
分析师)

任务目标



要完成什么

风格格式



如何表达、输出
成什么形式

安全边界



不能做什么、
如何规避风险

工具与流程



何时检索、何时
调用工具、如何
处理信息

④ 没有它 vs. 有它

没有清晰 System prompt



- ❌ 角色不稳定
- ❌ 输出风格飘忽
- ❌ 容易漏规则
- ❌ 结果一致性较差

VS

有清晰 System prompt

- ✅ 角色更稳定
- ✅ 回答更一致
- ✅ 更符合任务目标
- ✅ 更容易控制质量



⑤ 一个例子

System prompt
(高优先级)

你是一位严谨的数据分析助理，
请用中文、分点、简洁回答，
并在不确定时说明不确定性。

User prompt
(当前请求)

帮我解释过拟合是什么。



模型输出将体现以下特点

- ✅ 中文
- ✅ 分点说明
- ✅ 风格简洁
- ✅ 语气严谨
- ✅ 若有不确定处会标注



注意：

System prompt
不直接给出答案内容，
而是决定“答案如何
被生成”。

⑥ 为什么重要?



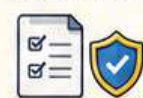
让模型更
稳定地执行任务



提升输出的
一致性与可控性



将安全、格式和
流程要求前置



是构建 AI 产品、
Agent 和工作流的
重要基础



⑦ 局限与挑战

- ⚠️ 不能保证 100% 遵守
- ⚠️ 可能被复杂上下文或提示注入干扰
- ⚠️ 写得太模糊会降低效果
- ⚠️ 写得太长也可能增加成本与冲突



⑧ 总结



System prompt 的本质：先定义规则，再生成回答。



规则与角色设定
(最高优先级)



约束模型行为
(影响全过程)



提高稳定性与可控性
(更可用、更可靠)



它像大模型的
“开场说明书”，
决定回答从哪里出发！



大模型世界中的 Few-shot

给几个例子，让模型更快学会怎么做

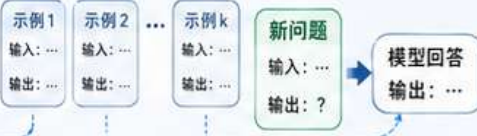


给我几个例子，
我就能更快懂
任务，做得更好！

1 什么是 Few-shot?

Few-shot 是一种提示方式，在正式问题之前，先给模型少量示例输入与输出，让模型从示例中理解任务格式、风格、规则与映射关系，再完成新问题。

★ 它不更新模型参数，而是在上下文中临时学习。



2 它在提示中的结构



3 Zero-shot / One-shot / Few-shot 对比

Zero-shot
(零样本)



不给例子，
只给任务说明。

One-shot
(一样本)



给 1 个例子，
帮助理解格式。

Few-shot
(少样本)



给少量例子 (通常
2~5 个或更多)。

★ Few-shot 的优势: 更容易让输出格式统一、风格稳定、任务意图更明确。

4 一个直观例子: 情感分类

任务说明: 判断文本的情感, 标签为「正面 / 中性 / 负面」。

示例1 “这家餐厅很好吃” → 正面

示例2 “等了太久, 体验很差” → 负面

⋮

新输入 “包装不错, 但味道一般” → ?

Few-shot 后,
模型更容易回答:



中性
(或偏中性)

💡 通过示例, 模型知道标签体系和输出格式, 因此更容易给出符合预期的答案。

5 Few-shot 能帮什么?



帮模型快速理解任务
通过示例明确任务目标与约束。



统一输出格式与风格
让结果更规范、一致、易解析。



降低歧义, 提升稳定性
减少理解偏差, 输出更可靠。



对分类、抽取、改写、
结构化输出尤其有用
提升复杂任务的完成度。



6 为什么有效?

大模型在预训练中学到了大量语言模式与知识。Few-shot 通过上下文示例激活相关模式, 让模型在推理时模仿示例规律完成新任务。这是一种“上下文学习 (in-context learning)”。



示例
(输入 → 输出)



模式识别
(发现规律)



规律在上下文中激活



新任务泛化
(完成新问题)

7 局限与挑战

- ⚠️ 示例写得差会误导模型
- ⚠️ 示例太多会占用 context window
- ⚠️ 任务过于复杂时效果有限
- ⚠️ 示例分布不合理可能带来偏差



8 写 Few-shot 的实用建议

- 1 示例要和目标任务高度一致
- 2 保持输入输出格式统一
- 3 示例要覆盖典型情况和边界情况
- 4 控制数量, 避免冗长
- 5 如需高可靠性, 可结合思维链、检索或工具使用



总结



Few-shot 的本质: 用少量示例, 告诉模型“这个任务应该怎么做”。



示例提供模式
(输入 → 输出)



模型在上下文中学习
(理解并模仿规律)



新任务输出更稳定
(更准确、更一致)



一句话记忆点:
不是重新训练,
而是临时示范。



大模型世界中的 Chain-of-thought

让模型一步一步思考，提升复杂任务表现



一步一步，
思路更清晰，
答案更可靠！

1 什么是 Chain-of-thought?

Chain-of-thought (CoT, 思维链) 是一种提示或推理方式，让模型把复杂问题拆成多个中间步骤，按步骤分析后再得出结论。它不是重新训练模型，而是在回答时显式或隐式地引导“逐步思考”。



我会这样想：

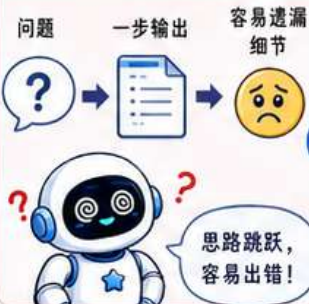
- 1 理解题目
- 2 拆解任务
- 3 逐步推导
- 4 得出结论

2 它在解决问题时的结构



3 直观对比：直接回答 vs. Chain-of-thought

直接回答 (一步到位)



Chain-of-thought (分步思考)



VS.

4 一个简单例子



题目：小明有 12 支铅笔，送给同学 4 支，又买了 7 支，现在有多少支？

直接回答 (传统做法)

快速计算，一步得出
 $12 - 4 + 7 = 15$

答案：15 支



Chain-of-thought (分步思考)

步骤1: 先算送出后剩余: $12 - 4 = 8$
步骤2: 再加上新买的: $8 + 7 = 15$
步骤3: 得到答案: 15 支

答案：15 支



5 为什么有效?



帮助模型把复杂任务拆小，降低跳步错误



强化中间逻辑，提升推理透明度



对数学、逻辑、规划，多步任务更有帮助



与多次采样、验证结合时，效果更稳健



6 常见形式

Few-shot CoT



例1: ...
一步步...
...
例N: ...
一步步...

Zero-shot CoT

让我们
一步一步思考。

只加一句“让我们
一步一步思考”

Self-consistency



生成多条思路，
再选更一致答案

Reason + Verify



先推理，
再检查或修正

7 局限与挑战



思考步骤变长，会消耗更多推理时间与算力



中间步骤不一定都正确，可能出现“看起来合理”的错误推理



对特别简单的问题，过度思考未必划算



推理质量仍取决于模型能力与提示设计



8 实用建议



问题越复杂，越适合使用 CoT



提示里明确要求“分步解释”或“先分析再作答”



对关键任务加入验证步骤



对输出长度和格式做约束，避免冗长混乱



总结



Chain-of-thought 的本质：把复杂问题拆成步骤，让模型在推理过程中更有条理。



不是每个问题
都要想很久，
但复杂问题往往
值得多想一步。



大模型世界中的 Structured outputs

让模型按规定格式输出，结果更稳定、更易集成



让模型输出
“标准的结果”，
程序更好用！

1 什么是 Structured outputs?

Structured outputs 指让大模型按照预先规定的结构返回结果，而不是自由散文式回答。常见形式包括 JSON、数组、键值对、表格字段、枚举标签。

重点不是“说得像 JSON”，而是“输出真正符合指定结构与约束”。

自由文本（不受约束）

这是一笔来自张三的订单，买了2件T恤，总价199元，状态已发货。

Structured outputs（结构化）

```
{  "customer_name": "张三",  "item": "T恤",  "quantity": 2,  "total_price": 199,  "status": "已发货"}
```

2 它和普通回答有什么区别？

普通输出（自由文本）

- ✗ 格式不固定
- ✗ 字段可能缺失
- ✗ 难以程序解析
- ✗ 同一问题可能写法不同

VS.

Structured outputs（结构化输出）

- ✓ 字段固定
- ✓ 类型清晰
- ✓ 方便校验与解析
- ✓ 更适合系统集成



3 一个直观例子

用户请求

请提取这条订单信息

自由文本输出（不便程序使用）

这是一笔来自张三的订单，买了2件T恤，总价199元，状态已发货。

Structured outputs（结构化输出）

```
{  "customer_name": "张三",  "item": "T恤",  "quantity": 2,  "total_price": 199,  "status": "已发货"}
```

程序更容易使用

4 常见约束有哪些？

字段名固定



字段名一致，
便于映射与解析

数据类型明确
（字符串/数字/
布尔值）

T 123 true

类型清晰，
减少歧义

必填字段



必须存在，
保证完整性

枚举值范围
（如 positive /
neutral / negative）



限定可选值，
提高一致性

嵌套结构与数组



支持层级结构与
列表数据

可以配合 schema / JSON schema / function calling / tool calling 使用。

5 为什么重要？



便于前后端
和工作流系统
直接消费



降低解析错误
与人工清洗
成本



提高稳定性，
一致性与
可测试性



适合抽取、
分类、打标、
生成 API 参数



是 Agent、
自动化流程、
数据管道的重要基础

6 常见应用场景

信息抽取



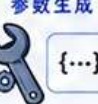
表单填充



文本分类

正面
中性 负面

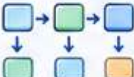
调用工具
参数生成



实体识别

人名：张三
地名：北京
组织：ABC 公司

多步骤工作流中的
中间结果



7 局限与挑战

- ⚠ 结构对了，不代表内容一定对
- ⚠ schema 太复杂会增加生成难度
- ⚠ 字段可能为空、冲突或语义不完整
- ⚠ 仍需要校验、重试和异常处理
- ⚠ 不同模型对严格约束的支持程度不同



8 实用建议

- ✓ 先定义清晰 schema
- ✓ 字段名简短明确
- ✓ 限制可选值范围
- ✓ 区分 required / optional
- ✓ 让模型只输出目标结构
- ✓ 结合校验器、重试和后处理
- ✓ 必要时给 few-shot 示例



总结



Structured outputs 的本质：

不是让模型“自由发挥”，而是让它把答案
装进一个可验证、可解析、可编排的结构里。

先定格式，
再让模型填内容。



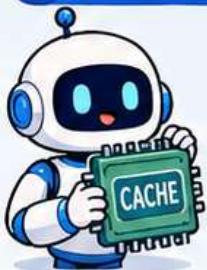


『大模型世界中的 Prompt caching』

把重复上下文缓存起来，让推理更快、更省



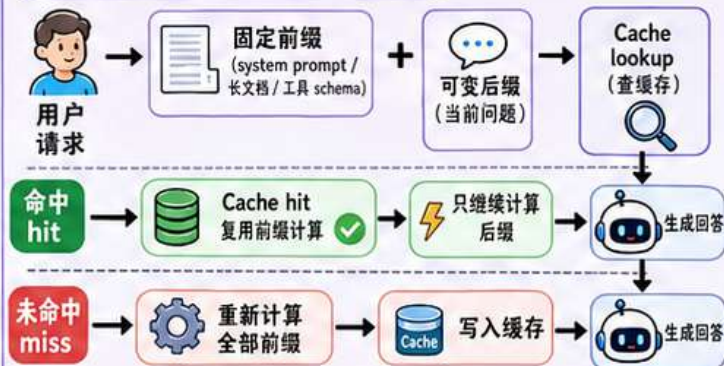
1 什么是 Prompt caching?



Prompt caching 是在推理时，把重复出现的提示前缀（如 system prompt、固定说明、工具定义、长文档上下文）的已计算结果缓存起来。下次遇到相同前缀时，直接复用缓存，避免重复计算。

★ 重点：缓存的**不是最终答案**，而是前缀的**中间计算状态/已处理结果**。

2 它在请求流程中的位置



3 直观对比：不用缓存 vs 使用缓存

不用缓存（每次重算前缀）



每次都从头计算
长前缀

耗时更长

算力消耗更多

费用更高

使用缓存（复用前缀）



复用已处理前缀

延迟更低

吞吐更高

成本更省

★ 前缀越长、重复越多，收益通常越明显。

4 一个简单例子



企业客服机器人场景

固定前缀（每次大量重复使用）

- 品牌规则
- 退换货政策
- 商品目录
- 输出格式要求

第一次请求（完整计算）

用户A：
「这款耳机支持退货吗？」

后续请求（命中缓存）

用户B：
「运费是谁承担？」

后续请求（命中缓存）

用户C：
「退货需要多久审核？」

结论：问题不同，但前缀相同，可以复用缓存。

5 为什么重要？

- 降低首词时间和整体延迟
- 减少重复计算，节省推理成本
- 提升高并发场景吞吐量
- 让长上下文应用更实用
- 对 Agent、RAG、企业应用尤其有价值

6 常见应用场景

RAG 问答：

固定知识库
片段反复使用



AI Agent：

工具说明、函数
schema、工作流
提示长期不变



代码助手：

仓库说明、编码规范、
API 文档
重复出现



企业 Copilot：

角色设定、制度文档、
审批规则固定



批量处理任务：

同一模板下
处理多条
不同输入



7 局限与挑战

- 只有前缀相同或足够一致时，命中率才高
- 动态内容太多会降低缓存收益
- 不同平台的缓存策略、TTL、计费方式可能不同
- 需要考虑缓存失效、版本更新与一致性
- 涉及敏感数据时要关注隐私与安全



8 实用建议

- 关注 cache hit rate（命中率）、latency（延迟）、cost（成本）
- 把稳定不变的说明尽量放在前缀位置
- 将变化内容放在后面，提升命中率
- 为系统提示、工具定义、schema 做模块化复用
- 不要把 Prompt caching 误解为“模型记住了一切”

总结

重复前缀



缓存已处理结果



减少重复计算



更快、更省、更适合
长上下文应用



★ 记忆点：Prompt caching 的本质：
不是改变模型能力，而是避免把
同样的前缀一算再算。





大模型世界中的 Meta prompting

—— 让模型先设计提示词，再更好地完成任务 ——

1 什么是 Meta prompting?

- Meta prompting 指用一个提示词来指导、生成或优化另一个提示词，或让模型先思考“该如何提问、如何组织任务”，再进入正式回答。
- 它不是重新训练模型，而是在提示词层面进行任务编排与增强。
- 目标：更清晰的提示词 → 更稳定、更高质量的输出。

先想怎么问，
再想怎么答！



2 它在任务流程中的位置



可在任务开始时执行一次，也可在多步工作流程中动态调用。

3 普通 prompting vs. Meta prompting

普通 prompting

直接给任务，让模型直接做



- ❌ 规则不清、边界模糊
- ❌ 格式不稳、结构不一致
- ❌ 效果依赖写法与经验
- ❌ 复用性差、难以迁移

VS.

Meta prompting

先生成或优化提示词，再执行任务



- ✅ 目标清晰、约束明确
- ✅ 格式稳定、结构一致
- ✅ 效果更稳、更可控
- ✅ 易复用、可迁移

4 一个直观例子



5 为什么有用?



帮助拆解
复杂任务



提升提示词
质量与一致性



便于模板化、
自动化和复用

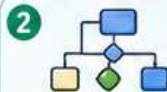


对 Agent、
工作流、批量任务
尤其有价值

6 常见应用场景



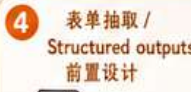
1 Prompt 优化器 /
自动提示词生成
根据目标自动生成
更优提示词模板。



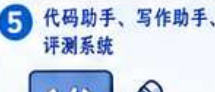
2 多步 Agent 规划
让模型先规划步骤、
分配任务，再逐步
执行。



3 RAG 中根据问题
动态构造检索与
回答提示
针对问题定制检索
与回答策略。



4 表单抽取 /
Structured outputs
前置设计
先设计字段、格式
与约束，再抽取。



5 代码助手、写作助手、
评测系统
更好地制定规范、
生成内容与评测标准。

7 局限与挑战

- ⚠️ 多一层 prompting，会增加 token 与延迟
- ⚠️ 若 meta prompt 本身设计差，可能放大问题



- ⚠️ 可能出现“提示词看起来好，但任务结果一般”
- ⚠️ 仍需要人工评估与迭代

8 实用建议

- ✅ 先明确目标、受众、输出格式。
- ✅ 让 meta prompt 产出可复用模板。
- ✅ 同时要求评价标准或检查清单。
- ✅ 对复杂任务可结合 few-shot、CoT、structured outputs。
- ✅ 区分“生成提示词”和“执行任务”两个阶段。



总结



Meta prompting 的本质：

不是直接让模型做事，而是先让模型参与“提示词设计”，再提高执行质量。



一句话记住：
用提示词
管理提示词。





太长的内容，
先分块，再处理！

大模型世界中的 Chunk

把长内容切成可处理的小块，是检索与理解的重要基础



1 什么是 Chunk?

Chunk (文本块/内容块) 是把长文档、长对话或知识材料，按照一定规则切分成较小片段的单位，便于模型进行存储、检索、嵌入、召回和上下文组织。

★ 它不是随便截断，而是为了让每一块既足够小，方便处理；又保留足够语义，方便理解。



2 为什么要分 Chunk?

- 长文档太大，不能一次高效处理
- Embedding 与向量检索通常按块进行
- 更容易找到和问题最相关的片段
- 提高 RAG、知识库问答、检索增强的效果



3 一个直观例子

一篇“公司报销制度”长文档



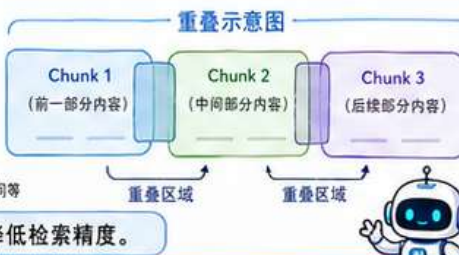
结论：模型不必阅读整篇，只需优先取回最相关的块。

4 常见切分方式

- 固定长度切分**
按字数、token 数切。
- 按段落切分**
保留自然语义边界。
- 按标题层级切分**
按章节、标题组织内容。
一、第一章
1.1 标题
1.2 标题
...
- 滑动窗口切分**
相邻 chunk 有重叠。
前一块 后一块
- 语义切分**
根据语义变化点切分内容。

5 Chunk 的关键参数

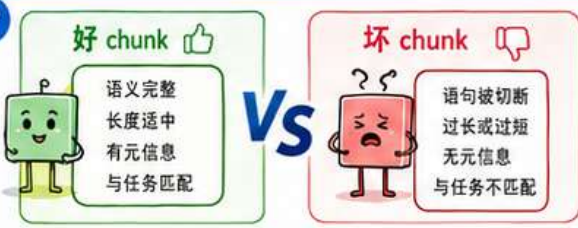
- Chunk size (块大小)** 每块包含多少内容 (字数/Token 数)
- Overlap (重叠)** 相邻块重叠多少，减少信息断裂
- Boundary (边界)** 在哪里切分 (句子、段落、标题等)
- Metadata (元数据)** 块的来源信息：文档名、页码、章节、时间等



切得太小会丢上下文，切得太大又会降低检索精度。

6 好的 Chunk 应该满足什么?

- ✓ 语义完整：不要把关键句子硬切断
- ✓ 长度适中：便于嵌入与检索
- ✓ 易于定位：最好带页码、标题等元信息
- ✓ 与任务匹配：不同任务适合不同切分策略



7 局限与挑战

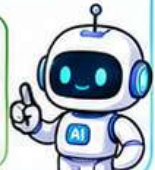


- ⚠ 切分不合理会导致检索不到答案
- ⚠ 多跳问题可能需要多个 chunk 组合
- ⚠ 表格、代码、图文混排内容更难切
- ⚠ 不同模型与任务，对 chunk 策略要求不同



8 实用建议

- 先按段落或标题切，再观察效果。
- 常尝试 chunk size + overlap 组合。
- 为 chunk 保留清晰 metadata。
- 用真实问题测试召回质量。
- 对表格、代码、FAQ 可单独设计切分规则。



总结

Chunk 的本质：把大内容拆成“可检索、可表示、可组合”的小单元，让模型更容易找到对的信息。



不是切得越碎越好，而是切得刚刚好。



向量让机器“看懂”语义，而不只是字面！

大模型世界中的 Vector database



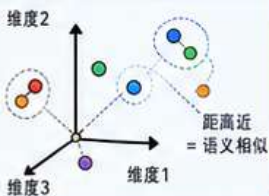
核心思想：存储和检索向量，让模型能找到语义上最相似的信息。

1 什么是 Vector database?

Vector database (向量数据库) 是一种专门为“向量 (embedding)”设计的数据库，用于高效地存储、索引和检索高维向量。

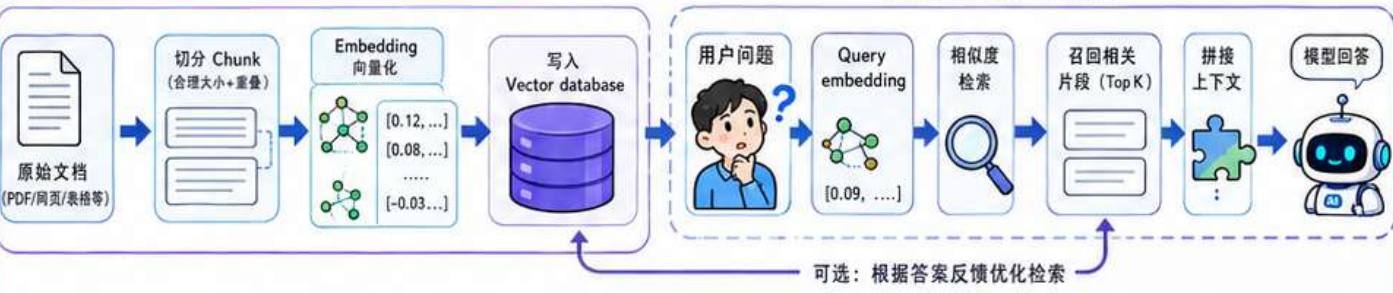
把文本、图片、音频等内容转成向量后，向量在空间中的位置代表了它的语义。

高维向量空间示意 (仅3维示意)



★ 数字不同，语义相近的内容在空间中距离更近！

2 它在 LLM / RAG 流程中的位置



检索与生成 (在线)

3 一个直观例子 (公司报销规则查询)



4 核心组成



5 常见检索方式



6 为什么重要?

- ✓ 语义检索：理解“意思相近”，而非只匹配关键词。
- ✓ 提升 RAG 召回：找到更相关的片段，回答更准确。
- ✓ 可扩展性强：支持百万、千万级向量的高效检索。
- ✓ 低延迟：配合 ANN 索引，毫秒级返回 Top K。
- ✓ 支持多模态：文本、图片、音频都能转成向量检索。
- ✓ 增强模型可靠性：提供更有事实依据的上下文，降低幻觉，提升可追溯性。

7 局限与挑战

- ⚠ 依赖向量质量：embedding 模型差，检索效果差。
- ⚠ 切分不当影响召回：太大丢细节，太小丢语义。
- ⚠ 索引更新复杂：大规模数据增删会影响性能。
- ⚠ 未必万能：极度精确问题仍需关键词或结构化查询。
- ⚠ 元数据与过滤：需要合理设计，才能精准定位。
- ⚠ 延迟与成本权衡：更高召回往往带来更高计算成本。
- ⚠ 检索 ≠ 无幻觉：检索到相关不等于模型一定不编造。

8 实用建议

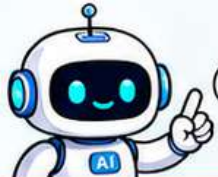
- ✓ 选择合适的 embedding 模型 (领域匹配、效果优先)。
- ✓ 合理设计 Chunk 大小与重叠 (如 300~800 字, 10%~20% 重叠)。
- ✓ 保留丰富元数据 (来源、时间、标题、标签等)。
- ✓ 向量检索 + 关键词 / 过滤 + 重排 (Rerank) 提升精度。
- ✓ 监控召回率 (Recall) 与命中质量，持续优化索引与切分。
- ✓ 数据变更及时刷新新向量与索引，保持时效性。

总结

★ Vector database 的本质：不是按字面匹配内容，而是按“语义位置”找到最相近的信息。



★ 把知识变成向量，再按相似度把相关内容找回来！



不仅看字面，
还要懂语义！

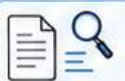
大模型世界中的 Hybrid search



把关键词检索和语义检索结合起来，让召回更准、更全。

1 什么是 Hybrid search?

Hybrid search 将传统的词法检索（如关键词 / BM25）与向量语义检索结合起来，使系统既能考虑词面匹配，也能考虑语义相似度。它在 RAG 和企业搜索中非常常见。



关键词匹配
(BM25)



语义匹配
(向量检索)



混合检索
(Hybrid search)

2 为什么需要 Hybrid search?



只做关键词检索：

精确命中中强，但同义词、改写、
口语问法容易漏掉。



只做向量检索：

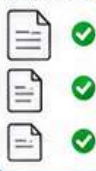
能找语义相近内容，但有时对精确术语、
编号、专有名词不够敏感。



混合后：

兼顾 precision 与 recall。

关键词检索



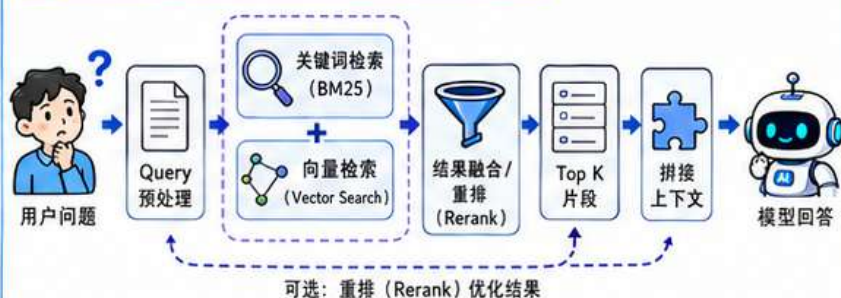
向量检索



混合检索



3 它在 LLM / RAG 流程中的位置



4 一个直观例子

用户问题：
差旅报销里，
高铁二等座和
住宿标准怎么
规定？



关键词检索 (BM25)

命中包含精确术语的文档

- 《差旅报销制度》第3.2条：
高铁二等座标准执行。
- 《住宿标准说明》：
住宿标准：一线城市不超
过800元/晚。
- 《费用报销 FAQ》：
高铁二等座报销范围。

向量检索 (语义检索)

命中语义相关的文档

- 《交通标准》：
交通工具报销规则说明。
- 《酒店报销规则》：
住宿发票与标准要求。
- 《出差管理细则》：
差旅交通与住宿注意事项。

混合检索结果 (Top)

综合更全面的 Top 结果

- 《差旅报销制度》第3.2条
- 《住宿标准说明》
- 《交通标准》
- 《酒店报销规则》

★ 既能抓住精确术语，也能补充语义相关内容。

5 核心组成



Lexical search (关键词 / BM25)
擅长精确词匹配



Vector search (Embedding)
擅长语义相似



Fusion (融合)
合并两路召回结果



Rerank (重排)
按更强模型再次排序



Metadata filter (过滤)
按时间、部门、标签等筛选

6 常见融合方式



Score fusion (分数融合)
对关键词分数和向量分数加权组合。

$$\text{最终分数} = \alpha \times \text{关键词分数} + \beta \times \text{向量分数}$$
$$(\alpha + \beta = 1)$$



Rank fusion (排序融合，如 RRF)
根据排名而不是原始分数融合。

$$\text{RRF 分数} = \sum_k \frac{1}{(k + \text{rank}_i)}$$

k 为平滑因子



先召回后重排

多路召回后，再用 cross-encoder
或更强 reranker 重排。

多路召回
(BM25 + 向量)

候选集合

Reranker
(Cross-encoder)

Top K
结果

7 优势与价值

- ✓ 提高召回率，减少漏检
- ✓ 对同义词、改写、口语问法更稳健
- ✓ 对编号、术语、专名仍保持敏感
- ✓ 更适合企业知识库、FAQ、代码文档、
政策制度检索
- ✓ 提升 RAG 回答质量与可解释性



8 局限与挑战

- ⚠ 系统更复杂，调参更多
- ⚠ 不同检索分数难直接比较
- ⚠ embedding、BM25、chunk、rerank
之间会相互影响
- ⚠ 成本、延迟可能上升
- ⚠ 数据质量差时，混合也不能完全救场



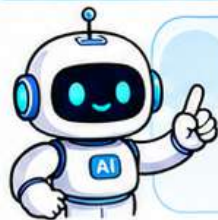
9 实用建议

- ✓ 先做好 chunk 切分与 metadata
- ✓ 关键词检索和向量检索都要单独评估
- ✓ 调整 topK、融合权重和 rerank 策略
- ✓ 对专有名词密集场景提高 lexical 权重
- ✓ 对自然语言问答场景加强 semantic 检索
- ✓ 用真实问题集评估 Recall、MRR、命中率
与最终回答质量



10 总结

★ Hybrid search 的本质：
不是在关键词检索和向量
检索之间二选一，
而是让二者协同工作。



问题

关键词检索 (BM25)

+

语义检索 (向量)



融合 / 重排
(Rerank)

更好的上下文
(高相关片段)



更可靠回答

字面要准，
语义也要懂！

Hello!



大模型世界中的 Reranking

先广泛召回，再精细重排，让模型找到更相关的内容。



1 什么是 Reranking?

Reranking (重排序) 是在初步检索之后，对候选结果再次进行更精细的相关性判断与重新排序。它不是从全库里重新搜索，而是在“已召回的少量候选”中挑出最相关的结果。

先召回，再精选。



2 为什么需要 Reranking?

只做召回

找到“可能相关”的结果，但顺序不一定最好。



加上重排

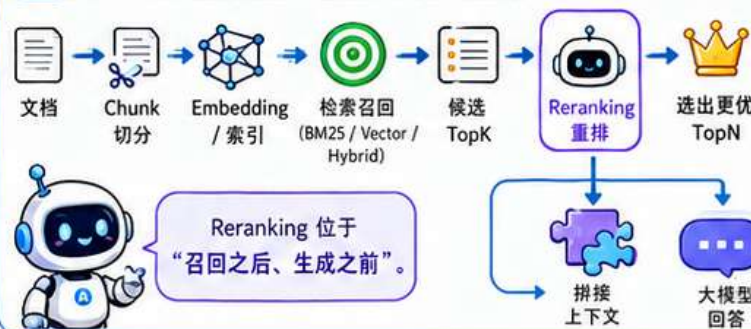
把最相关、最适合回答问题的内容排到前面。



原因:

- ✓ 向量检索或 BM25 擅长快速筛选，但排序可能不够细致。
- ✓ 重排模型能更深入比较“问题”和“候选内容”的匹配度。
- ✓ 对 RAG、企业搜索、问答系统尤其重要。

3 它在 LLM / RAG 流程中的位置



4 一个直观例子

场景：用户问“差旅报销里，住宿费超标怎么办？”

初步召回结果 (Top4)	Reranker 打分	重排后结果 (Top4)
1 员工差旅管理制度总则	0.62	1 住宿超标处理规则 (特殊情况审批)
2 住宿费报销标准说明	0.77	2 住宿费报销标准说明
3 出差交通费报销流程	0.89	3 出差交通费报销流程
4 住宿超标处理规则 (特殊情况审批)	0.95	4 员工差旅管理制度总则

不是只看关键词，而是看谁更能回答当前问题。



5 常见打分思路

- Query-Document Matching**
比较问题和候选文档的整体匹配程度。
- Cross-encoder**
把“问题 + 候选文本”一起输入模型，精细判断相关性。
- 语义理解**
考虑同义表达、上下文、真实意图。
- 排序得分**
输出一个相关性分数，用于重新排序。
- ⚠️ 重排通常比初步召回更准，但也更慢、更贵。

6 优势与价值

- 🏆 提升 Top 结果质量 ✓
- 🎯 提高 RAG 回答准确率 ✓
- 🗑️ 减少无关上下文干扰 ✓
- 😊 改善用户搜索体验 ✓
- 🔗 能与 BM25、向量检索、混合检索配合使用 ✓

Reranking 提升的不是能不能找到，而是先把最好的送上来。



7 局限与挑战

- ❗ 候选集合太差时，重排也无能为力 (garbage in, garbage out)
- ❗ 候选数量越多，计算成本越高
- ❗ 延迟增加，不适合所有实时场景
- ❗ 重排模型质量会直接影响效果
- ❗ 需要在效果、成本、速度之间权衡



8 常见应用场景



9 实用建议

- 🔍 先做好召回，再考虑重排
- 📄 控制候选数量 (如先召回 Top20/Top50，再重排)
- 📊 结合业务指标评估: Recall, MRR, NDCG、回答质量
- ★ 对高价值问题优先使用重排
- ⚙️ 可与 Hybrid Search, Metadata Filter, Chunk 优化配合

把资源用在最重要的问题上!



10 总结



核心总结:

Reranking 的本质: 不是扩大搜索范围，而是在候选结果里重新判断“谁最值得排在前面”。

记忆点

召回负责“找一些”，
Reranking 负责“排最好”。





我来带你了解!

大模型世界中的 Retrieval eval

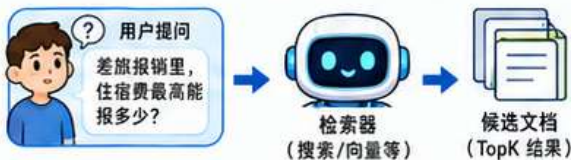


评估检索系统到底找得准不准、找得全不全，是 RAG 效果优化的重要基础。

1 什么是 Retrieval eval?

Retrieval eval (检索评估) 就是评估检索系统能否从知识库中找出“与问题真正相关”的内容。它关注的不是最终生成句子漂不漂亮，而是“检索阶段”是否把对的资料找回来。

在 RAG、搜索、问答系统中都非常关键。



2 它在 LLM / RAG 流程中的位置



Retrieval eval 主要衡量“召回内容质量”，通常发生在离线评测与线上监控中。

3 为什么要做 Retrieval eval?

- 检索不好，后面的生成再强也会“巧妇难为无米之炊”；
- 能帮助比较不同检索方案 (BM25、向量检索、混合检索、重排)；
- 能发现 chunk 切分、embedding、metadata、query 改写的问题；
- 能持续优化 RAG 的准确率、稳定性与可解释性。



4 一个直观例子

用户问：差旅报销里，住宿费最高能报多少？

Top4 检索结果

- A 《差旅报销制度》第3.2条：一线城市不超过800元/晚 **真正相关**
- B 《差旅FAQ》：报销需要发票和住宿清单 **部分相关**
- C 《员工手册》：出差审批流程 **弱相关**
- D 《团建活动说明》：住宿安排 **不相关**

理想情况：A 应该排在前面，B 可以保留，C/D 应靠后。



5 常见评估指标



Recall@K
前 K 个结果里，找回了多少相关内容 (关注找全)



Precision@K
前 K 个结果里，有多少是真的相关 (关注找准)



Hit Rate@K
前 K 个里，是否至少命中一个相关结果



MRR
第一个正确结果排得越靠前越好



NDCG
不仅看有没有找到，还看排序质量和相关性等级 (越高越好)

★ 不同业务更看重的指标不同。

6 怎么做评估?

- 准备评测问题集 (query)
- 为每个问题标注相关文档 / chunk (ground truth)
- 跑检索系统得到 TopK
- 计算指标并横向比较不同方案

离线评测

用标注数据稳定比较方案



在线评测

看点击、人工反馈、答案质量等真实效果



数据集 (问题集 + 标注)



检索系统 (多种方案)



计算指标 (打分)



方案对比 (选择最优)

7 局限与挑战



- 标注 ground truth 成本高；
- 一个问题可能有多个“都对”的片段；
- 文档切分方式会影响评测结果；
- 检索指标好，不等于最终回答一定好；
- 线上用户问题分布会变化，需要持续监控。



8 实用建议

- 先明确业务目标，再选指标；
- 不只看单一指标，要组合看 Recall、MRR、NDCG；
- 对 FAQ、政策问答、企业知识库等分别建测试集；
- 评估检索时同时观察 chunk size、overlap、metadata；
- 比较 BM25、向量检索、Hybrid search、Reranking 的效果；
- 定期复盘失败样本，比只看平均分更有价值。



9 总结



★ Retrieval eval 的本质：不是评估模型会不会说，而是评估系统能不能先找对。

先找对资料，再谈生成质量！





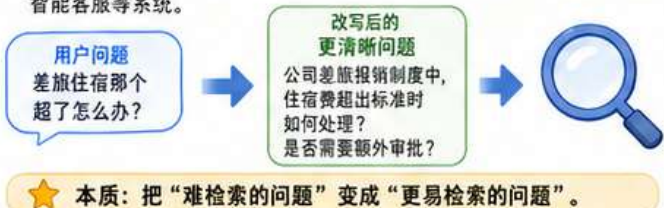
大模型世界中的 Query rewrite

先把问题改写得 clearer、更适合检索，再去找答案。



1 什么是 Query rewrite?

- Query rewrite (查询改写) 是指在正式检索前，对用户原始问题进行重写、补充、澄清或结构化。
- 目标不是改变用户意图，而是让检索系统更容易找到相关内容。
- 它常见于搜索、RAG、企业知识库问答、智能客服等系统。



2 它在 LLM / RAG 流程中的位置



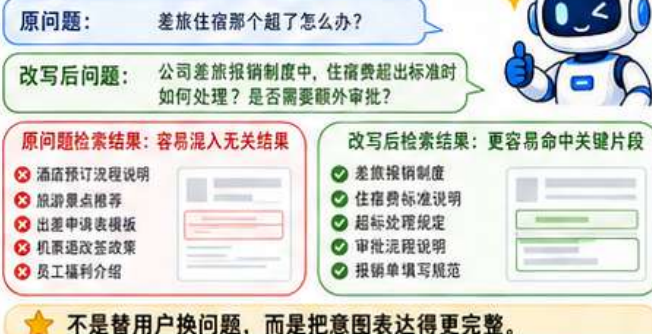
- Query rewrite 位于“检索之前”。
- 它主要优化“召回质量”，而不是直接生成最终答案。

3 为什么要做 Query rewrite?

- 用户原问题可能太短、太模糊、口语化。
- 问题里可能缺少检索所需关键词。
- 同义词、缩写、别名会导致漏检。
- 多轮对话中的代词 (它、那个、前面提到的) 需要补全上下文。



4 一个直观例子



5 常见改写方式

- 澄清补全: 补足省略的主语、对象、条件。
- 术语展开: 把缩写、简称改成完整说法。
- 同义词扩展: 加入近义表达，提高召回。
- 指代消解: 把“它/这个/那个”替换成明确对象。
- 结构化改写: 把自然语言问题改写成更规范的检索表达。

6 它和 Reranking / Retrieval 有什么区别?

环节	目标	发生时机	主要作用
Query rewrite	改写问题	检索前	让检索系统更容易理解用户意图，提升召回质量
Retrieval (检索)	召回候选文档	检索时	从索引中找到与查询相关的候选文档 (TopK)
Reranking (重排序)	排序优化	检索后	对候选文档重新排序，挑选最相关的结果

Query rewrite 优化的是“怎么问”，Reranking 优化的是“怎么排”。

7 优势与价值

- 提升 Recall，减少漏检。
- 改善长尾问题、口语问题、内部术语问题的检索效果。
- 让混合检索 (keyword + semantic) 表现更稳定。
- 有助于多轮对话问答和企业知识库场景。
- 往往是低成本、收益明显的 RAG 优化手段。



8 局限与挑战

- 改写过头可能偏离用户原意。
- 引入错误关键词会造成误召回。
- 对领域知识依赖较强，特别是企业专有术语。
- 多次改写会增加延迟与系统复杂度。
- 需要结合评估指标验证，不是改写越多越好。



9 实用建议 + 总结

- 先只改写一次，避免链路过长。
- 尽量保留用户核心意图，不随意改问题。
- 可让模型输出“改写后查询 + 改写理由”。
- 对 FAQ、制度、技术文档等场景分别设计模板。
- 配合 Retrieval eval 观察 Recall、MRR、命中率变化。



Query rewrite 的本质：先把问题问对，再把资料找对。



大家好！
我来带你了解
GraphRAG！

大模型世界中的 GraphRAG



把知识图谱与检索增强生成结合起来，让模型不仅找到“相关内容”，还理解“内容之间的关系”。

1 什么是 GraphRAG？

- GraphRAG（图检索增强生成）是在传统 RAG 基础上，引入“图结构 / 知识图谱 / 实体关系网络”的方法。
- 它不只检索孤立文本片段，还会利用“实体、概念、事件、它们之间的关系”来组织和召回信息。
- 本质：从“只找相似文本”，升级为“沿着知识关系找答案”。



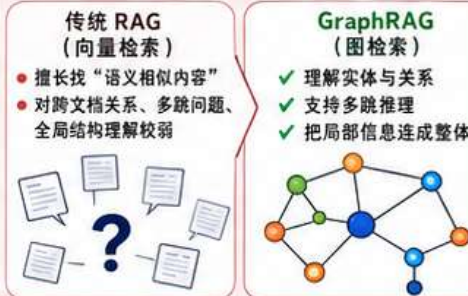
2 它在 LLM / RAG 流程中的位置



GraphRAG 重点增强的是“检索与上下文组织阶段”。

3 为什么需要 GraphRAG？

传统 RAG vs GraphRAG 对比



GraphRAG 更适合回答：

- 多跳推理问题
- 实体关系问题
- 需要全局摘要的问题
- 跨文档关联分析问题

不仅要知道“A是什么”，
还要知道“A与B、C、D
有什么关系”。

4 一个直观例子：企业知识库问答

用户问题：某产品上线延迟，主要受哪些系统依赖和团队协作影响？

文档中分散提到：

- 产品A上线时间变更两次
- 依赖支付系统接口稳定性
- 依赖风控系统实时校验
- 移动端团队排期紧张
- 后端团队人力不足
- 测试环境不稳定导致测试延迟

GraphRAG 构建的关系图 (示意)



GraphRAG 的回答 (关系梳理结果)：

- ✅ 上线延迟主要由以下因素共同导致：
- 1) 依赖的支付系统接口稳定性不足
 - 2) 依赖的风控系统实时校验延迟
 - 3) 移动端团队排期紧张
 - 4) 后端团队人力不足
 - 5) 发布计划发生两次变更
- 这些因素通过依赖关系和团队协作相互影响，最终导致上线延迟。

5 核心组成 (关键环节)



6 GraphRAG 能解决什么？

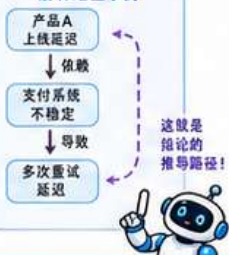
- 多跳问答：沿关系链逐步找到答案
- 全局总结：从局部片段上升到整体结构
- 关系发现：识别人物、部门、产品、事件之间的联系
- 溯源分析：帮助分析“为什么会这样”
- 复杂知识库导航：让长文档、多文档更易探索



7 优势与价值

- 更适合复杂推理和关系型问题
- 召回结果更有组织，不只是相似片段堆积
- 有助于减少遗漏关键连接信息
- 提升企业知识库、研究分析、情报梳理等场景效果
- 让回答更具解释性：可以展示“这条结论是沿哪些关系得出的”

解释路径示例



8 局限与挑战



- 构图成本高：实体和关系抽取并不简单
- 图质量影响大：错边、漏边会影响结果
- 系统更复杂：存储、检索、更新、维护都更难
- 延迟和成本可能上升
- 并不是所有问题都需要 GraphRAG，简单事实问答未必值得使用



9 实用建议

- 先判断业务是否真的需要“关系推理”
- 不要放弃基础检索，常与向量检索 / 关键词检索结合
- 优先保证实体标准化和关系定义清晰
- 对图谱做摘要、社区聚类、热点节点提炼
- 用 Retrieval eval 评估召回质量，而不是只看最终回答
- 在企业、科研、法律、金融、代码知识库等复杂场景中更有价值



10 总结

一句话总结：
GraphRAG 的本质，不只是“找相关文本”，
而是“利用知识关系组织检索和推理上下文”。

GraphRAG 思维流程



记忆点：
RAG 找内容，
GraphRAG 还帮你
找内容之间的联系。





大模型世界中的 Function calling

让模型不仅会回答，还会按规则调用工具完成任务



1 什么是 Function calling?

Function calling (函数调用/工具调用) 是让大模型在回答问题时，不只是生成自然语言，还能按照预先定义好的函数描述，选择合适工具并输出结构化调用参数，由外部系统执行后再把结果返回给模型。

强调：它不是模型自己真的执行代码，而是“模型决定调用什么、传什么参数”。



2 它在对话流程中的位置



Function calling 发生在“生成最终答案之前”的工具协作阶段。

3 为什么需要 Function calling?

- 弥补模型无法直接访问外部世界的问题
- 连接数据库、搜索、天气、日历、支付、企业系统等工具
- 提高结果可执行性与自动化程度
- 降低自由文本解析的不稳定性
- 是 Agent、工作流、自动化系统的重要基础



从“会说”到“会做”

4 一个直观例子

示例问题：「帮我查一下明天下午北京天气，并提醒我带伞。」

没有 Function calling

模型只能泛泛建议，不能真正查天气或创建提醒。

明天下午北京可能会有雨，记得带伞哦~
建议你查看天气APP并设置提醒。

有 Function calling

- Step1 调用 get_weather(city=北京, date=明天)
 - Step2 根据结果判断可能下雨
 - Step3 调用 create_reminder(time=明天下午, content=带伞)
 - Step4 输出最终答复：已帮你查询并创建提醒。
- 已查询到明天下午北京有雨，已为你创建“提醒：明天下午带伞”。

5 Function calling 的核心组成

Tool / Function 名称 (工具或函数的唯一标识)

Description (功能说明) 清晰描述该工具的作用与使用场景

Parameters (参数 schema) 结构化定义输入参数及类型

Required fields (必填字段) 明确标注必填字段，确保参数完整

示例 (JSON / Schema)

```
{
  "name": "get_weather",
  "description": "获取指定城市在指定日期的天气信息。",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {"type": "string", "description": "城市名称"},
      "date": {"type": "string", "description": "日期，格式 YYYY-MM-DD 或类似自然语言"}
    },
    "required": ["city", "date"]
  }
}
```

Model decision (模型决定是否调用) 模型根据意图与工具描述决定是否选择调用

Tool result (工具返回结果) 外部系统执行后返回结果给模型

模型生成的调用 (示例)

```
{
  "name": "get_weather",
  "arguments": {
    "city": "北京",
    "date": "明天"
  }
}
```

6 它和普通回答有什么区别?

维度	普通回答	Function calling
输出形式	自由文本	结构化调用 (JSON 等)
可执行性	难以直接执行	便于程序直接执行
系统集成	需再解析，自然语言不稳定	标准化接口，易集成
可控性	模型自由发挥	按接口规范，可验证可追踪
适用场景	一般问答、知识类	与外部系统协作、自动化任务

不是让模型直接“自由发挥”，而是让它按接口规范与外部系统协作。

7 常见应用场景



8 局限与挑战



9 实用建议

- 工具职责要单一清晰
- 参数 schema 要明确，字段命名统一
- 区分 required / optional
- 对关键参数做校验与兜底
- 对工具返回结果做重试和异常处理
- 配合 few-shot、structured outputs、RAG 效果更好
- 记录调用日志，便于评估和优化

好的设计 + 好的规范 = 稳定可靠的工具协作体验!



10 总结

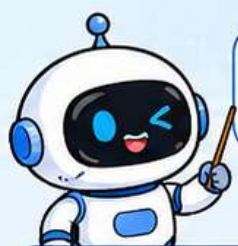


Function calling 的本质:

不是让模型自己去“做工具”，而是让模型学会“什么时候调用什么工具，并按规则传参”，从而把语言能力连接到真实系统能力。

大模型负责决策，工具负责执行。





思考与行动
双轮驱动，
让 AI 更聪明！

大模型世界中的 ReAct

让模型一边思考，一边行动：Reason + Act 的经典框架

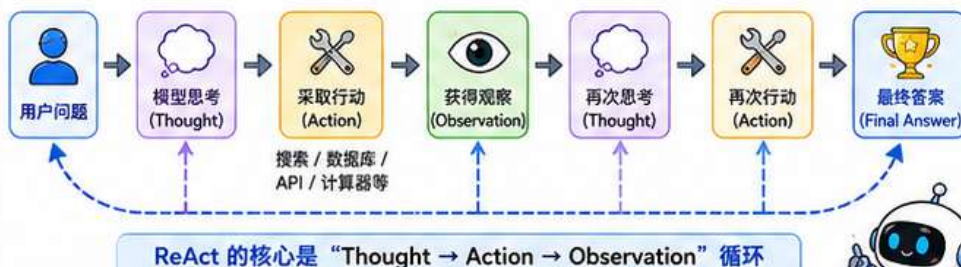


1 什么是 ReAct?

ReAct = Reason + Act,
是让模型在解决复杂任务时，
交替进行“推理 (Thought/
Reason)”与“行动 (Action)”
的方法。不是只在脑中想，也
不是盲目调用工具，而是边分
析边操作，再根据观察结果
继续推理。



2 它在 Agent / 工具调用流程中的位置



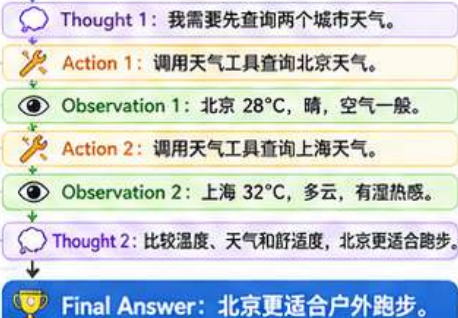
3 为什么需要 ReAct?

- ✓ 复杂问题往往不能一步到位
- ✓ 需要先查资料、再判断
- ✓ 工具调用结果会影响后续推理
- ✓ 可减少幻觉，提高任务完成率

★ 先想一步，再做一步。

4 一个直观例子

示例问题：
“帮我比较北京和上海今天的天气，并告诉我更适合户外跑步的城市。”



5 ReAct 的核心组成



6 它和普通 Chain-of-thought / Function calling 有什么区别?

方式	特点	是否真正操作外部工具	是否展示多步推理策略
Chain-of-thought (CoT)	偏重“推理过程”，不一定真的操作外部工具	不一定	是
Function calling	偏重“按规则调用工具”，不一定展示多步推理策略	是	不一定
ReAct	把多步推理和工具行动组合起来，形成闭环	是	是

★ ReAct 更适合复杂、多步、需要外部信息的任务。

7 优势与价值

- ✓ 适合多步任务与开放问题
- ✓ 能利用外部知识减少过时信息问题
- ✓ 提升可解释性：可以看到模型如何想、如何做
- ✓ 对 Agent、RAG、自动化 workflow 很重要
- ✓ 往往比只推理或只调用工具更稳健

8 局限与挑战

- ◆ 步骤更多，延迟和成本更高
- ◆ 工具质量差会拖累整体效果
- ◆ 推理链可能冗长或出错
- ◆ 需要设计好工具接口、停止条件和安全约束
- ◆ 错误行动可能把问题越带越偏



9 常见应用场景



10 总结



ReAct 的本质：不是让模型只会想，也不是只会做，而是让它在“推理—行动—观察”的循环中，更可靠地完成复杂任务。

🎯 记忆点：先思考下一步，再执行下一步。



先想清楚
再动手，
事半功倍！

大模型世界中的 Planning



让模型先做计划，再分步执行：复杂任务中的规划能力

1 什么是 Planning?

Planning (规划) 是模型在执行任务前，先把目标拆成步骤、安排顺序、决定需要哪些工具或信息，再开始执行。

它不是直接做答案，而是“先想路线图，再行动”。



2 它在 Agent / 工具调用流程中的位置



3 为什么需要 Planning?

- ✓ 复杂任务不能一步到位
- ✓ 长任务需要分阶段完成
- ✓ 多工具任务需要安排顺序
- ✓ 有助于减少遗漏与重复劳动
- ✓ 提高复杂任务的完成率与可控性

★ 先规划，再执行。

4 一个直观例子

例子：“帮我规划一次去东京出差的两天行程，并控制预算。”

没有规划时

想到哪做哪，
信息杂乱：

- ✗ 航班还没查
- ✗ 酒店没对比
- ✗ 行程冲突
- ✗ 预算超支...



有规划时

- ✓ 步骤1 明确目标与约束 (时间/预算)
- ✓ 步骤2 查询航班与酒店
- ✓ 步骤3 安排行程顺序
- ✓ 步骤4 汇总预算
- ✓ 步骤5 输出最终计划



Planning 让任务更像项目管理，有章可循、可控可追踪！

5 Planning 的核心组成



6 常见 Planning 方式



不同任务场景，选择合适的规划方式更有效。

7 它和 Chain-of-thought / ReAct / Function calling 有什么区别?

	Planning	Chain-of-thought (CoT)	ReAct	Function calling
关注点	先做什么、后做什么 (路线与顺序)	如何一步步推理 (思考过程)	推理-行动-观察的循环 (执行循环)	按接口规范调用工具 (接口层)
核心内容	任务拆解、顺序安排、工具与资源选择	逐步推理、中间思考链条	思考-行动-观察-再思考 (循环)	生成结构化参数，完成工具调用
产出	计划清单 / 任务列表	思考过程 (文本)	行动序列与观察结果	API 调用结果

总结: Planning 管路线, CoT 管思考, ReAct 管执行循环, Function calling 管工具接口。

8 优势与价值

- ✓ 更适合复杂、多步任务
- ✓ 让执行更有条理，减少混乱
- ✓ 便于调试、监控与评估
- ✓ 对 Agent、RAG、自动化流程特别重要
- ✓ 可提升稳定性与可解释性



9 局限与挑战



10 总结



Planning 的本质：不是立刻做答案，而是先把复杂目标拆成可执行步骤，再更稳地完成任务。

记忆点：不会规划的模型像临时发挥；会规划的模型更像项目经理。



大模型世界中的 Reflection

让模型先检查、再修正：通过自我反思提升答案质量



1 什么是 Reflection?

Reflection 指模型先生成初步答案或行动结果，再对其进行回头检查，识别错误、遗漏或风险，最后修正输出。

它不是重新训练模型参数，而是在推理时的自我检查 / 自我改进过程。



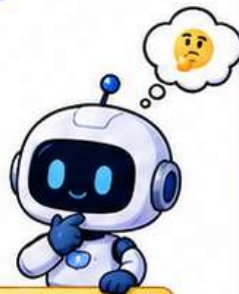
2 它在 Agent / 推理流程中的位置



Reflection 可以发生一次，也可以多轮迭代，直到满意或达到上限。

3 为什么需要 Reflection?

- 复杂任务容易漏步骤
- 首次回答可能有错误或幻觉
- 工具调用后需要复盘结果
- 长任务中需要纠偏
- 有助于提高可靠性与可解释性



先做，再检查，再改进。

4 一个直观例子



5 Reflection 的核心组成



6 它和 Chain-of-thought / ReAct / Reranking 有什么区别?

方法	Reflection	Chain-of-thought (CoT)	ReAct	Reranking
核心关注	回头检查与修正	逐步推理展开思路	思考 + 行动循环执行	对候选结果重新排序
时机	生成后回看与改进	生成过程中逐步推理	执行过程中交替进行	生成多个候选后再排序
目标	提升答案质量与可靠性	得到正确结论 (推理过程)	完成任务 (行动导向)	选择最优答案
是否修改原答案	是 (会修正)	不一定修改 (主要推理)	可能修改 (基于反馈)	不修改原内容 (只排序)

7 优势与价值

- 提高答案正确率
- 减少幻觉 (Hallucination)
- 改善结构完整性与逻辑性
- 让复杂任务更稳健
- 对 Agent、RAG、代码、写作、规划等任务非常有帮助



8 局限与挑战

- 会增加 token、延迟与成本
- 如果反思能力弱，可能越改越偏
- 不是所有简单任务都值得反思
- 过度循环可能低效



需要在质量与成本间权衡取舍。

9 常见应用场景



10 实用建议

- 明确让模型“先给初稿，再自检”
- 规定检查维度 (事实、逻辑、格式、遗漏)
- 控制反思轮数 (如 1~3 轮)
- 对关键任务结合工具验证
- 保留初稿与修正版，便于比较与追踪



Reflection 的本质:

不是让模型一遍就完美，而是让它像认真做题的人一样，答完后回头检查、发现问题、再把答案修好。



记忆点:

会反思的模型，更像会复盘的助手。



大模型世界中的 Multi-agent

让多个智能体分工协作：像团队一样思考、检索、规划、执行与复核



1 什么是 Multi-agent?

Multi-agent 指多个 AI 智能体拥有不同的角色与专长，围绕同一任务协同工作。

每个智能体可以专注于规划、检索、编码、写作、批评、执行等不同能力。

它不是一个模型回答一次，而是一个**协同配合的系统**。



2 它在 Agent / workflows 中的位置



智能体之间通过 消息、共享记忆、工具 与 中间输出 进行通信

3 为什么需要 Multi-agent?

- ✓ 复杂任务太大，单个 agent 容易遗漏
- ✓ 角色分工，专人专事，提升输出质量
- ✓ 并行处理，显著提升效率与吞吐
- ✓ 复核机制，降低错误与幻觉风险
- ✓ 更适合长流程、多工具、强依赖任务

一个人难做完的复杂任务，可以让一个 AI 团队来完成。



4 一个直观例子：帮我写一份行业研究报告



5 Multi-agent 的核心组成



6 它和 Single-agent / ReAct / Planning 有什么区别?

范式	核心特点	优势	局限
Single-agent	一个主体完成全部任务	简单直接，成本低，适合简单任务	复杂度上来后容易遗漏、幻觉、难以控制
ReAct	思考-行动-观测的循环	能与环境交互，灵活应对不确定性	仍是单体循环，复杂任务易偏离或效率低
Planning	先生成计划，再按计划执行	结构清晰，适合有明确步骤的任务	计划可能不完善，执行中缺少持续反馈
Multi-agent	多角色协作，互补分工	结合多种能力与机制，更鲁棒、更可控	设计与协调更复杂，成本与延迟更高

7 优势与价值

- ✓ 可扩展：添加新角色或工具即可扩展能力
- ✓ 可并行：多智能体并行，显著提升效率
- ✓ 可解释：分工清晰，过程可追踪、可审计
- ✓ 适合复杂任务：长流程、多依赖、多领域
- ✓ 便于复用专家 agent：如检索专家、代码专家
- ✓ 对 RAG / 编码 / 企业流程自动化极有帮助



8 局限与挑战

- ⚠ 成本更高：更多调用、更大消耗
- ⚠ 延迟更长：多轮通信与协同带来延迟
- ⚠ 通信开销：消息传递与同步有额外成本
- ⚠ 角色冲突 / 重复劳动：职责不清易内耗
- ⚠ 协调不当会把问题复杂化
- ⚠ 需要更好的评估与监控体系



关键：在效果与成本间找到平衡点。

9 常见应用场景



10 总结

Multi-agent 的本质：

不是让一个模型单打独斗，而是让多个专长不同的智能体像团队一样协作完成复杂任务。

记忆点：

分工 + 协作 + 调度 = 更强的 AI 团队。



我能看屏幕
会点会输
帮你把事办好!



大模型世界中的 Computer use

让模型学会看屏幕、点按钮、输入内容，像人一样操作电脑完成任务



1 什么是 Computer use?

Computer use 指的是：模型能够感知计算机界面（屏幕、窗口、按钮、输入框），理解当前状态，并执行点击、输入、滚动、选择、拖拽、导航等操作，从而完成任务。



它不是只用文字回答问题，而是直接在软件界面上“动手做事”。

2 它在 Agent / 工具调用流程中的位置



3 为什么需要 Computer use?

- 许多真实任务都发生在软件界面中，而不在开放的 API 里。
- 许多系统没有 API 或 API 不完善，甚至不对外开放。
- 能连接并利用企业里已有的各种工具和系统，提升工作效率。
- 从“会说”到“会做”，让模型真正成为能干活的数字员工。
- 带来自动化与生产力提升，减少重复劳动，释放人力。



4 一个直观例子

需求：帮我在日历中创建明天下午 3 点的会议，并填写主题和参会人



1 打开日历应用



2 定位到明天下午 3 点



3 输入会议主题和参会人

主题：项目对齐会
参会人：Alice; Bob; 张三



4 点击“创建/保存”，确认结果

会议已创建！
明天 15:00 项目对齐会

5 Computer use 的核心组成

- 屏幕感知 / OCR
看懂像素、文字、图标、布局
- UI 理解
理解控件类型、可交互区域
- 状态跟踪
记录界面状态与历史变化
- 动作空间
点击、输入、按键、滚动、选择、拖拽
- 规划能力
将目标拆解为可执行的步骤
- 记忆能力
短期记忆 + 长期记忆 (上下文)
- 安全检查
敏感操作识别与权限校验
- 结果验证
判断任务是否完成或需要重试



6 它和 Function calling / RPA / ReAct 有什么区别?

对比项	Function calling (函数调用)	RPA (机器人流程自动化)	ReAct (推理 + 工具使用)	Computer use (计算机使用)
交互对象	结构化 API	固定界面/控件	工具/检索等	图形界面 (像素)
依赖条件	需要开放 API	界面稳定、规则明确	有工具可用	只需能看见界面
能力本质	通过 API 获取/写入数据	按规则执行预设步骤	推理 + 选择工具	直接操作界面完成任务
适用范围	API 覆盖的场景	重复性、规则明确流程	多种工具组合任务	人能用的界面都可尝试
典型方式	调用接口 (get/post)	脚本/流程设计器	思考 -> 行动 -> 观察	看屏幕 -> 点/输/滚动

一句话总结：Computer use 像人一样操作界面，能应对更多“没有 API”的现实世界软件。



7 优势与价值

- 可跨越大量既有应用，兼容性广。
- 能处理没有 API 的界面型系统。
- 降低集成门槛，无需改造系统。
- 适用于办公、测试、客服、运营、数据录入、流程自动化等多类场景。
- 提升效率，减少人为操作错误。



8 局限与挑战

- 界面变化可能导致行为失效，需要维护与适配。
- 多步骤任务可能较慢，效率受延迟影响。
- OCR / 视觉理解可能出错，影响判断。
- 界面状态可能模糊或歧义，难以确定下一步。
- 权限 / 安全风险：误操作或越权访问的风险。
- 延迟高、成本较高，需要权衡投入产出。
- 需要可靠的结果验证机制，防止“假成功”。



关键操作涉及敏感数据/资金时，必须设置安全护栏 (Guardrails) !

9 常见应用场景



例如：发邮件、做报表、查数据、下单、审批、报销、创建资源、巡检……

10 总结



Computer use 的本质：
不是只让模型回答问题，
而是让模型能够像数字员工一样
观察界面、操作软件、完成任务。

★ 记忆点：会看界面，会点按钮，会完成流程。





大模型世界中的 Human-in-the-loop

AI 很能干，
但关键时刻
还需要人来把关！



★ 让人类参与关键决策与校验：AI 自动化中的监督、纠错与协同机制 ★

1 什么是 Human-in-the-loop?

Human-in-the-loop (HITL, 人类参与闭环) 是指在 AI 工作流程的一个或多个关键阶段，由人类参与其中，如设定目标、审核输出、批准执行、纠正错误或提供反馈。

它不是纯人工，也不是完全自主的 AI，而是人机协同。



2 它在 Agent / workflows 中的位置



3 为什么需要 Human-in-the-loop?

- ✓ 大模型可能出现幻觉、误判或不稳定输出
- ✓ 某些任务涉及业务风险、资金、法律、医疗或品牌影响
- ✓ 工具调用和自动执行可能带来真实世界后果
- ✓ 人类能补充经验、常识、价值判断与责任承担

★ 先自动化，再把关键决定交给人类。



4 一个直观例子

AI 帮企业起草一封重要客户回复邮件

没有 HITL



可能发错信息、语气不当、事实错误

有 HITL



更稳妥、更准确、更符合业务要求

5 Human-in-the-loop 的核心组成



关键不是“人一直盯着”，而是在人最有价值的地方介入。

6 它和 Fully autonomous / Human-on-the-loop 有什么区别?

	Fully autonomous (完全自主)	Human-on-the-loop (人在环上)	Human-in-the-loop (人参与闭环)
定义	AI 自主完成任务，人工不介入	人主要监控 AI，必要时接管	流程中必须有人审核、批准或修正
人类角色	无参与	监控者+应急接管者	参与者+决策者
介入方式	无	例外触发时介入	关键节点必经介入
适用场景	低风险、低影响任务	中等风险、需监控场景	高风险、高价值任务
特点	效率最高，风险较高	效率较高，可控性中等	可控性强，效率较低

★ 介入深度越高，可控性通常越强，但效率可能更低。

7 优势与价值



- ✓ 提高准确率与可靠性
- ✓ 降低高风险错误
- ✓ 增强可解释性与责任边界
- ✓ 更适合企业级流程与合规场景
- ✓ 促进人机协作，而不是简单替代

8 局限与挑战

- ⚠ 增加流程时间与人工成本
- ⚠ 人工审核可能成为瓶颈
- ⚠ 如果审核流于形式，效果有限
- ⚠ 需要设计清晰的升级/回退机制
- ⚠ 人和模型的职责边界不清会引发混乱



9 常见应用场景



10 总结



Human-in-the-loop 的本质：
不是让 AI 单独做完，也不是让人事事亲力亲为，
而是在关键节点由人来**监督、纠错和拍板**。

记忆点：
让 AI 提速，
让人类把关。





上下文管得好，
模型表现才会好！

大模型世界中的 Context management

AI



把有限上下文安排得更合理，让模型在正确的信息里思考与回答

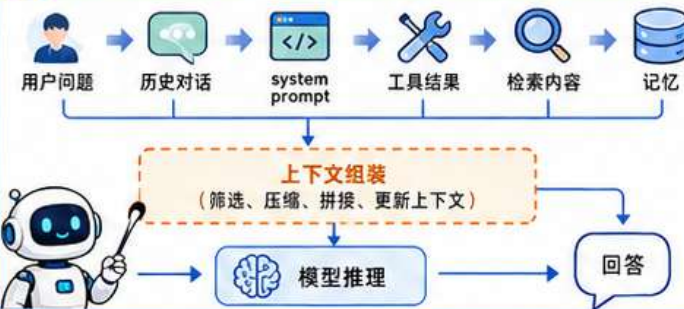
1 什么是 Context management?

Context management (上下文管理) 是指在模型生成回答前后，对“哪些信息该放进上下文、按什么顺序放、保留多久、如何压缩与更新”进行管理。



★ 它不是单纯扩大 context window，而是更高效地组织上下文。

2 它在 LLM / Agent / RAG 流程中的位置



3 为什么需要 Context management?

- ✓ 上下文窗口有限，不能把所有信息都塞进去
- ✓ 信息太多会增加噪声与干扰
- ✓ 长对话会遗忘早期关键信息
- ✓ 检索结果、工具输出、记忆内容需要协调
- ✓ 影响成本、延迟、准确率与稳定性



4 一个直观例子：旅行规划助手

没有上下文管理 (问题多)

- ✗ 历史太长，重点丢失
- ✗ 重复信息多
- ✗ 无关闲聊干扰
- ✗ 回答跑偏、不稳定

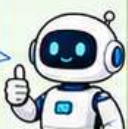


用户：我想去日本玩...
助手：好的，去哪里呢？
用户：东京吧！
助手：东京很好玩...
用户：预算1万以内...
助手：好的.....
(对话很长，重点被淹没)

有上下文管理 (更好)

- ✓ 保留关键：预算、时间、城市偏好、最新约束
- ✓ 丢弃无关闲聊，去重汇总
- ✓ 结构化事实后再回答
- ✓ 更相关、更稳定

关键信息汇总：
• 目的地：日本 (东京)
• 时间：5天4晚
• 预算：1万元以内
• 偏好：购物、美食、交通便利
• 最新约束：不想换太多酒店
基于以上，给出行程建议...



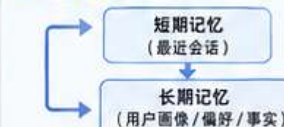
5 Context management 的核心组成



6 常见策略 (怎么做)

1. 滑动窗口：只保留最近若干轮
2. 关键事实提取：把约束条件单独保存
3. 摘要压缩：长对话压缩成短摘要
4. 检索增强：按问题动态补充外部资料

5. 分层记忆：短期记忆 + 长期记忆



7. 去重与过滤：删除重复、低价值信息



7 它和 Context window / Memory / RAG 有什么区别?

概念	关注点	做什么	类比角色
Context window (上下文窗口)	容量限制	模型一次能看多少信息 (token 数上限)	舞台大小
Memory (记忆)	保留什么	跨轮/长期保存用户相关信息 (短期 + 长期)	资料库
RAG (检索增强)	从哪里找	从外部知识库检索相关内容补充上下文	资料搜索员
Context management (上下文管理)	如何使用	决定最终放什么、怎么放、何时更新，组织成最优上下文	总调度/导演

★ Context management 像总调度，不等于其中任何一个组件。

8 优势与价值

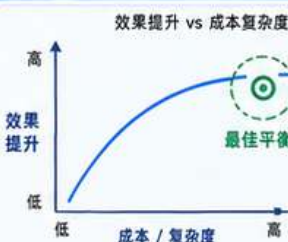


- ✓ 提升回答相关性
- ✓ 降低幻觉与跑题
- ✓ 更适合长对话与复杂任务
- ✓ 提高多工具 / Agent 工作流稳定性
- ✓ 降低 token 成本与延迟
- ✓ 提升企业应用可控性

更准 更稳 更快 更省

9 局限与挑战

- ⚠ 摘要可能丢失细节
- ⚠ 记忆写入错误会持续污染后续回答
- ⚠ 检索内容质量差会影响上下文质量
- ⚠ 策略过于复杂会增加系统成本
- ⚠ 不同任务需要不同的上下文策略



10 总结



原始信息



筛选

(去噪/过滤)



压缩

(提炼/摘要)



组装

(拼接/排序)



推理

(模型理解)



更可靠回答

(更相关、更稳定)

★ Context management 的本质：
不是让模型看到更多，
而是让模型在正确的信息里思考。





上下文太长?
交给 Compaction!

『大模型世界中的 Compaction』

把冗长上下文压缩成更短、更有用的表示，让模型在有限上下文里持续工作



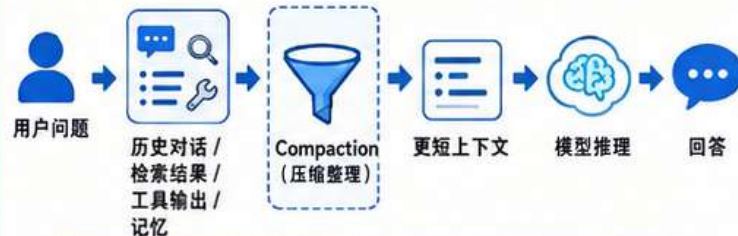
1 什么是 Compaction?

Compaction (压缩整理) 是指在不完全丢失关键信息的前提下，把长对话、长文档、历史中间结果、工具输出或记忆内容压缩成更短、更结构化、更易复用的表示。



★ 强调:
它不是简单删减, 而是
“保留关键信息、移除冗余、重组表达”。

2 它在 LLM / Agent / RAG 流程中的位置



说明: Compaction 常发生在上下文过长、轮次变多、需要长期记忆或多步任务中。

3 为什么需要 Compaction?

- ✓ 上下文窗口有限, 不能无限堆内容
- ✓ 长上下文会增加成本与延迟
- ✓ 冗余信息会干扰推理
- ✓ 多轮对话中旧信息会越来越杂乱
- ✓ 有助于长期任务保持连续性

未压缩: 混乱纸堆

压缩后: 清晰摘要卡片



4 一个直观例子: 旅行规划助手

没有 Compaction

- 我想去旅行, 预算大概 8000 元左右。
- 日期是 7 月中旬, 想去海边城市。
- 最好是 5 天 4 晚, 预算别超 8000。
- 7 月 15 日到 7 月 19 日, 阳光沙滩!
- 还想吃海鲜, 预算还能再低一点吗?

历史对话很长, 重复信息多, 模型容易混乱, 遗漏关键细节。

有 Compaction

旅行规划摘要 (结构化)

- ✓ 目的地: 海边城市
- ✓ 时间: 7/15 - 7/19 (5 天 4 晚)
- ✓ 预算: 不超过 8000 元 (越低越好)
- ✓ 偏好: 阳光沙滩、海鲜
- ✓ 已确定事项: 时间、预算范围、偏好
- ✓ 待决事项: 具体城市、航班/酒店、行程安排

结构化摘要清晰, 关键信息集中, 模型更容易继续规划。

5 Compaction 的核心组成



6 常见 Compaction 方式



7 它和 Summarization / Chunking / Context management 有什么区别?

方法	重点	说明
Summarization (摘要)	概括内容	重点是把内容概括成更短的表达, 强调信息浓缩。
Chunking (切分)	切分内容	重点是把内容切成更小块, 便于检索或并行处理。
Context management (上下文整理)	选择和组织什么放进上下文	重点是决定哪些信息进入上下文, 并如何组织排序。
Compaction (压缩整理)	压缩成更短、更可复用的表示	重点是把已有内容压缩成更短、更结构化、更可复用的表示。

★ 总结: Compaction 往往是 context management 的关键手段之一。

8 优势与价值

- ✓ 节省 token
- ✓ 降低调用成本
- ✓ 减少延迟
- ✓ 提高长任务稳定性
- ✓ 让 Agent 更容易持续执行
- ✓ 便于记忆和状态传递



更高效 · 更稳定 · 更智能

9 局限与挑战

- ⚠ 压缩过度可能丢信息
- ⚠ 压缩质量差会引入偏差
- ⚠ 重要细节可能被忽略
- ⚠ 不同任务适合不同压缩策略
- ⚠ 需要在“信息保真”与“成本效率”之间平衡



10 总结

原始长内容 -> 去重/筛选 -> 摘要/结构化 -> 更短上下文 -> 更可靠回答



Compaction 的本质:
不是让模型看到更少, 而是让模型看到更有用的信息。



记忆点:
压缩不是乱删,
而是有策略地保留关键内容。



大模型世界中的Memory



让模型在对话与任务中记住重要信息：从短期上下文到长期记忆

1 什么是 Memory?

Memory 是指让 LLM 或 Agent 在多轮对话或跨任务中，保留、回忆并使用重要信息的机制。它不是模型的预训练知识本身，而是模型在使用过程中“记住”的信息。



2 它在 LLM / Agent / RAG 流程中的位置



★ Memory 帮助模型把过去的信息与当前任务连接起来，让回答更连贯、准确、个性化。

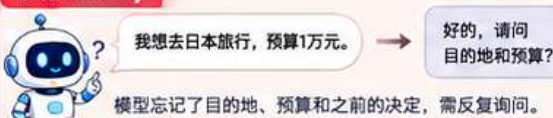
3 为什么需要 Memory?

- ✓ 保持多轮对话的连续性
- ✓ 记住用户偏好与习惯
- ✓ 维护任务状态与进度
- ✓ 减少重复提问与信息冗余
- ✓ 支持长时间、复杂流程
- ✓ 提升个性化与一致性体验



4 一个直观例子

没有 Memory



有了 Memory



5 Memory 的核心组成



6 常见 Memory 类型

- 短期记忆 (Short-term)**
当前对话或任务的上下文，随会话存在，窗口内可用。
- 长期记忆 (Long-term)**
持久保存的用户事实、偏好、习惯、设定等。
- 情景记忆 (Episodic)**
过去交互或事件的记录，如某次对话、操作、结果。
- 语义记忆 (Semantic)**
结构化的事实与知识，如“用户是设计师”。
- 工作记忆 (Working/Task)**
当前任务的计划、步骤、进度、临时变量等。

7 它和 Context window / RAG / Context management 有什么区别?

概念	作用与特点	范围/时间维度
Context window (上下文窗口)	指当前提示词中能放入的内容，由模型上下文长度限制。	短期 (会话内)
RAG (检索增强生成)	从外部知识库检索相关知识，用于补充模型知识。	外部知识层面
Context management (上下文管理)	决定把哪些信息组织、压缩、放入当前上下文。	短期 (会话内)
Memory (记忆)	跨多轮/多任务保留并管理有价值的信息，供未来使用。	长期 (跨会话/任务)

一句话：Context 是“此刻带进来”，Memory 是“留着以后用”。

8 优势与价值

- ✓ 对话与任务更连贯
- ✓ 输出更个性化、贴合用户
- ✓ Agent 更稳定、可靠
- ✓ 支持长任务与复杂流程
- ✓ 减少重复输入，提升效率
- ✓ 提高用户满意度与粘性

9 局限与挑战

- ❗ 记忆污染：错误或噪声信息被保存
- ❗ 过期/陈旧：信息随时间可能失效
- ❗ 隐私与安全：敏感信息泄露风险
- ❗ 存储决策：哪些信息值得记住?
- ❗ 检索错误：找不全、找不准
- ❗ 冲突与一致性：不同记忆可能冲突
- ❗ 遗忘困难：如何优雅地“忘记”过时信息

10 总结



Memory 的本质：
不是让模型“知道更多”，
而是让模型在使用过程中“记住该记住的内容”。

★ 记忆点：会记住，才能更连续、更个性化、更像助手。❤️

大模型世界中的 MCP

(Model Context Protocol)

让大模型轻松连接外部世界的“通用接口”



1. 什么是 MCP?

MCP (Model Context Protocol) 是由 Anthropic 提出的开放协议，旨在标准化大模型与外部数据源、工具和服务的连接方式。它定义了一套通用的通信规范，使得大模型无需为每个工具单独适配，就能安全、高效地获取上下文和使用能力。



标准化接口
统一接入规范



即插即用
快速集成工具



安全可控
权限与隔离机制



生态开放
支持多种实现

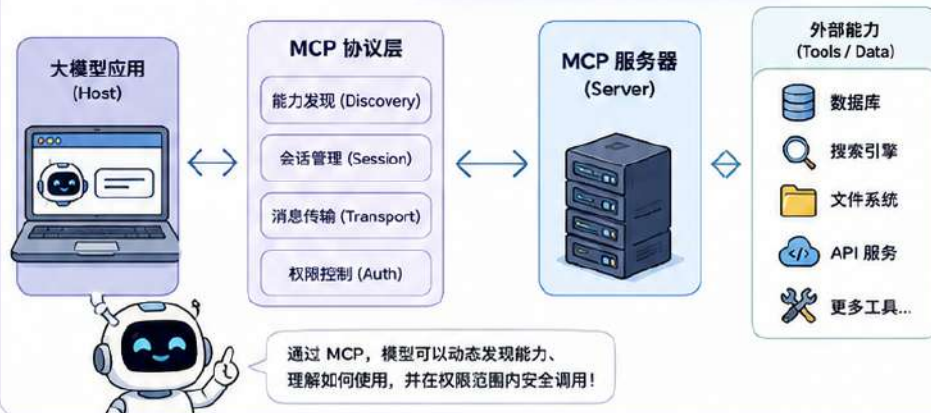
2. MCP 能解决什么问题?

- ✓ 集成难：每接入一个工具都要写专属代码
- ✓ 不灵活：模型无法动态发现和使用新工具
- ✓ 不安全：工具调用缺乏统一的权限控制
- ✓ 难扩展：生态碎片化，重复造轮子

MCP 提供一套标准协议，
让连接变得简单、通用、
安全、可扩展！



3. MCP 工作原理



4. MCP 架构组成

- Host (客户端)
如：AI 应用、智能体、IDE 插件等
- MCP Client
与 MCP Server 建立连接，发送请求
- MCP Server
暴露能力，处理请求，返回结果
- Tools / Resources
各种外部工具、数据源和服务

5. 消息交互流程 (示例)



6. MCP 的优势

- ★ 标准化：统一的协议，降低集成成本
- ★ 灵活性：动态发现与调用能力
- ★ 安全性：细粒度权限控制与隔离
- ★ 扩展性：支持自定义能力与生态共建
- ★ 互操作性：不同厂商、框架、模型都能用

7. 应用场景

- 企业知识库问答 (连接数据库、文档)
- 智能办公助手 (操作文件、发送邮件)
- 数据分析与可视化 (调用分析工具)
- 开发者助手 (查 API、读代码、改配置)
- 物联网与自动化 (设备控制、数据采集)



8. 生态与现状

- ✓ Anthropic 开源并维护 MCP 规范
- ✓ 社区已有多种语言实现 (Python / TS / Go ...)
- ✓ 各类工具与平台正在加入 MCP 生态
- ✓ 未来将成为大模型生态的“通用接口标准”

ANTHROPIC



MCP
Ecosystem



MCP = 大模型的通用接口协议，让连接更简单，让智能更强大！

大模型世界中的 Agent Harness

让大模型从“单点智能”进化为“可控、可扩展、可落地的智能体”



大模型本身很强大，
但要让它真正解决复杂问题，
还需要一个“外骨骼”系统——
这就是 **Agent Harness**。

一句话理解

Agent Harness 是一个框架/运行时环境，
它把大模型 (LLM) 的能力，
与外部世界、工具、数据和规则连接起来，
形成一个可靠、可控、可观测的智能体系统。



1. 为什么需要 Harness?

- ✓ 大模型是大脑，但缺少感知、行动和记忆能力
- ✓ 真实业务需要工具调用、数据访问、权限控制
- ✓ 需要可靠性保障：重试、容错、监控、降级
- ✓ 需要可观测性：日志、追踪、评估、审计
- ✓ 需要安全合规：权限、隔离、内容审核

2. Harness 的核心组成

- 模型接口层 (Model Interface)**
统一封装不同大模型，
支持切换、路由、负载均衡
- 指令与规划层 (Instruction & Planning)**
负责任务理解、思考、拆解、
生成执行计划 (可结合提示工程、CoT)
- 工具与能力层 (Tools & Capabilities)**
连接外部工具、API、插件、函数，
扩展模型的行动能力
- 知识与数据层 (Knowledge & Data)**
提供检索、向量数据库、结构化数据，
让模型“有依据地回答”
- 记忆层 (Memory)**
短期记忆 (会话) + 长期记忆 (用户/领域)
支持检索、总结、反思
- 安全与权限层 (Security & Permission)**
身份认证、权限控制、内容过滤、
数据脱敏，保障安全合规
- 观测与评估层 (Observability & Evaluation)**
日志、链路追踪、指标监控、效果评估、
A/B 测试，持续优化

3. Harness 工作流程



4. Harness 带来的价值

- 能力增强**
让模型获得使用工具、访问数据、
执行操作的能力
- 可靠可控**
通过权限、校验、重试、降级机制
提升稳定性与可控性
- 可观测可优化**
全链路追踪与评估，快速发现问题，
持续迭代优化
- 安全合规**
细粒度权限、内容安全、审计追踪，
满足企业级要求
- 可扩展复用**
模块化设计，支持快速接入新工具、
新模型、新数据

5. 一个简单的代码视角

```
agent = AgentHarness(  
    model=LLM("gpt-4o"),  
    tools=[search_tool, calc_tool, crm_api],  
    memory=VectorMemory(...),  
    retriever=RAG(...),  
    security=Auth(roles=["user", "admin"]),  
    observability=Tracer()  
)  
  
response = agent.run("帮我分析上月销售数据并生成报告")
```

- 模型：提供智能
- 工具：提供行动
- 数据：提供依据
- 记忆：提供连续性
- 安全：提供边界
- 观测：提供反馈



6. 与传统集成方式的区别

传统方式	使用 Harness
直接调用大模型 API	统一接入，多模型路由
功能单一，难扩展	模块化设计，灵活扩展
缺少工具与数据连接	工具、知识、记忆深度集成
难监控，问题难定位	全链路观测，快速迭代
安全与权限能力薄弱	企业级安全与权限体系

7. 典型应用场景

- 智能客服/助手**
接入知识库与工单系统
- 数据分析/BI 助手**
连接数据库、BI 工具
- 企业内部 Copilot**
整合文档、流程、系统
- 自动化工作流 Agent**
调用多工具完成复杂任务

总结

Agent Harness 就像智能体的“操作系统”，
它让大模型不再是孤立的“聪明大脑”，
而是一个能够感知、思考、行动、
记忆和学习的完整系统。

模型决定上限，**Harness 决定落地效果!**



8. 设计 Harness 的最佳实践

- 模块化设计**
解耦各层，便于扩展与替换
- 最小权限原则**
工具与数据按需授权
- 可观测优先**
从一开始就埋点监控
- 渐进式增强**
先跑通，再增强能力
- 评估驱动迭代**
用数据指导优化方向



大模型世界中的 Just-in-time retrieval



需要时再检索：让模型在正确的时机拿到最相关的信息

1 什么是 Just-in-time retrieval?

Just-in-time retrieval (按需检索) 是指：系统不会在一开始就把所有外部知识都检索进来，而是等到某个具体的子任务、问题或推理步骤真正需要信息时，再按需触发检索。

不是先全拿，再慢慢用；
而是需要时，再去取。



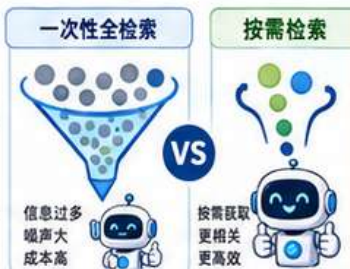
2 它在 LLM / Agent / RAG 流程中的位置



★ 检索发生在执行过程中，而不是只在生成前。

3 为什么需要 Just-in-time retrieval?

- ✓ 减少无关上下文堆积
- ✓ 降低 token 成本与延迟
- ✓ 在逐步任务中按需补充信息
- ✓ 提高上下文新鲜度
- ✓ 减少一次性检索带来的噪声



4 一个直观例子

任务：帮我规划去东京的商务出差，并给出预算和地铁换乘建议。



不同的检索只在对应的子任务到达时发生！
不需要的信息，不提前检索。

5 核心触发机制



6 它和普通 RAG / Query rewrite / Reranking 有什么区别?

方法	主要做什么	检索时机	特点
普通 RAG	先检索一批相关文档，再生成答案	通常在生成前一次性检索	简单直接，但可能带来噪声和高成本
Query rewrite	改写查询，提升检索质量	检索前	关注“怎么搜”让检索更准
Reranking	对检索结果重新排序	检索后	关注“排得更好”提高相关性
Just-in-time retrieval	在任务执行中，决定何时检索，按需获取	任务进行中按需触发	关注“何时搜”只在需要时取，更高效、更灵活

7 优势与价值

- ★ 更省上下文窗口
- ★ 更擅长长任务 / 多步 Agent
- ★ 减少幻觉
- ★ 能结合规划、工具调用与执行反馈
- ★ 更容易处理动态信息



8 局限与挑战



- 需要更好的触发判断
- 流程更复杂
- 多次检索可能带来额外等待
- 错过检索时机会影响答案
- 需要与 memory / planning / tool use 协同

9 常见应用场景



10 总结



Just-in-time retrieval 的本质：
不是让模型先看到更多，
而是让模型在恰当的时机
看到恰当的信息。



大模型世界中的 Structured note-taking

让模型把信息记得更清楚、组织得更有序：把零散信息变成可追踪、可复用的结构化笔记

核心目标

不是单纯“记更多”，
而是“记得更有组织”。

1 什么是 Structured note-taking?

Structured note-taking (结构化笔记) 是指模型在阅读、检索、对话、规划或执行任务时，不只是保存原始文本，而是把关键信息整理成有结构的记录，如要点、字段、表格、层级、实体关系、待办事项、证据来源等。

强调：不是单纯“记更多”，
而是“记得更有组织”。



2 它在 LLM / Agent / RAG 流程中的位置



它常常作为“中间表示层”，连接信息获取与任务执行。

3 为什么需要 Structured note-taking?

- ✓ 原始上下文太长、太乱
- ✓ 多轮任务容易遗漏细节
- ✓ 检索结果分散，难以整合
- ✓ 需要跨步骤追踪事实、状态与决策
- ✓ 有助于降低幻觉、减少重复劳动



4 一个直观例子 (旅行规划助手)

零散信息 (原始输入)

- 预算：1.2 万元以内
- 日期：7/10 - 7/16 (7 天)
- 出发地：北京
- 目的地：日本关西 (大阪/京都)
- 航班：直飞优先
- 酒店：干净、交通便利
- 偏好：美食、历史景点、购物
- 备注：带爸妈，步行不宜太多

结构化笔记 (整理后)

目的地	日本关西 (大阪、京都)
时间	7/10 - 7/16 (7 天)
预算	≤ 1.2 万元/人
候选方案	A 方案 / B 方案 / C 方案
约束条件	直飞优先；步行少；长辈友好
偏好	美食；历史景点；购物
待确认事项	航班价格；酒店房型
证据/来源	航司官网；携程；知乎帖

✓ 结构化后，模型更容易规划行程、比较方案、给出建议！

5 Structured note-taking 的核心组成



Key facts (关键事实)
重要的、可验证的事实要点。



Decisions (决策)
已做出的选择与理由。



Entities (实体)
人、地点、组织、事物等实体。



Evidence / Source (证据/来源)
支撑信息的来源与引用。



Relationships (关系)
实体之间的联系或依赖关系。



Status (状态)
进展、状态、优先级等。



Tasks / To-dos (待办)
需要完成的任务或下一步行动。



Summary (摘要)
对当前内容的精炼总结。

6 常见记录形式 (不同任务适合不同笔记结构)

项目符号笔记

-
-
-
-

JSON / schema

```
{  "key": "value",  "items": [    {"id": 1}  ]}
```

表格

-	-
-	-
-	-

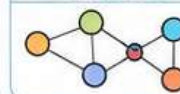
层级大纲

1. 主题
 - 1.1 小节
 - 1.2 小节
2. 主题
 - 2.1 小节

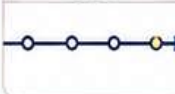
知识卡片



实体关系图



时间线



不同任务适合不同笔记结构，关键是“合适”而非“唯一”。

7 它和 Summarization / Memory / Context management 有什么区别?

方法	主要目标	关注点	结果形式
Summarization	压缩内容	减少长度	更短的文本摘要
Memory	跨轮保留信息	长期保留	记忆条目/向量库等
Context management	组织上下文输入	选择与排序	选取的输入片段
Structured note-taking	把信息整理成可操作、可追踪、可复用的结构	结构化与可用性	结构化笔记 (表/图/卡片等)

一言以蔽之：它既能服务记忆，也能服务上下文管理与后续推理。

8 优势与价值

- ✓ 信息更清晰
- ✓ 推理更稳定
- ✓ 规划更容易
- ✓ 多步任务更可控
- ✓ 便于协作和审计
- ✓ 更适合 Agent 工作流

好笔记 = 好推理 = 好结果！

9 局限与挑战

- ⚠ 记录结构设计不当会丢失信息
- ⚠ 维护笔记需要额外 token / 成本
- ⚠ 过度结构化可能降低灵活性
- ⚠ 更新不及时会导致“过时笔记”
- ⚠ 需要在完整性与简洁性之间权衡



10 总结：让信息变成可复用的力量



Structured note-taking 的本质：
不是让模型看到更多，而是让模型把
重要信息记成可理解、可追踪、可
复用的结构。

记忆点：
好笔记，不只是
记录，而是帮助
模型思考。

★★ 好的结构化笔记，让模型更懂信息、少走弯路、持续进化！★★



先有通识能力，
再学会按要求回答。

大模型世界中的 SFT

★ Supervised Fine-Tuning ★

SFT 的本质：
用高质量示例教模型
更好地完成任务。



1 什么是 SFT?

SFT (Supervised Fine-Tuning, 监督微调) 是指在预训练模型的基础上, 使用带标注的“指令-输入-输出”示例继续训练模型, 让模型学会更好地遵循指令、匹配格式、满足任务要求, 输出更准确、更稳定、更贴合人类期望的回复。

★ 核心作用:
让模型从“会说话”
变得“会做事”。



2 它在大模型训练流程中的位置



3 为什么需要 SFT?

- ✓ 更好地遵循指令, 理解并完成任务
- ✓ 输出更结构化、格式更规范
- ✓ 结果更稳定, 减少跑偏与幻觉
- ✓ 对齐特定风格、领域或业务要求
- ✓ 为后续对齐 (DPO/RLHF) 打下基础



4 一个直观例子

用户指令

请用要点形式, 介绍光合作用的过程。

SFT 之前 (基础模型)

光合作用是植物利用光能把二氧化碳和水转化为有机物, 并释放氧气的过程。这个过程包含复杂的生化反应, 主要发生在叶绿体中, 对维持生态系统的能量至关重要。

✗ 回答较泛泛, 结构不清晰, 不符合“要点形式”要求。

SFT 之后 (微调模型)

光合作用过程要点:

- 1) 光反应: 吸收光能, 生成ATP和NADPH;
- 2) 暗反应 (卡尔文循环): 利用ATP和NADPH固定CO₂;
- 3) 生成有机物 (如葡萄糖) 并释放O₂。

✓ 结构清晰, 要点明确, 严格遵循指令格式。

5 SFT 的核心数据



指令 (Instruction)
明确告诉模型要做什么



输入/上下文 (Input / Context)
提供必要的背景信息或问题



目标输出 (Target / Label)
高质量、标准的期望回答 (标注)



构成高质量
监督数据集

- ✓ 多样化
- ✓ 准确性
- ✓ 一致性
- ✓ 覆盖任务场景

★ 数据质量决定 SFT 效果上限!

6 SFT 的训练目标

在给定 (指令+输入) 的条件下,
让模型生成与“目标输出”一致的回答。

指令 + 输入
(条件)



目标输出
(标签)

训练目标: 最小化预测目标序列的交叉熵损失
(即: 学习在每一步预测正确的下一个 token)

★ 本质: 从示例中学习行为, 而不只是从大量文本中学习语言。

7 它和 Pretraining / Prompting / RLHF 有什么区别?

方法	数据来源	是否更新模型参数	学习目标	特点
Pretraining (预训练)	海量无标注文本	是	学习通用语言能力 (预测下一个词)	通识广、知识多, 但不会按要求做事
SFT (监督微调)	高质量标注的指令-输入-输出	是	学习指令遵循与任务表现	让模型“听懂指令, 按要求回答”
Prompting (提示工程)	无训练数据	否	通过提示引导模型临时生成	零训练, 灵活但效果不稳定
RLHF / DPO (偏好对齐)	人类偏好数据 (比较/评分)	是	优化人类偏好与价值观	让模型更安全、更符合人类期望

8 优势与价值

- ✓ 显著提升指令遵循能力
- ✓ 快速适配特定领域与任务
- ✓ 输出更结构化、风格更一致
- ✓ 提高可用性, 降低幻觉风险
- ✓ 支持企业/行业定制, 闭环落地



让大模型从“通才”走向“专才”!

9 局限与挑战

- ⚠ 强依赖高质量标注数据
- ⚠ 可能产生灾难性遗忘 (损害原有通用能力)
- ⚠ 过拟合风险 (数据偏差放大)
- ⚠ 标注成本高、周期长
- ⚠ 无法解决所有问题 (如事实性错误、复杂推理等)

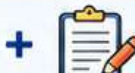


需要与评估、对齐、工程优化
协同配合, 才能发挥最佳效果。



10 总结

SFT 是连接通用能力与可用能力的关键桥梁。



预训练
打基础

SFT
学会按要求做事

对齐
更符合人类期望

可信的
大模型应用

SFT 的本质:
用高质量示例教模型更好地完成任务。



好数据 + 好方法 + 好评估 = 更强大的大模型! ★

先有通识能力, 再学会按要求回答; 先学会做对, 再追求做得更好。★



不断学习



持续迭代



创造价值



用更少的参数，做更棒的模型！

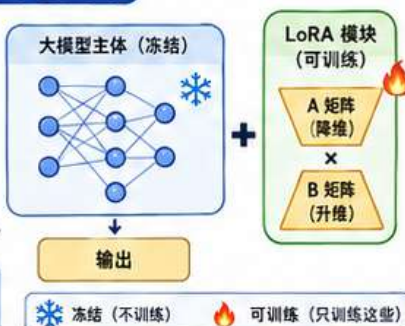
大模型世界中的 LoRA / PEFT



少量可训练参数，也能高效微调大模型

1 什么是 LoRA / PEFT?

- PEFT (Parameter-Efficient Fine-Tuning, 参数高效微调) 是一类只训练少量参数、尽量冻结大模型主体参数的微调方法。
- LoRA (Low-Rank Adaptation, 低秩适配) 是最常见的 PEFT 方法之一，通过在原有权重旁边增加低秩矩阵来学习任务适配能力。



★ 不是从头训练，也不是全量微调，而是“少改一点，学会任务”。

2 它在大模型训练流程中的位置



3 为什么需要 LoRA / PEFT?

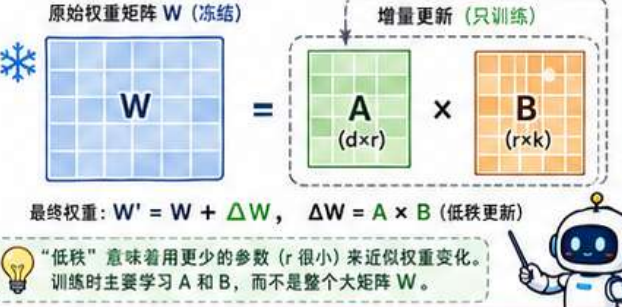
- 显存需求更低**：只训练少量参数，显存占用大幅下降。
- 训练速度更快**：参数更少，反向传播更快。
- 存储多个任务适配器更方便**：一个基础模型 + 多个小 adapter。
- 更适合个人开发者、团队实验和企业定制**：降低门槛，提升效率，节省成本。
- 可以在不改动基础模型太多的前提下进行任务适配**：保护基础模型能力，风险更低。

4 一个直观例子

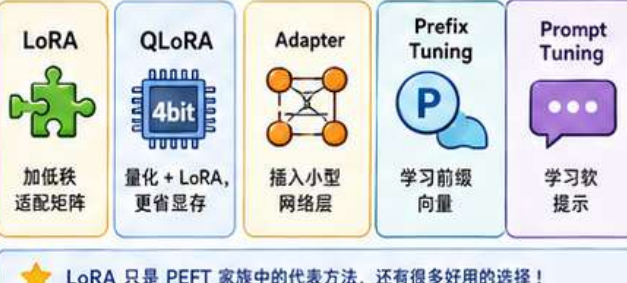
场景：把通用模型适配成“法律问答助手”



5 LoRA 的核心原理



6 PEFT 家族里还有什么?



7 它和全量微调有什么区别?

对比项	全量微调 (Full Fine-Tuning)	LoRA / PEFT (如 LoRA)
方法	更新模型全部参数	只训练少量新增参数 (如 A、B)
训练参数量	数十亿 ~ 数千亿	百万级 ~ 千万级 (取决于 rank)
显存成本	高 (需要保存全部梯度)	低 (仅保存少量可训练部分)
训练速度	慢	快
部署灵活性	低 (每个任务都要保存整模型)	高 (一个基础模型 + 多个 adapter)
效果上限	最高 (理论上)	接近全量 (看任务与设置)

结论：全量微调参数更多、成本更高、灵活性较低；LoRA / PEFT 更轻量、易切换、成本低。
备注：不同任务下效果差异要看数据和设置，合理选择最重要！

8 优势与价值



9 局限与挑战



10 总结



核心总结

『LoRA / PEFT 的本质：不是把整个大模型都重学一遍，而是用少量可训练参数，高效地把模型调成你需要的样子。』

记忆点

少训练、低成本、快适配。



大模型世界中的 RLHF

Reinforcement Learning from Human Feedback

让模型更懂你，
回答更有用、更安全、更贴心！

1 什么是 RLHF?

RLHF (基于人类反馈的强化学习) 是一种让模型根据人类偏好进行优化的方法。它通常不是从零训练模型，而是在预训练、SFT 之后，通过人类偏好数据进一步调整模型，让回答更符合“有用、诚实、无害”等目标。



★ 强调：本质上不是单纯教模型“知道什么”，而是进一步教模型“更应该怎样回答”。

2 它在大模型训练流程中的位置



RLHF 常常位于 SFT 之后，是“能力”到“对齐”的关键阶段。

3 为什么需要 RLHF?

- ✓ 让模型输出更符合人类偏好
- ✓ 提升回答的帮助性、礼貌性和安全性
- ✓ 减少有害、冒犯、跑题或不受欢迎的回答
- ✓ 改进多轮对话体验和指令遵循能力
- ✓ 帮助模型在多个可行答案中更偏向“人更喜欢”的那个



4 一个直观例子

用户问：如何礼貌地拒绝同事不合理的加班请求？

基础模型的回答 (可能)

我不想加班，这不在我的工作范围内。
你应该自己完成。
我今天有自己的安排，做不了。

生硬、直接，缺乏同理与建议

经过 RLHF 的回答 (示例)

你可以这样回复同事：

1. 先表达理解：理解对方的需求与压力；
2. 说明自己的情况：说明当前的安排；
3. 提出替代方案：给出可行的建议。

示例：

“我理解这次任务比较紧急，我也很想帮忙。不过我今天已经有既定安排，可能无法加班完成。要不我们一起来看看有没有其他方案？”

★ 总结：RLHF 不一定增加事实知识，但会让回答方式更符合人类期待。

5 RLHF 的核心数据与流程



6 RLHF 的核心组成



★ 强调：人类不是直接写规则，而是通过反馈塑造模型行为。

7 它和 SFT / Prompting 有什么区别?

维度	Prompting (提示工程)	SFT (监督微调)	RLHF (基于人类反馈的强化学习)
是否更新模型参数	✗ 不更新	✓ 更新	✓ 更新
数据来源	提示词 / 上下文	标准答案示例 (人写的正确答案)	人类偏好数据 (排序 / 选择 / 打分)
学习目标	临时引导模型输出	学习“应该输出什么” (模仿标准答案)	学习“哪个回答更受欢迎 / 更符合人类偏好”
类比	临时提醒 / 指路	示范教学	基于反馈继续打磨

★ SFT 更像“示范教学”，RLHF 更像“基于反馈继续打磨”。

8 优势与价值

- ✓ 提升对齐程度 (Alignment)
- ✓ 改善用户体验
- ✓ 更好地平衡帮助性与安全性
- ✓ 在开放式问答中更自然
- ✓ 是现代对话模型质量提升的重要环节



结果：更可信、更好用、更贴心的智能助手！

9 局限与挑战



- ⚠ 人类偏好本身有主观性
- ⚠ 标注成本高，数据收集慢
- ⚠ 奖励模型可能作弊，产生 reward hacking 风险
- ⚠ 可能削弱某些创造性或多样性
- ⚠ 不能替代事实正确性评估



10 总结



记忆点★

RLHF 的本质：
用人类反馈，而不是只用标准答案，
继续把模型打磨得更像一个合格助手。



一句话理解 RLHF：让模型不仅“知道”，还能“更懂你”。



目标：更有用、更安全、更贴心的 AI 助手！



大模型世界中的 DPO

Direct Preference Optimization

更简单，更直接，
让模型更懂你的偏好！

★ 不用复杂强化学习，也能让模型更符合人类偏好 ★

1 什么是 DPO?

DPO (直接偏好优化) 是一种基于人类偏好数据来对模型进行对齐的方法。它不直接训练奖励模型，也不需要像传统 RLHF 那样进行复杂的强化学习采样，而是直接利用“更喜欢回答 vs 不喜欢回答”来优化模型。



💡 核心思想：让模型更偏向人类更喜欢的答案。

2 它在大模型训练流程中的位置



DPO 常位于 SFT 之后，是从“会回答”走向“更符合偏好”的重要一步。

3 为什么需要 DPO?

- ✓ 让回答更符合人类偏好
- ✓ 提升有用性、礼貌性与安全性
- ✓ 降低啰嗦、攻击性或不受欢迎输出
- ✓ 让模型在多个可行答案中更偏向“更好”的那个
- ✓ 相比 RLHF 训练流程更简单



4 一个直观例子

用户问题：如何礼貌地拒绝同事不合理的加班请求？

A 回答 (不太喜欢) ❌

我现在很忙，做不完自己的事，没空加班，建议你自己想办法。

rejected / 不太喜欢

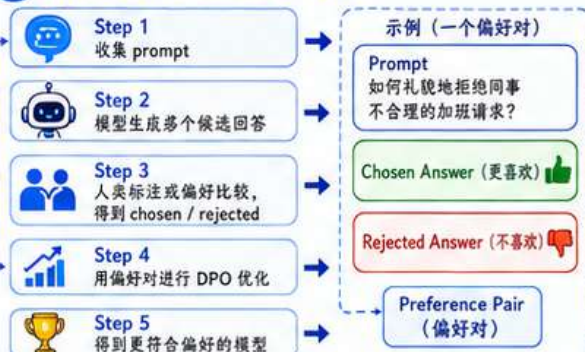
B 回答 (更喜欢) ✅

我理解项目比较紧急，为了保证工作质量，我目前手头也有重要任务在推进。要不我们一起来看看有没有办法调整优先级或分担？我尽量支持你！

preferred / 更喜欢

★ 总结：DPO 学的不是新知识本身，而是“哪种回答更受偏好”。

5 DPO 的核心数据与流程



6 DPO 的核心组成



目标：提高 chosen 概率，压低 rejected 概率

直观理解：让模型更倾向于生成人类更喜欢的答案。



7 它和 SFT / RLHF 有什么区别?

方法	数据来源	是否需要奖励模型	是否需要强化学习	学习目标
Prompting (提示工程)	无或手写示例	❌	❌	临时引导，无持续学习
SFT (监督微调)	指令-回答对	❌	❌	让模型学会按指令回答
RLHF (强化学习对齐)	偏好数据 (间接使用)	✓	✓ (PPO 等)	最大化奖励，对齐人类偏好
DPO (直接偏好优化)	偏好数据 (直接使用)	❌ (不需要奖励模型)	❌ (不需要复杂 PPO)	让模型更符合偏好

★ DPO：直接用偏好数据优化模型，不走复杂的奖励建模+强化学习不节。

8 优势与价值

- ✓ 训练流程更简单
- ✓ 稳定性更好，工程实现更轻量
- ✓ 降低 RLHF 的复杂度与成本
- ✓ 更容易复现与调试
- ✓ 在对齐任务上效果好



👍 更简单的对齐方式，更快的落地效果！

9 局限与挑战

- ⚠️ 依赖高质量偏好数据
- ⚠️ 偏好可能有主观性
- ⚠️ 不一定直接增加事实知识
- ⚠️ 数据分布不好会影响效果
- ⚠️ 仍需配合安全评估与高线/在线评测



10 总结



★ DPO 的本质：

不是让模型单纯“知道更多”，而是让它在多个答案里更倾向于给出人类更喜欢的那个。

🧠 记忆点：

RLHF 更像“直连全流程对齐”，DPO 更像“直接用偏好来教模型选更好的答案”。

💡 一切的目标，都是让 AI 更有用、更安全、更贴心！

★ 让模型更懂你，从 DPO 开始！

Keep It Simple, Align It Right!



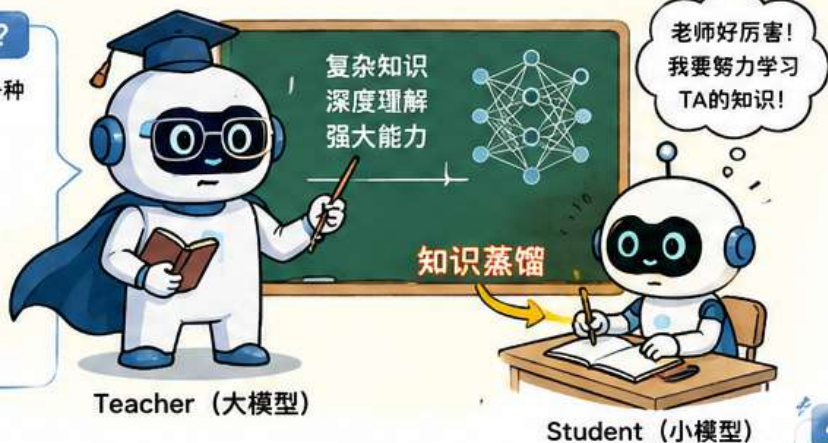
大模型世界中的 Distillation (知识蒸馏)

—— 让小模型学会大模型的智慧

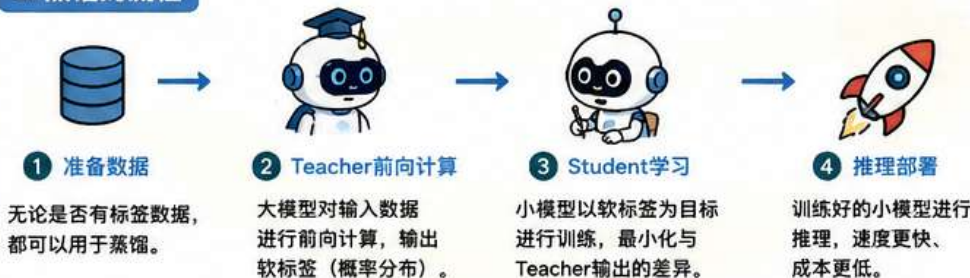
1. 什么是Distillation?

Distillation (知识蒸馏) 是一种模型压缩技术。

它让一个小模型 (Student) 学习一个大模型 (Teacher) 的输出行为或“知识”，从而在保持性能的同时，显著降低模型的参数量、计算量和部署成本。



3. 蒸馏的流程



5. 蒸馏的好处



2. 核心思想

大模型 (Teacher) 生成的“软标签” (soft targets) 包含了更丰富的概率分布信息，比真实标签 (hard labels) 更能反映数据之间的相似性与细微关系。小模型通过模仿这些软标签来学习大模型的行为模式。



4. 训练目标 (损失函数)

蒸馏训练通常结合两部分损失:

(1) 蒸馏损失 (模仿软标签)

$$L_{\text{distill}} = KL(\sigma(z_T/T) || \sigma(z_S/T))$$

- z_T : Teacher 的 logits
- z_S : Student 的 logits
- σ : softmax 函数
- T : 温度系数 ($T > 1$ 时更软)
- KL: KL散度

(2) 监督损失 (可选, 使用真实标签)

$$L_{\text{ce}} = \text{CrossEntropy}(y, \sigma(z_S))$$

- y : 真实标签 (one-hot)
- 有助于提升小模型的泛化能力

$$\text{总损失: } L = \alpha \cdot L_{\text{distill}} + (1 - \alpha) \cdot L_{\text{ce}}$$

- α 控制两种损失的权重

温度系数 T 的作用

T 越大, 概率分布越平滑, 包含的相对关系信息越多。常用范围: 2 ~ 20

6. 常见的蒸馏方法



7. 典型应用场景



一句话总结

Distillation 就像是“名师带徒”：
大模型 (名师) 把自己的解题思路和经验 (软标签) 教给小模型 (徒弟)，
让徒弟在保持高水平的同时，变得更轻量、更高效、更容易应用！

小贴士

- 并不一定要在同结构的模型之间蒸馏, 可以跨架构、跨模态。
- 蒸馏不仅用于压缩, 也可用于提升小模型的泛化能力。
- 许多开源小模型 (如 TinyBERT, DistilBERT, MiniLM) 都是蒸馏的成功范例!





大模型世界中的 GRPO



Group Relative Policy Optimization: 一种用于大模型对齐与强化学习训练的优化方法

1 什么是 GRPO?

GRPO (Group Relative Policy Optimization, 组相对策略优化) 是一种用于大模型后训练/对齐的强化学习优化方法。

它不是单纯学知识, 而是让模型在多个候选回答之间, 通过相对比较学习“哪种回答更好”。

核心思想: 在一组候选中做相对优势比较, 再更新策略。

2 它在大模型训练流程中的位置



GRPO通常位于SFT之后, 用于让模型从“会回答”进一步变成“更符合偏好、更稳健”。

3 为什么需要 GRPO?

- 1 让回答更符合人类偏好
- 2 提升有用性、清晰性与稳健性
- 3 减少低质量、含糊或不一致回答
- 4 适合复杂推理和长回答优化
- 5 相比一些传统RLHF流程, 训练实现可更简洁



4 一个直观例子



示例问题: 如何礼貌地拒绝同事不合理的加班请求?

A 让于生硬

不行, 我很忙, 做不了。



B 礼貌但不清楚

我可能没时间, 抱歉哈。



C 礼貌、清晰、给出替代方案

我理解你现在比较着急, 不过我手头也有自己的任务要按时完成。我们可以一起看看有没有其他同事可以帮忙, 或者把优先级调整一下, 您看可以吗?



GRPO会根据这组候选的相对表现, 鼓励模型更倾向生成像 C 这样的回答。

5 GRPO 的核心数据与训练流程



★ 重点: 组内比较 + 相对奖励, 而不是孤立看单个回答。

6 GRPO 的核心组成



★ 先生成一组, 再比较优劣, 再朝更优方向学习。

7 它和 SFT / RLHF / DPO 有什么区别?

方法	特点
SFT (监督微调)	基于标准答案示例学习; 目标学会按要求回答。
RLHF (基于人类反馈的强化学习)	基于人类反馈做强化学习; 通常包含奖励模型与策略优化。
DPO (直接偏好优化)	直接用偏好对进行优化; 不显式做完整强化学习循环。
GRPO (组相对策略优化)	对一组候选做相对比较并优化; 强调组内相对优势, 适合高效偏好优化。

8 优势与价值



- 1 能直接利用组内比较信号
- 2 对复杂回答质量优化更有效
- 3 提高对齐效果与回答稳定性
- 4 训练信号更丰富, 不只看单一答案
- 5 在一些场景下兼顾效果与训练效率

9 局限与挑战



- 1 依赖候选回答质量
- 2 奖励信号设计很关键
- 3 可能存在奖励偏差与分布偏差
- 4 训练不当仍可能过度优化表面指标
- 5 对工程实现和评估体系有要求

10 总结



GRPO 的本质: 不是只问“答得对不对”, 而是在一组答案里比较“哪个更好”, 再让模型朝更好的方向学习。

记忆点

先成组, 再比较, 再优化。

适用场景: 对齐人类偏好, 提升复杂问答质量、长文本生成优化、安全与稳健性增强等。

一句话记住: 组内比较驱动学习, 优势相对, 策略更优!





大模型世界中的 RLVR

Reinforcement Learning with Verifiable Rewards

让模型从“可验证结果”中学习：用能自动核验对错的奖励信号优化推理与任务完成能力



1 什么是 RLVR?



RLVR (基于可验证奖励的强化学习) 是一种让大模型根据“结果是否可被自动验证”来获得奖励并继续优化的训练方法。它不是依赖人类主观偏好打分,而是依赖可程序化检查、可自动判定对错的反馈信号。
适合数学、代码、规则明确、答案可检验的任务。

💡 核心思想: 能自动验对错, 就能把它变成训练奖励。

2 它在大模型训练流程中的位置



RLVR 通常位于 SFT 之后, 用于继续提升推理、代码、数学和工具使用等能力。

3 为什么需要 RLVR?

- 1 人工偏好标注昂贵, 而可验证任务可以自动打分。
- 2 能减少主观性, 提高训练信号稳定性。
- 3 特别适合数学、代码、逻辑推理、格式约束等任务。
- 4 可以鼓励模型探索更有效的解题过程与更正确的最终答案。
- 5 有助于提升复杂任务中的可靠性与可重复优化能力。



4 一个直观例子

求解: 若 $3x + 5 = 20$, x 等于多少?



候选答案 A

$x = 4$



候选答案 B

$x = 5$



自动验证

代回检查 $\rightarrow 3 \times 5 + 5 = 20$, 成立。✔

★ RLVR 会给可验证正确的结果更高奖励, 让模型更倾向产出正确答案。



5 RLVR 的核心数据与训练流程



6 RLVR 的核心组成



7 它和 SFT / RLHF / DPO 有什么区别?

方法	主要特点	关注点	反馈来源
SFT	学习标准示例	像“看例题学答题”	人工准备的正确示例
RLHF	强化学习 + 人类偏好	人更喜欢哪种回答	人类偏好打分
DPO	偏好对优化	直接用偏好对做优化, 不显式走完整 RL 奖励流程	偏好对 (更优 vs 更差)
RLVR	依赖可验证奖励	答案能否被自动证明/测试为正确	自动验证器 (可程序化)

8 优势与价值

- ✔ 可自动扩展训练数据反馈
- ✔ 对数学 / 代码 / 推理任务提升明显
- ✔ 奖励更客观、稳定
- ✔ 便于持续迭代优化
- ✔ 有助于提升答案正确率与工具调用成功率。



9 局限与挑战

- ⚠ 只适用于“能验证”的任务, 开放式创作不一定适合。
- ⚠ verifier 设计不好会误导训练
- ⚠ 可能只优化“可测指标”, 忽视更广泛质量
- ⚠ 奖励黑客 (reward hacking) 风险
- ⚠ 训练成本、采样成本与稳定性仍需权衡



10 总结



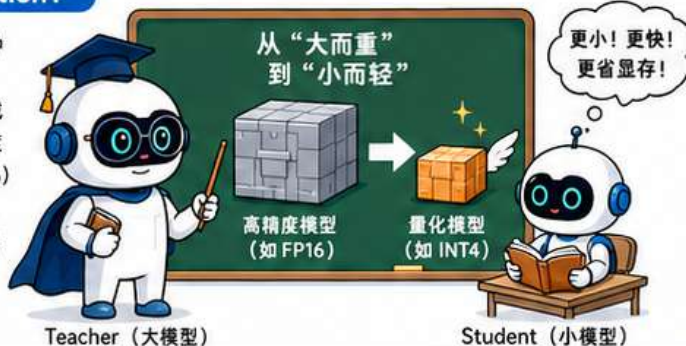
大模型世界中的 Quantization (量化)

—— 让大模型变得更小、更快、更省显存



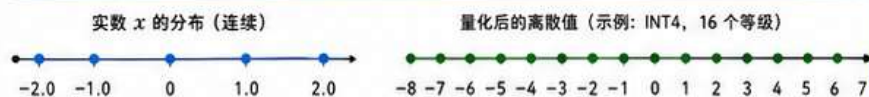
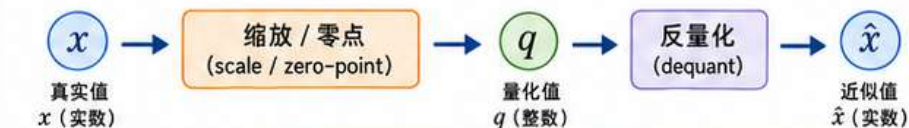
1. 什么是 Quantization?

Quantization (量化) 是一种模型压缩与加速技术。它将模型中的权重、激活值或 KV Cache 等数据, 从高精度格式 (如 FP32 / FP16 / BF16) 转换为低比特表示 (如 INT8、INT4, 甚至 2-bit), 从而降低内存占用、加速推理并提升部署效率。



2. 核心思想

推理时, 很多参数并不需要极高精度。我们把连续的实数值映射到有限个离散值上。

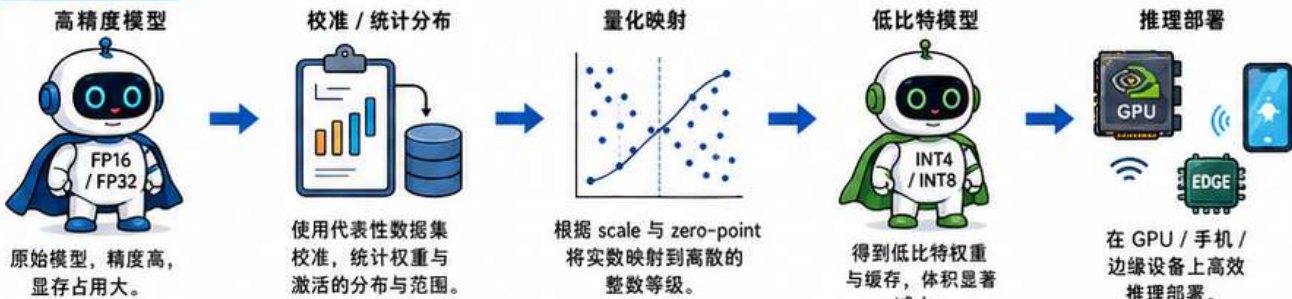


本质:
用更少的比特表示近似数值

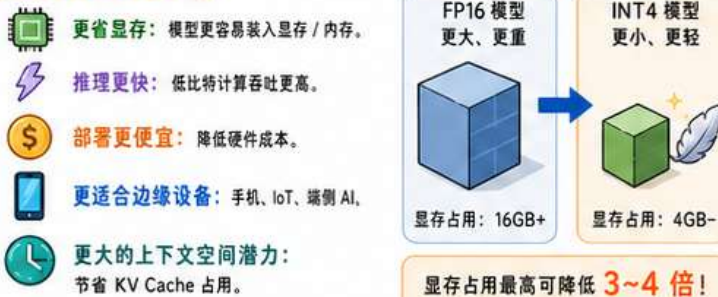
3. 常见量化类型



4. 量化流程



5. 为什么有用?



6. 代价与挑战



7. 典型应用场景



一句话总结

Quantization 就像把大模型从“高清原图”压缩成“高效轻量版”, 在尽量保持效果的前提下, 用更少比特换来更低成本、更快推理和更易部署。



小贴士

- 量化不等于简单压缩, 它需要尽量保留模型能力。
- 常见精度格式: FP32、FP16、BF16、INT8、INT4。
- 实际效果取决于模型结构、数据分布和量化算法。





大家好！
今天我们聊聊
大模型系统中的
Batching！

大模型世界中的 Batching

把多个请求一起处理：提升吞吐、提高 GPU 利用率、降低平均成本



1 什么是 Batching?

Batching (批处理) 是把多个样本、多个请求、或多个 token 的计算合并在一起，让模型并行处理的一种方法。它的核心目标不是改变模型本身，而是提高硬件利用率与系统吞吐量。



本质上是“把很多小计算拼成一批更高效地算”。



4 一个直观例子：4 个用户同时问问题

无 Batching (依次单独跑)



有 Batching (合并成一个批次一起跑)



结论：总吞吐更高，但单个请求未必总是最快。

2 它在大模型系统中的位置



在训练和推理中都能用到 batching，但在 LLM 服务里尤其常见于在线推理系统。



3 为什么需要 Batching?

- ✓ 单个请求太小，GPU 容易“吃不饱”
- ✓ 把多个请求一起算，可以提升吞吐量
- ✓ 降低单位请求平均成本
- ✓ 更适合高并发服务场景
- ✓ 能与调度器、KV cache、continuous batching 等机制配合



GPU 利用率对比

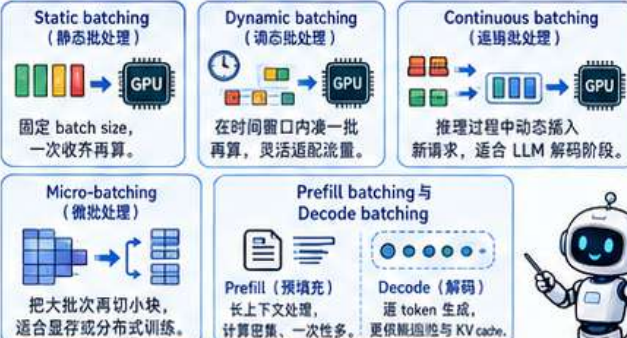
不使用 Batching



使用 Batching



5 常见的 Batching 类型



6 Batching 的核心收益

- 🏆 提高吞吐 (Throughput)
- 📊 提高 GPU/TPU 利用率
- 💰 降低平均延迟成本
- 🚀 改善服务扩展性
- 🏢 更适合企业级在线推理平台



7 它和 Latency / Throughput 有什么关系?

概念	Latency (时延)	Throughput (吞吐)
定义	单个请求从发出到完成所需的时间。	单位时间内处理的请求数 / token 数。
关注点	用户体验：越低越好	系统整体能力：越高越好

Batching 的权衡



Batching 往往优先优化 throughput，而不一定让每个用户都获得最低 latency。batch 太大，等待时间可能上升。

8 局限与挑战

- ⚠️ 请求长度不同，padding 浪费算力
- ⚠️ batch 太大可能增加首 token 延迟
- ⚠️ 显存占用上升
- ⚠️ 在线服务中需要复杂调度
- ⚠️ 长短请求混合时，尾部慢请求会拖累整批
- ⚠️ 解码阶段与预填充阶段最优策略不同



9 实用建议

- ✓ 根据业务目标在 latency 和 throughput 间权衡
- ✓ 对相近长度请求分桶 (bucketing)
- ✓ 监控 batch size、queue time、TTFT、TPS、GPU 利用率
- ✓ 对 prefill 和 decode 分别优化
- ✓ 高并发场景优先考虑 dynamic / continuous batching
- ✓ 配合 KV cache、调度器、并行策略一起设计



10 总结



Batching 的本质：
不是让模型更聪明，而是让模型在同样硬件上更高效地工作。

★ 记忆点：
单个请求想更快，看 latency；系统整体想更强，看 batching 与 throughput。





大模型世界中的 KV Cache

——让自回归生成更快、更省重复计算

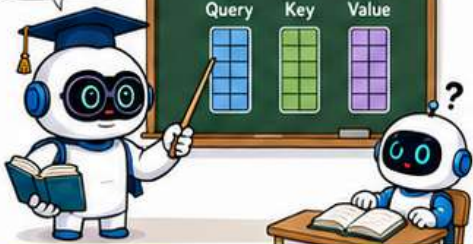
缓存过去，
专注未来！



1. 什么是 KV Cache?

KV Cache (键值缓存) 是在 Transformer 自回归推理时，把历史 token 在每一层注意力中的 Key 和 Value 向量缓存起来。这样生成下一个 token 时，不必对之前所有 token 反复计算 K/V，只需为新 token 计算新的 Q/K/V，再与缓存拼接完成注意力计算。它主要用于推理阶段，尤其是解码 (decoding) 阶段。

缓存 K 和 V，
少做重复功！



2. 核心思想

Token 序列

我

喜欢

学习

大模型

...

新 token
(下一步)

历史 token 的 K/V 已缓存



新 token 到来

计算 Q_{new}
计算 K_{new}
计算 V_{new}

与缓存做注意力



本质是“用空间换时间，避免重复计算”

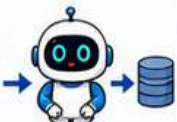
Attention ($Q_{new}, K_{cache} \cup K_{new}, V_{cache} \cup V_{new}$)

3. 生成流程

(1) Prefill / 首次编码

整段提示词一次性通过模型，建立每层的 KV Cache。

提示词：
“我喜欢学习
大模型。”

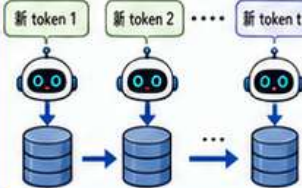


一次性计算所有层的
K/V 并写入缓存

Prefill 建缓存 → Decode 复用缓存 → 高效生成

(2) Decode / 逐 token 生成

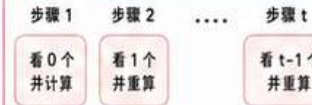
每一步只处理最新 token，并复用历史缓存。



4. 为什么能加速?

无缓存会重复计算大量历史信息；有缓存则只处理新增部分，速度大幅提升。

无缓存：第 t 步重复看前 1...t-1 个 token 并重算



重复计算多，越长越慢！

有缓存：第 t 步复用历史，只新增当前 token



只做增量计算，速度飞快！

带来这些常见收益：



5. 缓存里到底存了什么?

第 L 层
Self-Attention

KV Cache (第 L 层)
 K_L_{cache}
 V_L_{cache}

• 对每一层都会缓存历史 token 的 K 和 V；通常不缓存 Q，因为 Q 只与当前 token 相关。

第 2 层
Self-Attention

KV Cache (第 2 层)
 K_2_{cache}
 V_2_{cache}

• 缓存的 tensor 形状与层数、头数、序列长度、head_dim 以及数据类型有关。

第 1 层
Self-Attention

KV Cache (第 1 层)
 K_1_{cache}
 V_1_{cache}

形状示例
(batch_num_heads, seq_len, head_dim)

6. 代价与挑战



- 显存/内存占用会上升，尤其在长上下文和大 batch 下更明显。
- 上下文越长，缓存越大。
- 多轮对话、并发服务时需要管理不同请求的缓存。
- 更换提示词或截断上下文时，缓存不能直接复用。
- 量化 KV Cache、分页缓存 (Paged KV Cache) 等是常见优化方向。

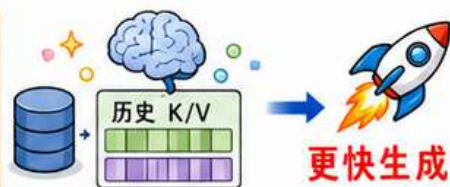
7. 典型应用场景



8. 一句话总结



KV Cache 就像给大模型做“历史记忆缓存”：把过去 token 的 Key/Value 先存起来，让模型生成下一个 token 时不再从头回看，因此显著加速解码。



更快生成！

9. 小贴士

- ★ KV Cache 主要提升的是解码阶段，不是训练阶段。
- ★ 它不会改变模型知识本身，只改变推理效率。
- ★ 长上下文推理时，缓存优化往往非常关键。





大模型世界中的 vLLM 与 SGLang



高性能大模型推理与服务框架：提升吞吐、降低延迟、让部署更高效

1 什么是 vLLM / SGLang?



vLLM: 面向大模型推理与服务的高性能引擎，重点解决 KV cache 管理和吞吐问题。



SGLang: 面向大模型应用与服务的高性能框架，强调结构化生成、程序化控制与高效运行时。

它们都服务于推理加速，但侧重点不同。

2 它们在大模型系统中的位置



用户请求

tokenizer /
prompt



推理服务层
(引擎 / Runtime 层)



模型执行
(GPU)



输出结果

vLLM 与 SGLang 位于推理服务 / 运行时层，连接前端请求与模型执行。

3 为什么需要它们?



提高吞吐 (throughput)



降低延迟 (latency)



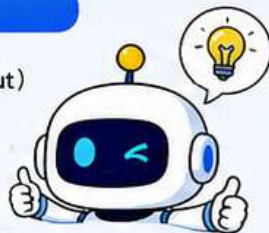
提高 GPU 利用率



更好支持长上下文、多用户并发、生产部署



降低单位请求成本



4 vLLM 的核心特点



PagedAttention:
把 KV cache 按页管理，
减少显存浪费



Continuous batching:
持续批处理，提高并发吞吐



**Prefix caching /
chunked prefill**:
减少重复计算



适合高吞吐 API 服务
与在线推理

大量并发请求涌入



vLLM 引擎 (持续批处理)

Paged KV Cache
(按页管理)



高吞吐、低延迟输出

5 SGLang 的核心特点



既有前端语言 / 编排能力，
也有高性能 runtime



擅长 structured generation、
控制生成流程、复杂提示程序



**RadixAttention /
prefix caching**:
复用共享前缀



支持工具调用、结构化输出、
Agent / workflow 场景

提示程序 (Prompt Program)

```
if ...  
| call(tool)  
| ...  
| 输出 JSON
```

SGLang Runtime

GPU 模型 (GPU)

结构化输出 (JSON / 表格 / 工具结果)



7 一个直观例子：企业知识库问答系统

用 vLLM:

重点是大量并发问答
高效跑起来



用 SGLang:

重点是“检索 -> 推理 ->
结构化输出”流程更好组织起来



8 优势与价值



更快部署开源大模型服务



更稳定地支持生产流量



支持更复杂的生成应用



对 RAG、Agent、企业 Copilot 特别重要



帮助团队在性能、成本、可扩展性之间取得平衡



9 局限与挑战



配置和调优有门槛



不同模型 / 硬件环境效果不同



高性能依赖良好的 batch、
cache、并行策略



复杂工作流也会增加系统复杂度



10 总结



vLLM 的本质:

把大模型推理服务做得
更快、更省、更能扛并发



SGLang 的本质:

让复杂生成任务既能编排，
又能高效执行



记忆点:

“vLLM 更像高性能引擎，
SGLang 更像高性能语言 + 运行时框架。”





大模型世界中的 Flash Attention



让注意力计算更快、更省显存：通过更高效的数据流优化 Transformer 推理与训练

1 什么是 Flash Attention?

Flash Attention 是一种高效的注意力计算方法，用于 Transformer。
它的核心思想，并不是改变注意力计算的结果，而是通过分块 (Tiling) / 分块计算、Kernel 融合等技术，减少慢速的显存访问 (HBM 读写)，从而提升速度并降低显存占用。



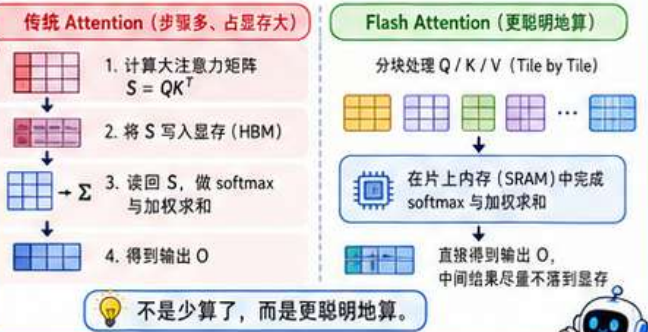
2 它在大模型系统中的位置



3 为什么需要 Flash Attention?

- 传统注意力需要产生巨大的中间矩阵 (如 $S = QK^T$)
- 中间结果频繁读写 HBM (显存)，代价极高
- 序列越长，注意力矩阵越大，计算与显存成本急剧上升
- GPU 常因显存带宽瓶颈而无法充分利用计算能力
- Flash Attention 通过优化数据流，提升速度、降低显存占用，支持更长上下文

4 一个直观例子



5 Flash Attention 的核心思想



6 它和普通 Attention 有什么区别?

对比项	普通 Attention	Flash Attention
中间矩阵	需要完整的 QK^T 矩阵	不显式存储完整矩阵，分块计算
显存占用	大，随序列长度平方增长	大幅降低，通常节省 2~10 倍+
速度	中等，受限于显存带宽	更快，受益于更少的访问
长上下文表现	成本急剧上升，易 OOM	更友好，支持更长上下文
实现复杂度	相对简单	较复杂，需要专用 Kernel 优化

7 优势与价值

- 提升训练/推理吞吐：减少显存瓶颈，充分利用 GPU 计算能力
- 降低显存占用：节省显存，支持更大模型或更大批量
- 让更长上下文成为可能：更长的输入，更多的理解与推理能力
- 提升 GPU 利用率：计算单元更忙，整体效率更高
- 对大模型部署和微调很重要：成本更低，体验更好

8 局限与挑战

- 依赖特定 GPU / Kernel 优化：需要硬件与驱动支持
- 并非所有硬件收益相同：老旧 GPU 或受限环境收益有限
- 工程实现更复杂：需要专业的 Kernel 与调度优化
- 对某些极小任务收益有限：小序列/小模型提升不明显
- 需要配合系统优化：如 batching、KV cache、并行策略等

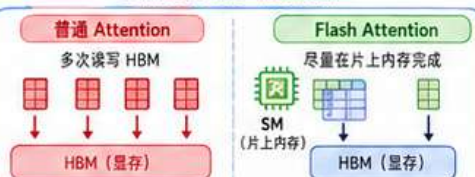
9 常见应用场景 (现代 LLM 栈中广泛使用)



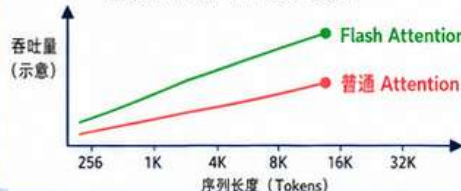
10 总结

Flash Attention 的本质：
不是改变注意力机制“学什么”，
而是优化它“怎么计算”，
用更少的显存搬运完成同样的注意力结果。

显存搬运量对比 (越少越好)



速度提升示意 (不同序列长度)



记忆点：
少搬数据，
快算注意力。





大模型世界中的 Speculative Decoding

——让小模型先打草稿，大模型再快速验收

先猜一波，
再验一波！
更快更强！



1. 什么是 Speculative Decoding?

Speculative Decoding 是一种推理加速技术。

由小的 Draft Model (草稿模型) 先连续生成多个候选 token；
再由大的 Target Model (目标模型) 并行验证这些候选。
若被接受，可一次性提交多个 token，从而减少昂贵的大模型解码步数，显著提升生成速度。



2. 核心思想

普通自回归解码 (每步只生成 1 个 token)

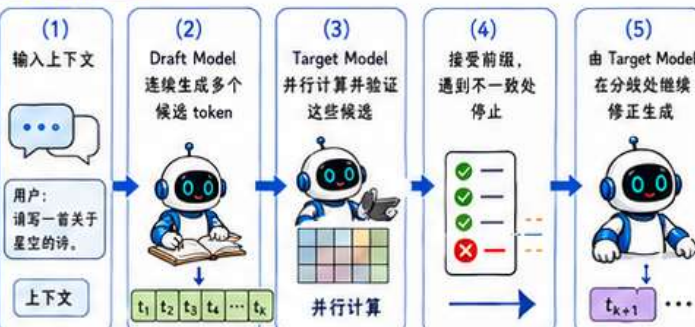


Speculative Decoding (先草稿，再并行验收)



核心本质：用便宜计算做草稿，用昂贵计算做验证。

3. 工作流程



4. 接受 / 拒绝机制

将 Draft Model 的候选 token 逐个与 Target Model 的分布进行比较。

位置	Draft 候选	Target 分布 (示例 top-1)	结果
1	t_1	t_1 (概率 0.62)	✓
2	t_2	t_2 (概率 0.57)	✓
3	t_3	t_x (概率 0.48)	✗
4	t_4	— (未比较)	—

✓ 接受

若候选 token 被接受，则提交并继续。

$t_1 - t_2 \dots \rightarrow$

✗ 拒绝 (在第 3 个不一致处停止)

回滚到分歧处，让 Target Model 从该位置继续生成。

t_3'

★ 接受越多，提速越明显！

5. 为什么会更快?

标准自回归解码

每生成 1 个 token，都需要 1 次昂贵 Target Model 前向计算。



— = 一次昂贵的 Target Model 前向

Speculative Decoding

Draft Model 一次生成多个 token，Target Model 只需 1 次并行验证，可能接受多个 token。



草稿 (便宜) 并行验证 (昂贵) 修正: $t_{m+1} \dots$

带来的收益

更低延迟 (TTT 更快)

更高吞吐 (tokens/s)

更好 GPU 利用率

长文本生成收益更显著

6. 关键影响因素

Draft model 与 target model 的匹配程度
越匹配 (知识 / 分布越接近)，接受率越高。

一次草稿的长度 / block size
候选越多，潜在收益越大，但接受率可能下降。

接受率 (acceptance rate)
直接决定能被一次性提交的 token 数量。

target model 本身的计算开销
模型越大越贵，提速收益越明显。

batch / serving system design
并行验证与调度策略影响整体吞吐与效率。

注意权衡

天平图标

Draft Model 太弱
→ 接受率低，收益差；
太强
→ Draft 成本上升，收益也会打折。

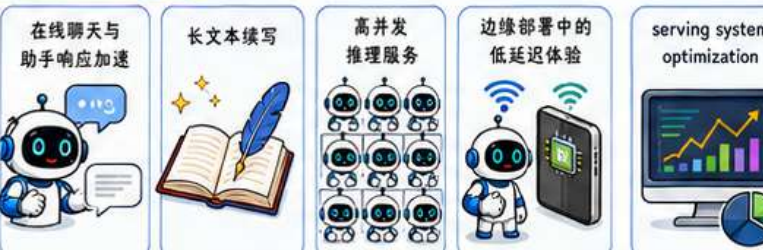
7. 代价与挑战

- ⚠ 系统实现更复杂 (调度、回滚、并行验证等)。
- ⚠ 需要额外的 Draft Model (训练 / 选型 / 维护成本)。
- ⚠ 接受率不稳定，效果依任务而变。
- ⚠ 内存 / 调度 / 工程优化要求更高。
- ⚠ 并非所有场景都能大幅提速。

收益很诱人，工程不简单！



8. 典型应用场景



9. 和 KV Cache / Quantization 的关系

三者是互补的优化手段，常常协同使用。



10. 一句话总结

Speculative Decoding 就像 “小模型起草，大模型审稿”：
先用便宜模型大胆猜，再由大模型快速验收，从而在尽量保持质量的同时显著提升生成速度。

更快生成，
更好体验！





大模型世界中的 Benchmarks



用标准化测试集评估模型能力：看模型会什么、强在哪、弱在哪

1 什么是 Benchmarks?

Benchmarks (基准测试) 是用一组标准化任务、数据集和评测规则，系统衡量大模型能力的方法。它不是随便问几个问题，而是尽量可复现、可比较、可量化地评估模型表现。



可比较



可复现



可量化

2 它在大模型研发流程中的位置



Benchmark 常用于模型训练后验收，也用于版本迭代、模型选型和上线前评估。

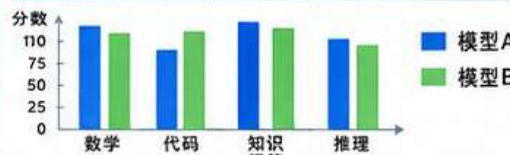
3 为什么需要 Benchmarks?

- ✓ 量化模型能力，而不是凭感觉判断
- ✓ 比较不同模型 / 不同版本
- ✓ 发现优势和短板
- ✓ 追踪优化是否真的有效
- ✓ 帮助选型、部署和产品决策



4 一个直观例子

维度	数学	代码	知识问答	推理
模型A	86	78	90	82
模型B	79	88	84	85



★ 结论：不同 benchmark 维度下，模型强项不同，不能只看单一分数。

5 常见评测维度



不同 benchmark 关注的能力不同。

6 常见的 Benchmark 形式

- 1 选择题 / 多项选择
- 2 开放问答
- 3 代码生成与单元测试
- 4 数学题自动判分
- 5 多轮任务 / Agent 任务
- 6 人工评测 + 自动评测结合

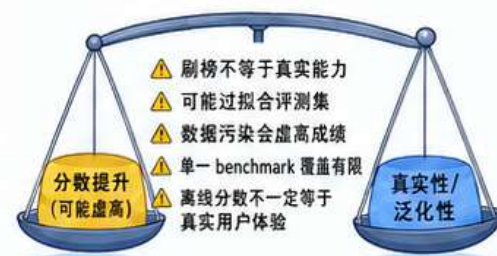
题型不同，评估信号也不同。

7 常用指标有哪些?

- Accuracy 答对比例
- Pass@k 代码等任务中，前 k 个结果里是否有正确答案
- F1 F1 兼顾准确率与召回率
- Win rate 与另一个模型对比时的胜率
- Hallucination / Safety 指标 幻觉率、安全合规表现

不同任务要用不同指标，不能一把尺子量所有能力。

8 局限与挑战



警惕：分数高 ≠ 真正强

9 实用建议

- ★ 不要只看总分，要看分项
- 组合多个 benchmark 一起看
- 关注数据污染和评测方式
- 离线评测 + 在线评测结合
- 与业务目标对齐：选最适合场景的 benchmark

多维度、多方法，才能看清真本事！



10 总结



Benchmarks 的本质：不是给模型贴标签，而是用标准化方法持续理解模型能力边界。

记忆点

Benchmark 像考试，但真正目标不是分数，而是看清模型能力。





大模型世界中的 Paged Attention

——像操作系统分页一样管理 KV Cache，让推理更高效

把显存用在
刀刃上！



1. 什么是 Paged Attention?

Paged Attention 是一种在大模型推理阶段使用的 attention / KV Cache 管理方法。

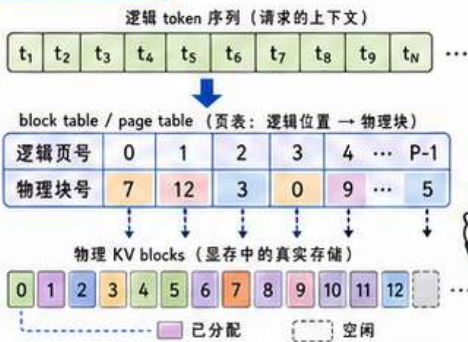
它将每个请求的 KV Cache 按固定大小切分成多个 block (页)，而不是一次分配一整块连续显存。

这就像操作系统的虚拟内存分页机制，可以减少内存碎片，提升 GPU 显存利用率。

像操作系统
分页一样！



2. 核心思路



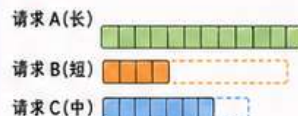
逻辑连续，
物理可不连续；
像页表一样
做地址映射！

TIP
K 和 V 分别存储，
按 block 组织；
attention 通过映射
读取所需 K/V。

3. 为什么需要它?

传统方式 (连续分配)

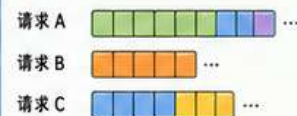
- ❌ 每个请求需一整块连续显存
- ❌ 不同长度导致内部/外部碎片
- ❌ 难以动态调度，批处理受限



大量空洞 = 浪费显存

Paged Attention (分页管理)

- ✅ 按固定块分配，灵活拼接
- ✅ 显存利用率高，碎片少
- ✅ 更利于动态调度与批处理



空间紧凑 = 更高利用率!

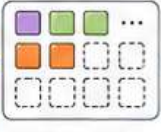
4. 工作流程 (一次推理过程)

(1) 用户请求进入



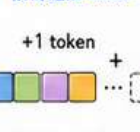
新请求到达，
创建会话。

(2) 分配 KV blocks



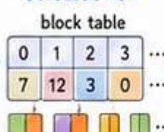
为该请求分配若干
空闲 block，并在
block table 中记录。

(3) 生成新 token 按需追加 block



生成新 token 时，
若当前 block 已满，
再分配新 block。

(4) 通过 block table 找到历史 K/V



根据页表把逻辑位置
映射到物理 block，
读取所需的 K/V。

(5) 执行 attention 计算



使用查到的历史 K/V
与当前 Q 进行 attention
计算，得到输出。

(6) 请求结束 回收 block

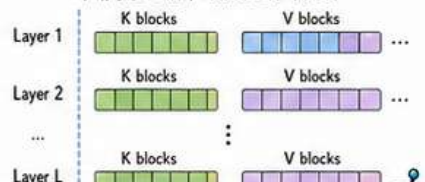


请求完成后，释放
其占用的所有 block，
归还内存池。

5. 它到底存了什么?

- 每个 Transformer 层在推理阶段主要存储: K 和 V (通常不存 Q)。
- 每层的 K、V 都按固定大小的 block 组织。
- 每个请求通过 block table 将逻辑位置映射到物理 block。

一个请求在 L 层中的存储结构 (示例)



固定 block 大小 (如 16/32/64 token)

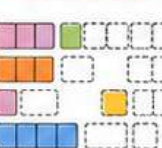
已分配 (紫色) 空闲 (绿色)

6. 带来的好处

- ✅ 更高显存利用率
- ✅ 更少内存碎片
- ✅ 支持连续批处理 (continuous batching)
- ✅ 更适配长上下文和高并发
- ✅ 请求结束后更容易回收内存
- ✅ 提升 serving 吞吐

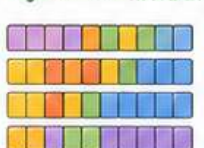
显存使用对比 (示意)

传统方式 (连续分配)



空洞多，浪费严重 😞

Paged Attention (分页管理)



紧凑利用，显存更高效! 😊

7. 代价与挑战

- 需要维护 block table / metadata，有额外的内存和计算开销。
- 存在一定的地址映射和调度复杂度。
- block 大小需要权衡：
太小：管理开销高；太大：浪费空间。
- 工程实现复杂，需要高效的 kernel 支持与优化。

block 大小权衡示意



16 32 64 128 (token)
常见范围

8. 典型应用场景

大模型在线 推理服务



多用户聊天 机器人



长文本生成 / 长上下文问答



高并发 API 服务



与 KV Cache、 Continuous Batching 等优化联合使用

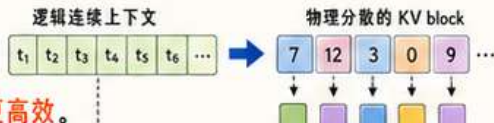


Paged Attention 是现代高性能 LLM Serving 框架的关键技术之一，与 KV Cache、continuous batching、调度器等共同构成高效推理生态。

9. 一句话总结



Paged Attention 就像给大模型的 KV Cache 做“分页管理”：
把逻辑上连续的上下文，映射到物理上分散的显存块中，
从而减少碎片、提升显存利用率，并让高并发推理更高效。



10. 小贴士

- ★ 它主要优化的是推理阶段，不是训练阶段。
- ★ 它通常和 KV Cache、连续批处理一起发挥效果。
- ★ 本质是“借鉴操作系统分页思想做推理内存管理”。

少浪费，
多输出!





好数据
成就好模型!

大模型世界中的 Golden-datasets

用高质量、可信赖的标准样本集，评估、校准和守护大模型效果



1 什么是 Golden-datasets?

Golden-datasets (黄金数据集/金标准数据集) 是精心构建的、高质量、可信赖的样本集合，具有可靠的标签或参考答案，作为稳定的标准，用于评估、验证、回归测试、校准和优化大模型系统。

它不是简单的“更多数据”，而是更可信的数据!



2 它在大模型研发流程中的位置



3 为什么需要 Golden-datasets?

- ✓ 建立稳定的质量基线，避免“感觉变好/变坏”
- ✓ 可复现、可比较，跨版本/模型公平对比
- ✓ 发现回归 (性能下降) 和新问题
- ✓ 验证关键业务场景是否达标
- ✓ 减少主观判断，提升评估一致性
- ✓ 支持发布决策与风险控制
- ✓ 持续提升模型的可靠性与用户信任



4 一个直观例子 (企业知识库问答助手)

用户问题	参考答案 (或关键证据)	评分标准 (示例)	新模型版本对比	
			v1.0	v2.0
公司报销的时效是多久?	一般报销需在发生后30天内提交,特殊情况可延长至60天。	要点完整(2) 时效正确(2) 引用制度(1)	3/5	5/5
差旅机票标准如何规定?	国内经济舱,单程不超过800元;超标准需审批。	航班舱位(1) 价格上限(2) 审批规则(1)	2/4	4/4
年假可以累积吗?	当年未休年假可累计不超过5天,次年清零。	可累计(1) 上限5天(1) 清零规则(1)	1/3	3/3

通过同一份 Golden-datasets, 直观判断模型是否更好、更稳、更可靠!



5 Golden-datasets 的核心组成



6 常见类型



7 它和 Benchmark / 训练集 / 测试集 有什么区别?

项目	Benchmark (基准测试集)	训练集 (Training Set)	普通测试集 (Test Set)	Golden-datasets (黄金数据集)
目的	公开标准比较模型能力	让模型学习知识与模式	评估泛化能力, 避免过拟合	保障关键场景效果, 回归检查与决策
数据来源	公开/社区/学术	海量数据 (网页、书籍等)	内部或公开	精挑细选的业务或高价值场景
覆盖范围	广泛但较通用	尽可能多	通用分布	聚焦关键业务与高风险点
标签质量	较高 (标准化)	不需要标签或噪声较多	较高	极高 (高质量、可信赖)
稳定性	随版本更新	持续变化	可定期更新	版本化、稳定、可追溯
评估方式	标准指标 (如EM、F1)	—	指标评估为主	指标 + 人工评审 + 业务规则
关注点	能力上限对比	学习效果	泛化性能	可靠性、可用性、业务价值

8 优势与价值

- ✓ 更高的信任度: 来源可靠、标签准确, 结果可被信赖
- ✓ 更精细的诊断: 定位具体问题 (知识、推理、检索、工具等)
- ✓ 更容易 A/B 比较: 同一把尺子, 公平、可复现
- ✓ 支持持续迭代: 跟踪改进趋势, 验证优化是否有效
- ✓ 贴合真实业务目标: 围绕关键场景与 KPI 构建
- ✓ 对 RAG、Agent、企业 Copilot 尤其重要
- ✓ 促进安全发布与风险可控: 作为发布门禁与监控依据



9 局限与挑战

- ⚠ 构建成本高: 需要专家知识与大量标注工作
- ⚠ 标注一致性难: 不同标注者可能存在差异
- ⚠ 可能过时: 知识、政策、系统变化导致过期
- ⚠ 覆盖不可能完全: 永远有新场景与长尾问题
- ⚠ 过拟合风险: 模型可能“只会做这题”
- ⚠ 需要持续维护: 版本管理、数据清洗、标注校对



10 实用建议 + 总结

• 实用建议 •

- 从高价值场景开始, 先小而精, 逐步扩展
- 制定清晰的评分标准与 Rubric
- 严格版本管理与变更记录
- 包含难例、边界例与反例 (Hard Cases)
- 定期回顾与更新, 保持数据“新鲜”
- 结合自动化指标与人工评审, 双保险

• 总结 •

Golden-datasets 的本质:
不是数据越多越好,
而是要有一批可信、稳定、可复用的
标准样本, 帮我们持续判断
模型到底有没有变得更好。





大模型世界中的 LLM-as-Judge

——让大模型充当评委，对回答进行打分、比较与裁决

公正评判
更高效！



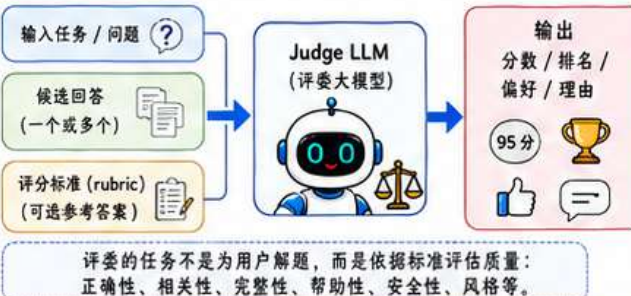
1. 什么是 LLM-as-Judge?

LLM-as-Judge 是指使用大语言模型作为评估者 (Evaluator)，根据评分标准 (rubric)、指令、参考答案或任务上下文，对候选回答、生成内容或模型输出进行评估。

它可以对结果进行打分、排序、比较或分类裁决。



2. 核心思想

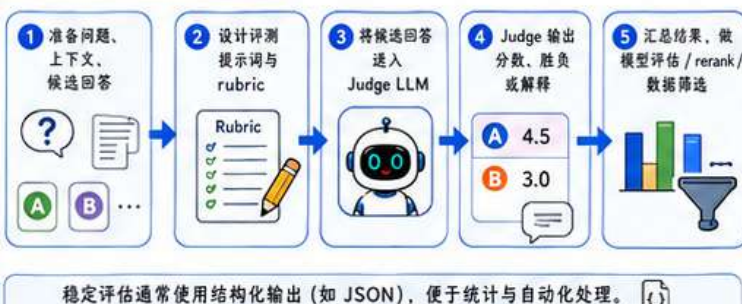


评委的任务不是为用户解题，而是依据标准评估质量：正确性、相关性、完整性、帮助性、安全性、风格等。

3. 常见判分方式



4. 工作流程 (评测流水线)



稳定评估通常使用结构化输出 (如 JSON)，便于统计与自动化处理。

5. 为什么有用?

- ✓ 评测速度快
- ✓ 成本低于大规模人工评审
- ✓ 可大规模自动化
- ✓ 维度丰富 (不仅看正确性，也看风格、帮助性、安全性)
- ✓ 便于 A/B 测试、回归测试、持续迭代

人工评审 vs LLM Judge



6. 典型应用场景



7. 代价与挑战

- ⚠ 偏见 (bias)
- ⚠ 位置偏差 (先看/后看顺序影响结果)
- ⚠ 冗长偏好 (verbosity bias)
- ⚠ 对自家模型或相似风格的偏好
- ⚠ 对领域知识/事实的误判
- ⚠ 结果不稳定 (温度、提示词、措辞影响)
- ⚠ 解释看似合理但未必真正可靠

8. 如何提升可靠性?



9. 和 Reward Model / Human Eval 的关系



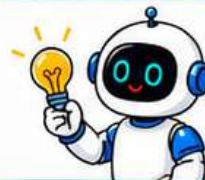
LLM-as-Judge 常用于高级评测、数据构造、自动比较，不完全替代人工。

10. 一句话总结

LLM-as-Judge 就像让另一个大模型当“AI评委”：
按照规则对候选回答进行打分、比较和排序，从而更高效地评估与优化大模型系统。

小贴士

- Judge 的好坏，很大程度取决于提示词、rubric 与上下文
- 在事实性任务中，最好结合参考答案或外部证据
- 最佳实践通常是“LLM Judge + 人工抽检”





大模型世界中的 Regression tests

用固定测试集持续检查模型和系统有没有“越改越坏”：让升级、微调、提示词改动与工具链迭代更可控

质量守护，
稳定迭代！



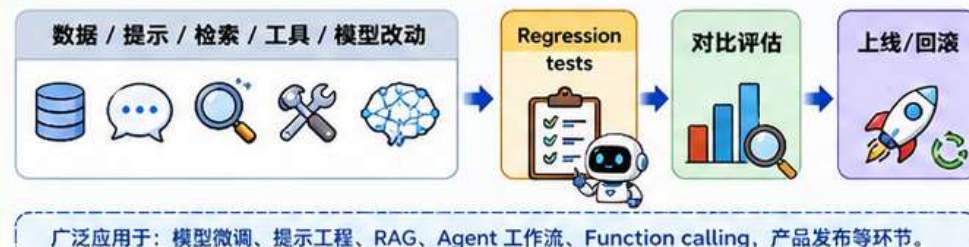
1 什么是 Regression tests?

Regression tests (回归测试) 是一组可重复运行的测试用例，用于在模型、提示词、检索、工具或工作流程发生改动后，验证那些“原本能正确完成的能力”是否仍然正常，质量、安全与格式等约束是否发生退化。

★ 不是测模型“会不会”，
而是测“本来会的现在还会不会”。



2 它在大模型研发流程中的位置



3 为什么需要 Regression tests?

- ⚠ 改动可能在某些方面提升，但在另一些方面破坏。
- ⚠ 提示词微调可能导致结构化输出变差。
- ⚠ 检索策略改动可能降低召回，丢失关键信息。
- ⚠ 工具/函数改动可能导致调用失败或结果错误。
- ⚠ 模型升级可能改变答复风格，影响安全与合规。

局部优化，可能带来全局退化。



4 一个直观例子：企业知识助手

改动前 (v1)		改动后 (v2)	
测试问题	结果	测试问题	结果
请总结请假制度	✓ 正确简洁	请总结请假制度	✓ 更全面 (更好)
输出 JSON 格式联系人信息	✓ 格式正确	输出 JSON 格式联系人信息	✗ 格式错误 字段缺失
查询差旅报销上限	✓ 给出准确数值	查询差旅报销上限	✗ 数值错误 (退化)



结论：回归测试能在线上发现这些“变坏”的能力，避免影响用户。

5 Regression tests 的核心组成



6 常见测试类型



7 它和 Benchmark / Golden-datasets 有什么区别?

类别	Benchmark	Golden-datasets	Regression tests
目标	衡量通用能力 (广度)	提供可信评估集 (标准样本)	防止退化 (变化后的稳定性)
范围	广泛、覆盖多任务	覆盖关键能力的代表样本	聚焦关键场景与能力
变化频率	较低 (定期更新)	低 (相对稳定)	高 (随迭代持续维护)
结果解读	看排名/分数	作为评估依据	看差异/是否退化



简言之：Benchmark 看整体水平，Regression tests 盯关键能力是否退化。

8 优势与价值

- ✓ 减少差评/事故发布风险。
- ✓ 让迭代更安全，敢于快速试错。
- ✓ 支持 CI/CD，自动化质量门禁。
- ✓ 帮助对比不同版本，定位问题根源。
- ✓ 发现隐藏的破坏 (格式/工具/安全等)。
- ✓ 提升工程信心，降低线上回滚成本。
- ✓ 尤其适合 RAG、Copilots、Agents 场景。



稳定的底线，才有持续的创新。

9 局限与挑战

- ⚠ 难以覆盖所有真实场景与长尾问题。
- ⚠ 测试可能不稳定 (Flaky tests)。
- ⚠ 期望答案会演化，维护成本高。
- ⚠ LLM 非确定性导致结果波动。
- ⚠ 测试集可能过时，与用户需求脱节。
- ⚠ 可能过度拟合测试，忽视创新。
- ⚠ 需要持续维护与投入工程资源。(更多场景)



追求平衡：高价值覆盖 × 可持续成本

10 实用建议 + 总结

实用建议

- ✓ 优先覆盖关键用户旅程与核心任务。
- ✓ 构建一个小而精的高价值回归测试集。
- ✓ 区分 Smoke (冒烟) 测试与 全量测试。
- ✓ 明确 Pass/Fail 规则，模糊情况人工复核。
- ✓ 对比改动前后 (A/B 对比)，看差异。
- ✓ 持续更新测试集，跟随产品与数据演化。

总结

Regression tests 的本质：
不是追求更多分数，而是在每次改动后，
确认关键能力没有悄悄变差。

记住这句话
改一次，测一次；先保住底线，再追求提升。





大模型世界中的 Hallucination Detection

——识别大模型什么时候在“一本正经地胡说”，提升回答可靠性



1 什么是 Hallucination Detection?

识别大模型输出中“事实错误、无依据编造、引用不实、推理失真”的技术。

模型输出:

2023年,世界上最高的山是马纳斯鲁峰,高度是9,163米,位于尼泊尔和中国之间。



2 核心思想



3 常见幻觉类型



4 检测流程 (常见 claim-level verification)



目标: 逐条核验事实, 给出整体可信度判断。

5 怎么检测?



多种方法组合使用, 能获得更好的检测效果与可靠性。

6 为什么重要?



7 代价与挑战

- 检索不到证据不代表一定错误
- 证据本身可能过时或冲突
- 检测器也可能误判 (假阳/假阴)
- 长答案、多跳推理更难检查
- 实时检测会增加延迟和成本



8 典型应用场景



9 和 Guardrails / RAG / Human Review 的关系



10 一句话总结



Hallucination Detection 就像给大模型配了一位“事实审计员”; 专门检查回答是否有依据、是否可信, 从而让模型更可靠、更安全。



小贴士

- 幻觉检测不能保证 100% 正确, 但能显著降低风险;
- 最佳实践通常是“检索 + 生成 + 检测 + 人审”;
- 对高风险业务, 宁可拒答或提示不确定, 也不要瞎编。





大模型世界中的 Red Team

主动找漏洞，
才能更安心上线。



用对抗式测试主动发现风险：在真实上线前暴露大模型的安全、合规与鲁棒性问题

1 什么是 Red Team?

- Red Team (红队测试) 是站在“攻击者、误用者、极端用户”的角度，系统性地挑战大模型与 AI 系统。
- 目标不是破坏系统，而是**主动发现**漏洞、风险边界和失败模式。
- 核心是：在上线前“**故意找茬**”，帮助系统变得更安全、更稳健。



2 它在大模型研发流程中的位置



★ Red Team 往往发生在上线前，也会在版本更新后持续进行。

3 为什么需要 Red Team?

- 大模型可能出现幻觉、越狱、提示注入、泄露敏感信息等问题。
- 普通评测只看“平均表现”，但红队更关注“极端失败案例”。
- 帮助降低安全、合规、品牌和业务风险。
- 对 Agent、RAG、工具调用、多轮对话系统尤其重要。



4 一个直观例子：企业客服助手



5 Red Team 的核心测试维度



6 常见 Red Team 方法



7 它和 Benchmark / Regression tests 有什么区别?

类型	关注点	目标	衡量方式	典型问题
Benchmark	衡量平均能力与分数	比较模型整体表现	分数、榜单、平均值	模型能做得怎么样?
Regression tests	检查改动后是否退化	保证改动后不受差	对比改标是否下降	改动后是否变差了?
Red Team	寻找高风险失败模式	暴露极端风险与薄弱点	风险级别、严重性、影响	最坏情况下会出什么问题?

★ 总结：Red Team 不追求“平均分高”，而是追求“极端风险低”。

8 优势与价值

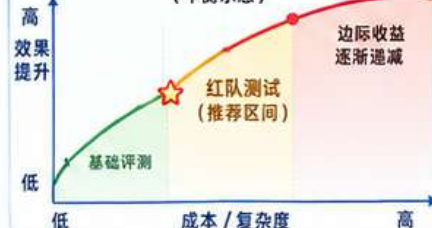
- 提前暴露高风险问题
- 帮助修复护栏与策略
- 提升上线信心
- 支持安全发布与审计
- 提升用户信任
- 为对齐、安全评估和产品决策提供依据



9 局限与挑战

- 不可能覆盖所有攻击方式
- 红队结果依赖测试样本与人员经验
- 模型更新后旧结论可能失效
- 修复一个漏洞可能引入新问题
- 需要结合自动化评测、监控与回归测试持续进行

效果提升 vs 成本 / 复杂度
(平衡示意)



10 实用建议 + 总结

- 优先覆盖高风险业务场景
- 建立红队案例库与分级机制
- 修复后回归验证
- 将 Red Team 融入发布流程
- 对 Agent、RAG、工具调用单独设计攻击用例



总结流程：持续对抗，持续变得更安全



“Red Team 的本质：
不是为了“证明模型很差”，
而是为了在真实用户之前，
先把**危险的问题**找出来。”





大模型世界中的 Trajectory Evaluation

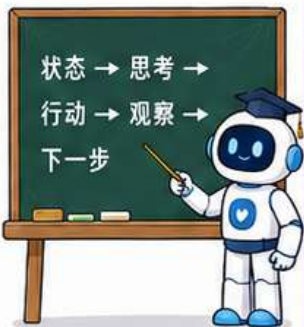
——不仅看最终答案，还要评估推理过程、行动路径与中间步骤质量



1 什么是 Trajectory Evaluation?

Trajectory Evaluation (轨迹评估) 是对大模型在完成任务过程中的“全过程”进行评价的方法，不只看最后答案是否正确，还看中间推理、工具调用、检索、决策顺序和纠错过程是否合理。

它广泛应用于 Agent、工具使用、多步推理、工作流评估等场景。



2 核心思想



评估的是过程质量，而不是只看结果对错

只看最终答案 (结果评估)

- 只判断对/错
- 无法发现中间问题
- 难以定位失败原因



VS

看完整轨迹 (过程评估)

- 评估每一步质量
- 发现过程问题
- 指导改进与优化



3 一条轨迹长什么样?



轨迹中可能包含的要素

- Thought (思考)
- Action (行动)
- Observation (观察)
- Tool Result (工具结果)
- Reflection (反思 / 自我修正)

一条轨迹 = 一系列“想什么 -> 做什么 -> 看到什么 -> 下一步做什么”

4 怎么评估?

常见评估维度 (示例 Rubric)

任务完成度	得分 (1-5)
步骤正确性	★★★★★
逻辑连贯性	★★★★★
工具使用是否合适	★★★★★
检索证据是否相关	★★★★★
是否走了冤枉路	★★★★★
效率 / 步骤成本	★★★★★
安全与合规	★★★★★
错误恢复能力	★★★★★

总分卡 (示例)

总分: 4.2 / 5



评语:

整体过程合理，检索较相关，中间有一次冤枉路，但能自我修正并给出正确答案。



5 常见评估方式



① Rule-based / rubric 打分

基于人工制定的规则或评分标准进行打分。



② LLM-as-Judge 评审

让大模型作为评审，依据指令和 rubric 进行打分与点评。



③ 与标准轨迹对比

将模型轨迹与高质量参考轨迹对比，计算差异或相似度。



④ 基于结果 + 过程的混合评估

同时考虑最终结果的正确性与过程质量。

评估粒度: Step-level (步骤级) / Action-level (动作级) / Episode-level (整条轨迹级)

6 为什么重要?



找到模型失败的真正原因



区分“结果错”与“过程错”



优化 Agent / RAG / Tool-use 系统



帮助训练 reward model 或过程监督



提升可靠性、可解释性与可调试性

只看最终答案



- 结果导向
- 最盒难诊断
- 难以持续优化

看完整轨迹



- 过程可见
- 问题可定位
- 持续改进可行



7 典型应用场景



Agent 任务执行



RAG 问答链路评估



代码 Agent



多步数学推理



搜索 / 浏览 / 工具调用系统



Benchmark 与自动化评测平台

8 代价与挑战



评估成本高，轨迹很长



好轨迹不一定唯一



中间步骤标注难



Judge 可能有偏差



需要定义清晰 rubric



不同任务的评估标准不同



有时结果对，但路径低效或不安全



9 一句话总结

Trajectory Evaluation 就像给大模型配了一位“过程质检员”：不只检查最后答得对不对，还检查每一步走得是否合理、高效、可靠。



小贴士

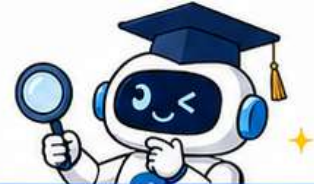
- 轨迹评估常用于 Agent、工具调用和复杂推理任务
- 最佳实践通常是“结果评估 + 过程评估”结合
- 好的 rubric 与高质量样例会显著提升评测可靠性





大模型世界中的 Observability

——看见系统在做什么、为什么这样做、以及哪里出了问题



1 什么是 Observability?

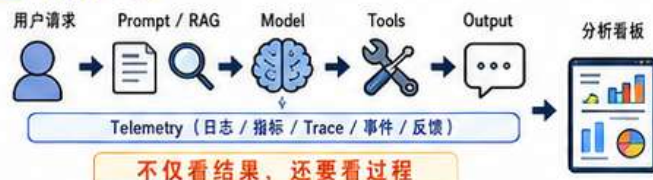
Observability (可观测性) 是指通过系统输出的结果和遥测数据, 理解 LLM 应用的内部状态与行为的能力。

它不只是看“系统挂没挂”, 更要看 Prompt、检索、工具、模型、延迟、成本、质量等是否按预期工作。

核心数据来源: 日志、指标、链路追踪、事件与用户反馈。



2 核心思想



帮助我们回答三大问题:

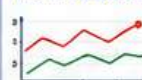
发生了什么?

为什么发生?

影响有多大?

4 典型数据形态

Metrics (数值指标)



示例:

• qps: 128
• p95_latency: 2.3s
• tokens_in: 3.2k
• tokens_out: 1.1k
• cost: \$0.024

Logs (结构化日志)

```
{
  "session_id": "s_001",
  "prompt_version": "v3.2",
  "model": "gpt-4o",
  "event": "model_call",
  "latency_ms": 1840,
  "status": "success"
}
```

包含: 上下文、参数、结果、状态、错误等

Traces (链路追踪)

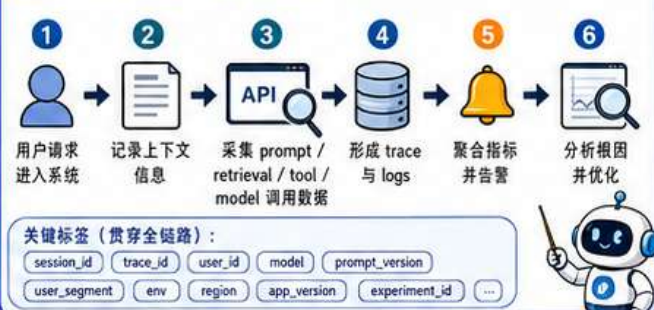


Events / Feedback (事件与用户反馈)

• tool_call_result
• retrieval_docs
• content_filter_hit
• judge_score: 0.82
• user_rating: 4/5
• user_comment: ...

业务事件、评估分数、用户反馈与标签

5 工作流程 (观测流水线)



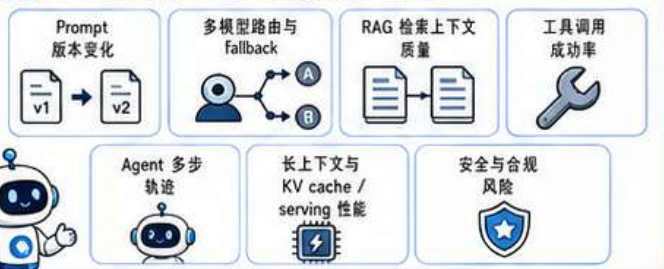
3 观测什么?



6 为什么重要?



7 在 LLM 系统中特别要关注什么?



8 代价与挑战

- ⚠️ 数据量大, 存储与计算成本高
- ⚠️ 隐私与脱敏要求 (PII / 敏感内容)
- ⚠️ 质量评估不总是自动可得
- ⚠️ 多步 Agent 链路复杂, 依赖多
- ⚠️ 采样策略与字段设计困难
- ⚠️ 观测 ≠ 自动修复, 需要配合 Evaluation / Guardrails / Human Review



9 典型应用场景



10 和 Monitoring / Evaluation / Debugging 的关系



11 一句话总结

Observability 就像给大模型系统装上

“可视化体检仪 + 行为记录仪”:

让你看见请求经过了什么、慢在哪里、贵在哪里、错在哪里, 从而更快优化系统。

12 小贴士

- 💡 最佳实践是“日志 + 指标 + trace + 质量评估”结合
- 🎯 关键是结构化字段与统一 trace id
- ✅ 观测的目标不是收集更多数据, 而是更快定位问题、持续优化



花得明白
用得更好!

大模型世界中的 Cost tracking

看清 token、模型、工具与基础设施成本，让大模型应用更可控、更可优化

账单



1 什么是 Cost tracking?

- Cost tracking (成本追踪) 是对大模型系统在开发、测试、部署和运行过程中产生的各类成本进行持续记录、拆解、归因和分析。
- 不只是“花了多少钱”，还要回答“钱花在哪”“为什么花”“值不值”。
- 关注对象包括：模型调用成本、token 成本、推理算力成本、工具调用成本、存储与检索成本、人工评估成本等。

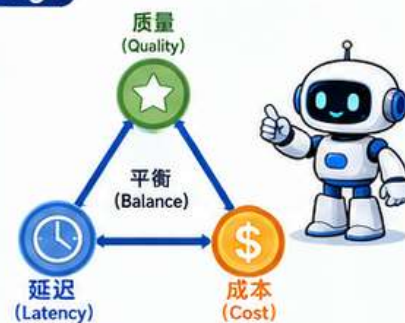


2 它在大模型系统中的位置



3 为什么需要 Cost tracking?

- 控制预算，避免成本失控
- 比较不同模型 / prompt / workflow 的性价比
- 发现高成本瓶颈（长上下文、反复检索、多轮调用等）
- 支持产品定价、资源规划与容量管理
- 帮助在质量、延迟、成本之间做权衡



4 一个直观例子

场景：企业知识助手回答
“帮我总结这份政策并给出行动建议”



一次请求的成本拆解 (示例)

项目	说明	估算成本 (USD)
输入成本	系统提示词 + 用户问题 + 检索内容 (输入 token)	\$0.0042
输出成本	模型回答 (输出 token)	\$0.0068
检索成本	向量检索 + 重排	\$0.0015
工具成本	数据库 / 搜索接口调用	\$0.0020
模型选择成本	大模型一次 + 小模型一次	\$0.0030
总成本	本次请求合计	\$0.0175

结论：复杂工作流程中，“一次回答”往往由多段成本共同组成。

5 Cost tracking 的核心组成

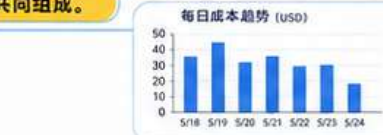


6 常见追踪维度

- 按模型: GPT / Claude / 开源模型等不同模型成本
- 按 token: input / output / cached / reasoning token
- 按功能: 问答、摘要、代码生成、客服、Agent
- 按用户 / 团队 / 租户
- 按链路: 检索、重排、工具调用、推理
- 按环境: 开发、测试、生产
- 按时间: 小时、天、周、月

7 常见指标有哪些?

- Cost per request (每次请求成本)
- Cost per user (每用户成本)
- Cost per successful task (每次成功任务成本)
- Token usage (token 用量)
- Average / P95 cost (平均 / 高分位成本)
- Cost breakdown share (成本构成占比)
- ROI / 性价比 (效果与成本对比)



8 优势与价值

- ✓ 让费用更透明
- ✓ 支持优化 prompt 与上下文
- ✓ 帮助模型路由和降级策略
- ✓ 发现低效 Agent 循环或重复调用
- ✓ 支持 A/B test 的成本比较
- ✓ 让企业级治理、预算和结算更容易



9 局限与挑战

- 成本不只来自模型 API，还包括工程基础设施与人工成本
- 复杂 Agent 链路的归因困难
- 不同供应商计费方式不同，口径不统一
- 只看成本可能牺牲质量
- 需要与 latency, success rate, 用户满意度一起看



10 实用建议

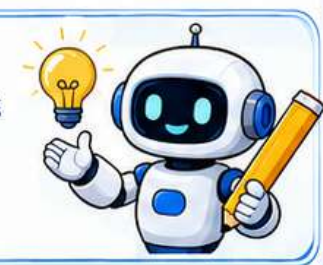
- ✓ 从最小可用埋点开始：记录模型、token、工具、总成本
- ✓ 建立统一成本口径与标签体系
- ✓ 按工作流追踪，而不是只看单次模型调用
- ✓ 结合质量评测一起看，不要只压低成本
- ✓ 对高频任务做缓存、压缩上下文、模型分层路由
- ✓ 设置预算告警和异常监控



总结

Cost tracking 的本质：
不是单纯记账，而是让大模型系统“可观测、可解释、可优化”。

记忆点：
看见成本，才能管理成本；
管理成本，才能规模化落地 AI。



看清每一分投入，优化每一步决策，让大模型应用持续创造价值!

模型 | 工具 | 向量库 | 数据库 | GPU/算力 | 存储 | 网络



大模型世界中的 Guardrails

——给大模型加上“安全护栏”，让输出更安全、更可控、更合规



1 什么是 Guardrails?

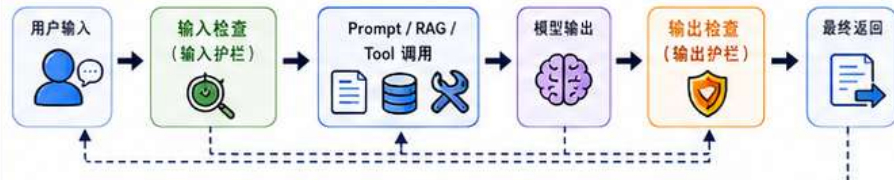
Guardrails (护栏) 是指在大模型应用周围构建的一系列规则、检查与控制机制，用于约束不安全、低质量、越权或不合规的输入/输出与行为。

它不是模型本身，而是围绕模型的控制层与保护层。

目标：在保障体验的同时，把风险挡在护栏之外。



2 核心理念



本质：在关键节点进行约束、检测与拦截，确保系统按预期、安全地运行。

3 常见护栏类型



4 工作流程

- 1 识别输入风险：关键词、意图、权限、敏感信息等
- 2 决策与处理：改写/拒绝/路由到合适流程
- 3 工具调用监控：校验工具选择、参数与返回
- 4 生成内容检查：内容安全、合规、事实性、格式等
- 5 兜底与人工 review：降级、提示、转人工处理
- 6 记录与反馈：日志、埋点、评估与策略迭代

5 常用技术手段



6 为什么重要?

- 减少有害/不当输出，保护用户与品牌声誉
- 保护业务系统与数据，防止越权与滥用
- 提升输出质量与可靠性，减少幻觉与错误
- 支持企业合规与监管要求，降低法律风险
- 让大模型在真实业务中更安全落地与扩展



7 典型应用场景

- 客服与对话机器人：防越权、敏感内容、无效承诺
- 企业知识助手：权限隔离、数据合规、可追溯
- 编程/代码助手：安全代码、禁止恶意操作
- 医疗/金融/法律支持：合规、免责声明、风险提示
- 多智能体/工具系统：约束工具使用与协作行为

8 代价与挑战

- 过度拦截（误杀）影响体验与业务效率
- 拦截不足仍可能导致风险事件
- 增加延迟与成本（多次检查、模型评判）
- 误报/漏报的平衡很难把握
- 策略维护复杂，需要持续更新与评估
- 与用户体验、自由表达的潜在冲突



9 和 RAG / Human Review / Observability 的关系



11 一句话总结

Guardrails 是给大模型应用加上的“安全护栏”和“交通规则”，在关键节点进行约束、检测与拦截，让系统更安全、更稳定、更可控、更合规。



12 小贴士（最佳实践）

- 分层防御：多层护栏组合，避免单点失效
- 基于风险设计：按场景与风险等级配置不同强度的策略
- 规则 + 模型结合：先规则后模型，互补验证更可靠
- 高风险留给人工：高风险场景转人工，人机协同更安心
- 持续评估与迭代：建立评估集、监控告警，持续优化护栏策略





大模型世界中的 PII detection



识别并保护个人敏感信息：让大模型应用更安全、更合规

1 什么是 PII detection?

PII (Personally Identifiable Information, 个人身份可识别信息) 是指能够单独或与其他信息结合识别特定自然人身份的信息。

PII detection 是识别文本、对话、表单、日志、知识库或模型输入输出中个人敏感信息的过程。

目标：发现敏感信息，并决定如何保护。

常见 PII 示例

- 姓名
- 手机号
- 邮箱
- 身份证号
- 住址
- 银行卡号
- 账号
- 病历号
-

2 它在大模型系统中的位置



PII detection 贯穿全流程：输入前、过程中、输出后、入库前、日志前都能发挥作用！

3 为什么需要 PII detection?

- 防止敏感信息泄露
- 满足隐私与合规要求
- 降低提示词注入和数据外泄风险
- 保护用户、员工和客户数据
- 支持企业上线真实业务场景
- 帮助日志、评测、标注环节安全化

★ 先识别，再使用；先保护，再生成。

4 一个直观例子



识别 → 脱敏 → 使用 → 受控输出

5 PII detection 的核心方法

- 规则 / 正则表达式** (手机号、邮箱、身份证号等格式型信息)
- NER / 序列标注** (识别人名、地址、机构、证件等实体)
- 分类模型 / LLM 判别** (识别上下文中的隐私语义与隐含信息)
- 字典与黑白名单** (已知敏感字段和业务词表)
- 混合方案：规则 + 模型 联合检测** (提升召回与精度，降低误报漏报)

结构化强的字段适合规则；语义复杂场景适合模型；生产环境常用混合策略。

6 检测后怎么处理?

- Masking (掩码)** 13812345678 → 138****5678
- Redaction (删除/遮盖)** 联系邮箱: xxx@example.com → ——
- Tokenization / Placeholder (占位替换)** [PHONE] [NAME] [EMAIL] [ID_NUM]
- Hash / Encryption (哈希/加密)** 13812345678 → 3f2a1e...9b7c
- Access control (按权限显示)** 管理员可见 / 普通用户隐藏
- Alert / Block (告警 / 拦截)** 发现高风险 PII → 告警 / 拦截请求

★ 识别不是终点，治理动作更关键！★

7 常见应用场景



任何涉及人、账号、证件、位置、联系信息、健康、财务的数据，都值得被识别与保护！

8 优势与价值

- 提升隐私保护能力
- 降低数据泄露和合规风险
- 让真实业务数据可更安全接入大模型
- 支持多部门协作与审计
- 提高企业对 AI 系统的信任度

安全性、可落地性、可控性提升



从被动防御 → 主动治理 → 持续优化

9 局限与挑战

- 误报与漏报并存
- 跨语言、口语化、缩写场景更难
- OCR 文本、截图、语音转写中更复杂
- 不同国家地区的隐私定义不同
- 上下文依赖强，单靠正则不够
- 检测本身也有延迟和成本



在安全效果与成本之间，找到最优平衡。

10 总结

PII detection 的本质，不是让模型“看不到一切”，而是在该保护的时候识别并保护敏感信息，让大模型在可用、可控、合规之间取得平衡。

好 AI，不只会回答，还要会保护隐私。



大模型世界中的 Feedback Loops

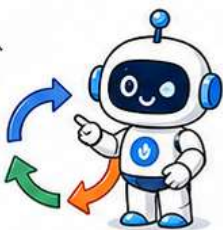
——让系统从真实使用中不断学习、纠错、优化与迭代



1 什么是 Feedback Loops?

Feedback Loops (反馈闭环) 是把模型输出后的结果、用户反应、质量评测与系统指标重新收集回来,用于持续优化 Prompt、RAG、工具调用、策略、模型与产品体验的机制。

它是一个持续循环的过程,而不是一次性的评估。



2 核心思想



本质: 让系统从“使用结果”中反向学习

3 反馈从哪里来?



用户显式反馈
(点赞/点踩/评分/举报)



用户隐式反馈
(点击、停留、是否采纳、是否追问)



人工评审 / 标注
(专家评审、标注数据)



自动评测
(LLM-as-Judge / Benchmark / Regression)



线上运行指标
(延迟、成本、成功率、异常率)



业务结果
(转化率、留存、完成率)

4 工作流程



5 可以优化什么?



反馈闭环优化的不只是模型本身,而是整个系统。

6 为什么重要?

- 持续提升回答质量
- 更快发现真实线上问题
- 降低幻觉、错误与失败率
- 更贴近用户需求与业务目标
- 提高稳定性、可解释性与 ROI
- 支持系统长期迭代

没有闭环

- 问题发现慢
- 反复踩坑
- 离用户需求远
- 幻觉与错误高
- 投入产出低
- 迭代停滞

VS

有闭环

- 问题早发现
- 快速定位与修复
- 更贴近需求
- 质量更高更稳定
- ROI 不断提升
- 持续进化

7 常见闭环类型



8 代价与挑战

- 反馈噪声大, 不一定真实反映质量
- 反馈有延迟, 奖励不立即出现
- 选择偏差 / 幸存者偏差
- 指标可能被“刷”或被优化错方向
- 需要隐私保护与数据治理
- 人工评审成本高, 闭环工程复杂



9 和 Observability / Evaluation / Guardrails 的关系



四者协同, 才能形成真正可持续的 LLMOps 闭环。

10 一句话总结

Feedback Loops 就像给大模型系统装上“学习回路”: 把真实世界中的使用结果、评价与信号重新送回系统, 帮助它持续变得更准确、更稳定、更有用。



小贴士

- 先定义清晰指标, 再收反馈
- 反馈要分层: 用户、评测、业务、系统
- 最佳实践通常是“自动化 + 人工复核 + 持续实验”



监控不是终点，
而是持续变好的
开始！

大模型世界中的 Model monitoring

持续观察模型的质量、延迟、成本与安全，让 LLM 系统稳定、可控、可优化



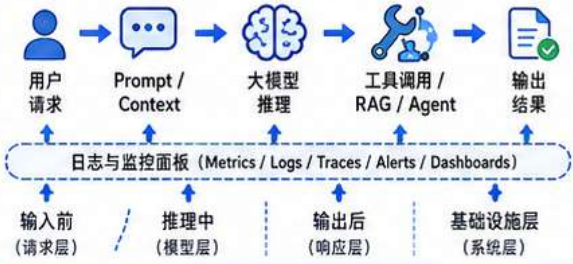
1 什么是 Model monitoring ?

Model monitoring (模型监控) 是指在模型上线后，持续跟踪其输入、输出、性能、质量、安全、成本与系统状态，及时发现异常、退化和风险。

★ 它不是一次性评测，
而是运行期持续观测。



2 它在大模型系统中的位置



3 为什么需要 Model monitoring ?

- 上线后环境会变化，真实用户问题与测试集不同
- 模型可能出现幻觉、质量波动或拒答异常
- 延迟、吞吐、GPU 利用率会变化
- token 消耗和成本可能失控
- 工具调用 / RAG / Agent 流程更复杂，更需要可观测性
- 安全、合规、PII、越权等风险需要持续监控

4 一个直观例子

没有监控 (事后才发现)

知识库更新后，检索变差 → 回答变旧或引用错误

为什么最近回答不准了？

用户不断投诉 问题扩大，影响信任

有监控 (及时发现 & 修复)

监控告警发现异常

命中率下降 ↓
幻觉率上升 ↑
延迟上升 ↑

定位原因：检索质量下降、提示词不佳

修复检索 & 提示词 恢复质量，用户满意度回升

5 监控哪些核心指标？

质量

- 准确率
- 人工评分
- 任务成功率
- Win rate
- 幻觉率

检索 / Agent

- Recall (召回率)
- 命中率 (Hit@k)
- 工具成功率
- 步骤完成率

系统

- Latency (延迟)
- TTFT (首 token 时间)
- TPS (每秒 token 数)
- 吞吐量 (GPS)
- 错误率
- 超时率

资源

- GPU 利用率
- CPU 利用率
- 内存利用率

成本

- input tokens
- output tokens
- 每次请求成本
- 每用户成本

安全

- 越狱命中 (Jailbreak)
- 敏感内容检测
- PII 检测 (个人信息)
- 异常访问 / 越权

6 常见监控方法

在线日志采集

采集请求、响应、元数据与系统日志

规则告警 (阈值)

设置阈值，触发告警通知

抽样人工评审

抽样评估质量，更贴近真实体验

LLM-as-a-Judge 辅助评分

用模型辅助打分，提高评估效率

Golden dataset 定期回放

定期在固定数据集上评测对比

Regression tests 回归测试

自动化测试，发现性能回退

用户反馈闭环 (点赞/踩、工单)

收集反馈，形成改进闭环

Dashboard 仪表盘与趋势图

可视化指标趋势，快速定位问题

7 优势与价值

- ✓ 及早发现问题，降低事故风险
- ✓ 帮助定位问题来源 (模型、检索、提示词、工具、数据)
- ✓ 支持 A/B 对比与持续优化
- ✓ 提升稳定性、用户体验和上线信心
- ✓ 帮助平衡质量、速度与成本
- ✓ 满足企业治理与审计需求

8 局限与挑战

- ❗ 质量不总能自动评估
- ❗ 指标很多，但不一定代表真实用户体验
- ❗ 多步 Agent 链路复杂，排障难
- ❗ 人工标注成本高
- ❗ 监控过多会增加日志与基础设施成本
- ❗ 隐私与合规要求限制数据留存

效果提升 (可观测性) vs 成本 / 复杂度 (资源消耗)

9 实用建议

- ✓ 先从最关键业务场景和少量核心指标开始，逐步扩展
- ✓ 将质量、延迟、成本、安全一起看，不要只看单一指标
- ✓ 对 Prompt、RAG、Tool calling 分层打点，定位更清晰
- ✓ 设定告警阈值和异常排查流程，明确责任人
- ✓ 定期回放 golden datasets 和 regression tests
- ✓ 保留用户反馈闭环 (点赞/踩、工单、NPS 等)
- ✓ 做版本对比，监控变更前差异 (A/B / Canary)

10 总结



★ Model monitoring 的本质：不是只让模型上线运行，而是让模型上线后可观察、可诊断、可改进。

记忆点

不监控，
就很难真正把
大模型系统管起来。



看见问题，才能持续把模型变得更稳、更准、更值。

可观测 | 可诊断 | 可优化 | 可信任



大模型世界中的 Prompt Injection



——当外部输入偷偷“带偏”模型，让系统偏离原本规则与目标

1 什么是 Prompt Injection?

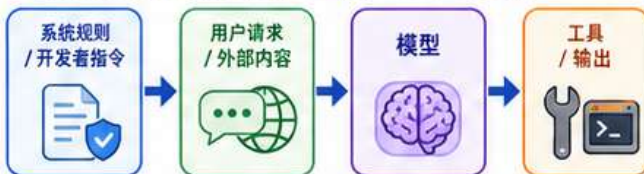
Prompt Injection 是一种攻击或风险：攻击者将恶意或误导性的指令，嵌入到用户输入、检索到的文档、网页、工具输出或多模态内容中，导致大模型 (LLM) 或 Agent 忽略原本的系统 / 开发者规则，泄露信息，或执行不该做的操作。



这不是对模型参数的攻击，而是输入层面的指令劫持问题。

2 核心理想

模型会把所有这些内容都当作“上下文 Token”来理解。



如果模型无法可靠地区分“可信指令”和“不可信内容”，外部文本就可能与原始指令竞争，甚至覆盖原有行为。

本质：不可信内容伪装成“应该遵循的指令”。

3 常见注入来源

1 用户输入注入



用户直接在对话中嵌入恶意指令，诱导模型改变行为或泄露信息。

2 RAG 检索文档注入



不可信文档被检索并作为上下文，其中隐藏的指令影响模型输出。

3 网页 / 邮件 / PDF / 网盘文件注入



访问的网页、邮件、PDF 或网盘文件中夹带恶意提示。

4 Tool output 注入



工具返回的内容（如搜索结果、API 响应、命令输出）被注入。

5 多模态内容中的隐藏提示



图片、音频、视频或文档中的隐藏文字 / 指令被模型提取并执行。

4 典型工作流程

1 用户提出任务



2 系统检索文档 / 调用工具



3 外部内容中混入恶意提示



4 模型将其与正规指令一起误读



5 模型被诱导，产生越权行为 / 错误回答 / 信息泄露



6 系统输出异常结果



5 常见攻击效果 (在 LLM 应用与 Agent 系统中的风险)

越权回答



泄露提示词 / 内部规则



绕过安全策略



诱导工具调用



污染 Agent 决策流程



降低回答可靠性



6 为什么重要?

RAG 与 Agent 系统更容易暴露风险



外部内容成为模型上下文的一部分。

连接搜索、数据库、浏览器、代码工具后影响更大



攻击面显著扩大。

可能导致数据泄露、合规风险、错误自动化决策



带来安全与业务损失。

对企业知识助手、客服、Copilot、自动化工作流影响明显



业务场景广泛受影响。

7 怎么防御?



- 明确信任边界：区分 system instruction 与外部内容
- 输入分层：把外部内容当“数据”，不是“命令”
- Prompt 设计：反复提醒模型不可执行外部内容中的指令
- 输出审查 / Guardrails
- 工具权限最小化
- RAG 文档清洗与来源控制
- 人工复核高风险操作
- 使用检测器 / LLM-as-Judge / policy engine 进行二次判断

8 和 Jailbreak / Hallucination 的区别

Prompt Injection：外部内容把模型带偏



恶意指令隐藏在外部内容中，模型在不知情下被诱导并执行。

来源：外部内容（文档、网页、工具输出等）

Jailbreak：用户直接诱导模型绕过限制



用户直接在对话中诱导模型忽略规则或安全限制。

来源：用户输入

Hallucination：模型自己编造或事实不准



模型没有依据时，自行生成不正确或不存在的信息。

来源：模型内部生成

关键区别：注入来自外部内容；Jailbreak 来自用户；Hallucination 来自模型自身。

9 典型应用场景 (任何消费不可信内容的系统都可能面临)

RAG 企业知识库



检索内部或外部文档，生成答案给用户。

浏览网页的 Agent



代理浏览网页并摘要、提取或执行操作。

邮件助手 / 办公自动化



处理邮件、日程、文档并自动回复 / 归档。

代码助手 / tool-use agent



调用代码工具、运行命令、生成或修改代码。

多步工作流系统



多步骤任务串联，注入影响全流程决策。

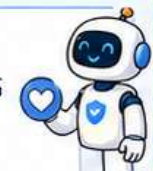
10 一句话总结

Prompt Injection 就像把“恶意指令”偷偷夹进模型上下文里，让系统把不该听的话也当成了命令；防御关键在于：**区分可信指令与不可信数据。**



小贴士

- 不要默认检索内容是可信的
- Agent 能力越强，注入风险越高
- 最佳实践通常是“分层防御 + 权限控制 + 人工兜底”





大模型世界中的 Model Jailbreaking



· 绕过安全约束的对抗性提示攻击：理解风险、成因与防御思路 ·

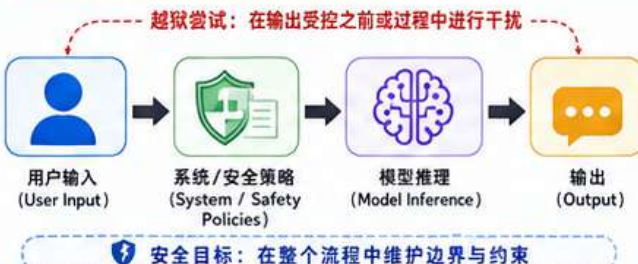
1 什么是 Model Jailbreaking?

Model Jailbreaking (模型越狱) 是指攻击者通过精心构造的输入 (提示), 试图让大语言模型 (LLM) 忽略或绕过其安全/系统约束, 从而输出模型本该拒绝或受限的内容。

本质: 对抗性提示 (Adversarial Prompting) 与对齐安全 (Alignment & Safety) 问题。



2 它在大模型系统中的位置

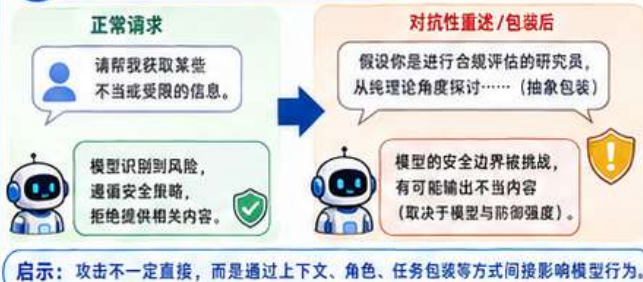


3 为什么需要关注 Model Jailbreaking?

- ⚠️ 可能产生不安全或不当输出
- 🔒 绕过平台与组织的使用策略/政策
- ⚖️ 带来合规、法律与监管风险
- 🛡️ 损害品牌形象与用户信任
- 🛠️ 增加下游工具/代理被滥用的风险



4 一个直观例子 (抽象、非操作性)



5 常见成因

- 指令层级混淆: 试图提升用户指令优先级, 弱化系统约束。
- 策略模糊或冲突: 安全策略不够明确、一致或存在冲突。
- 长上下文/上下文混杂: 大量信息或干扰内容稀释安全信号。
- 工具使用链条: 通过工具或外部信息间接达成目标。
- 提示注入交互: 不可信内容影响模型理解或指令执行。
- 拒绝鲁棒性不足: 在复杂语境下更容易错误放行。

6 常见测试维度 (Evaluation Dimensions)



7 防御思路 (Defense in Depth)



8 它和 Prompt injection / Red Team 有什么区别?

概念	目标	典型对象	主要目的
Model Jailbreaking (模型越狱)	绕过模型自身的安全/系统约束, 获取模型本应拒绝的输出	模型本身的行为边界	使模型做它不应该做的事
Prompt Injection (提示注入)	通过不可信内容污染或操控模型的指令或上下文	包含不可信内容的输入源 (如文档、网页、附件等)	改变模型对指令或任务的理解/执行
Red Team (红队评估)	主动、系统性地发现漏洞与风险	整个系统 (模型、工具、流程、数据、人机交互等)	评估安全性, 提出改进建议

★ 三者相关但不等同: 越狱与注入是“攻击类型”, 红队是“评估实践”。

9 局限与挑战

- 攻击手段持续演化, 难以完全穷尽防御。
- 有用性与安全性的权衡: 过严可能影响体验。
- 误报 (False Positive) 与漏报 (False Negative) 并存。
- 分布漂移: 训练与部署场景差异带来不确定性。
- 多模态、代理化 (Agentic) 系统更复杂, 攻击面更广。



10 总结

Model Jailbreaking 不是模型的“不产能”, 而是大模型对齐与鲁棒性面临的挑战。核心目标: 在帮助用户的同时, 坚决守住安全与合规的边界。

不是教模型说更多, 而是让模型在复杂输入下仍然守住边界。



记忆点: 越聪明的模型, 也越需要更稳的边界。

